

**ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN**



ĐỒ ÁN TỐT NGHIỆP

NGÀNH: CÔNG NGHỆ THÔNG TIN

**CHUYÊN NGÀNH: KHOA HỌC DỮ LIỆU VÀ TRÍ TUỆ
NHÂN TẠO**

ĐỀ TÀI:

**XÂY DỰNG HỆ THỐNG HỖ TRỢ TỔ CHỨC,
CHẤM THI VÀ ÔN LUYỆN CHO OLYMPIC TRÍ
TUỆ NHÂN TẠO SINH VIÊN VIỆT NAM**

Người hướng dẫn: **TS. PHẠM MINH TUẤN**
Sinh viên thực hiện: **NGUYỄN VĂN THƯỜNG**
Số thẻ sinh viên: **102220042**
Lớp: **22T_KHDL**

Đà Nẵng, 06/2026

**ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN**



ĐỒ ÁN TỐT NGHIỆP

NGÀNH: CÔNG NGHỆ THÔNG TIN

**CHUYÊN NGÀNH: KHOA HỌC DỮ LIỆU VÀ TRÍ TUỆ
NHÂN TẠO**

ĐỀ TÀI:

**XÂY DỰNG HỆ THỐNG HỖ TRỢ TỔ CHỨC,
CHẤM THI VÀ ÔN LUYỆN CHO OLYMPIC TRÍ
TUỆ NHÂN TẠO SINH VIÊN VIỆT NAM**

Người hướng dẫn: **TS. PHẠM MINH TUẤN**
Sinh viên thực hiện: **NGUYỄN VĂN THƯƠNG**
Số thẻ sinh viên: **102220042**
Lớp: **22T_KHDL**

Đà Nẵng, 06/2026

[illegible]

Người hướng dẫn

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

Người phản biện

NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP

1. Thông tin chung:

1. Họ và tên sinh viên: Nguyễn Văn Thương
2. Lớp: 22T_KHDL. Số thẻ SV: 102220042
3. Tên đề tài: Xây dựng hệ thống hỗ trợ tổ chức, chấm thi và ôn luyện cho Olympic Trí tuệ Nhân tạo Sinh viên Việt Nam
4. Người hướng dẫn: TS. Phạm Minh Tuấn Học hàm/ học vị: Tiến sĩ

II. Nhận xét, đánh giá đồ án tốt nghiệp:

1. Về tính cấp thiết, tính mới, khả năng ứng dụng của đề tài: (điểm tối đa là 2đ)
.....
.....
2. Về kết quả giải quyết các nội dung nhiệm vụ yêu cầu của đề án: (điểm tối đa là 4đ)
.....
.....
3. Về hình thức, cấu trúc, bố cục của đồ án tốt nghiệp: (điểm tối đa là 2đ)
.....
.....
4. Đề tài có giá trị khoa học/ có bài báo/ giải quyết vấn đề đặt ra của doanh nghiệp hoặc nhà trường: (điểm tối đa là 1đ)
.....
.....
5. Các tồn tại, thiếu sót cần bổ sung, chỉnh sửa:
.....
.....

III. Tinh thần, thái độ làm việc của sinh viên: (điểm tối đa 1đ)
.....

IV. Đánh giá:

1. Điểm đánh giá:/10 (lấy đến 1 số lẻ thập phân)
2. Đề nghị: ☐ Được bảo vệ đồ án ☐ Bổ sung để bảo vệ ☐ Không được bảo vệ

Đà Nẵng, ngày tháng năm 202

Người hướng dẫn

TÓM TẮT

Tên đề tài: Xây dựng hệ thống hỗ trợ tổ chức, chấm thi và ôn luyện cho Olympic Trí tuệ Nhân tạo Sinh viên Việt Nam

Sinh viên thực hiện: Nguyễn Văn Thương

Số thẻ SV: 102220042

Lớp: 22T_KHDL

Sự phát triển của các cuộc thi Trí tuệ nhân tạo (AI) trong môi trường đại học đã làm gia tăng nhu cầu về các nền tảng hỗ trợ tổ chức, đánh giá và quản lý cuộc thi một cách hiệu quả. Khác với các cuộc thi lập trình truyền thống, cuộc thi AI thường bao gồm nhiều giai đoạn đánh giá, nhiều dạng bài nộp khác nhau như tệp kết quả, mô hình huấn luyện hoặc chương trình suy luận, đồng thời yêu cầu các cơ chế chấm điểm dựa trên những độ đo chuyên biệt. Điều này đặt ra nhiều thách thức trong việc quản lý bài nộp, phân phối tài nguyên tính toán và cập nhật kết quả theo thời gian thực.

Mặc dù hiện nay đã có nhiều nền tảng phục vụ chấm bài trực tuyến, phần lớn được thiết kế cho các bài toán lập trình truyền thống hoặc các cuộc thi AI quy mô lớn trên nền tảng thương mại. Các hệ thống này chưa thực sự phù hợp với nhu cầu tổ chức và ôn luyện cho các kỳ thi học thuật trong nước, đặc biệt là Olympic Trí tuệ Nhân tạo Sinh viên Việt Nam, nơi yêu cầu khả năng tự chủ hạ tầng, linh hoạt trong quy trình đánh giá và tận dụng hiệu quả nguồn tài nguyên sẵn có.

Xuất phát từ thực tiễn đó, đồ án tập trung nghiên cứu, thiết kế và xây dựng hệ thống BKDNAIOJ nhằm hỗ trợ tổ chức, chấm thi và ôn luyện cho Olympic Trí tuệ Nhân tạo Sinh viên Việt Nam. Hệ thống được xây dựng theo kiến trúc phân tán, kết hợp backend điều phối trung tâm với mạng lưới máy chấm tình nguyện để thực hiện chấm bài trên các máy tính có cấu hình phần cứng khác nhau. Bên cạnh đó, hệ thống áp dụng cơ chế nộp bài hai bước thông qua Presigned URL và MinIO Object Storage nhằm hỗ trợ xử lý các tệp dữ liệu lớn, đồng thời sử dụng Redis Streams để quản lý và phân phối tác vụ chấm bài.

Ngoài các chức năng quản lý cuộc thi và chấm điểm tự động, BKDNAIOJ còn hỗ trợ nhiều hình thức đánh giá khác nhau, cơ chế giám sát worker, tự động khôi phục các tác vụ bị gián đoạn và cập nhật bảng xếp hạng nhanh chóng. Các thực nghiệm được tiến hành trên môi trường triển khai thực tế nhằm đánh giá tính đúng đắn, hiệu năng và khả năng mở rộng của hệ thống. Kết quả đạt được cho thấy nền tảng có khả năng đáp ứng tốt yêu cầu vận hành của các cuộc thi AI quy mô vừa, đồng thời tạo nền tảng cho việc mở rộng và phát triển trong tương lai.

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ tên sinh viên: Nguyễn Văn Thương Số thẻ sinh viên: 102220042

Lớp: 22T_KHDL

Khoa: Công Nghệ Thông Tin

Ngành: Khoa học dữ liệu và Trí tuệ nhân tạo

1. *Tên đề tài đồ án:*

Xây dựng hệ thống hỗ trợ tổ chức, chấm thi và ôn luyện cho Olympic Trí tuệ Nhân tạo Sinh viên Việt Nam

2. *Đề tài thuộc diện:* ☐ Có ký kết thỏa thuận sở hữu trí tuệ đối với kết quả thực hiện

3. *Các số liệu và dữ liệu ban đầu:*

Không có các số liệu và dữ liệu ban đầu

4. *Nội dung các phần thuyết minh và tính toán:*

- **Mở đầu:** Trình bày bối cảnh nghiên cứu, lý do chọn đề tài, mục tiêu, đối tượng, phạm vi và phương pháp thực hiện.
- **Chương 1 - Tổng quan:** Phân tích hạn chế của các hệ thống liên quan, xác định đóng góp chính của đề tài và mô tả kiến trúc tổng thể của BKDNAIOJ.
- **Chương 2 - Cơ sở lý thuyết:** Trình bày các khái niệm nền tảng: mô hình cuộc thi bốn giai đoạn, hàng đợi phân tán với Redis Streams, cơ chế presigned URL, mô hình tính toán tình nguyện và bảng xếp hạng được tính sẵn.
- **Chương 3 - Phân tích và Thiết kế hệ thống:** Phân tích yêu cầu theo các tác nhân, đặc tả use case, thiết kế cơ sở dữ liệu, kiến trúc backend, hệ thống máy chấm tình nguyện, đặc tả API và giao diện người dùng.
- **Chương 4 - Cài đặt và Kiểm thử:** Chi tiết cài đặt các thành phần hệ thống, kết quả kiểm thử chức năng với 24 trường hợp kiểm thử và quy trình triển khai lên môi trường thực tế.
- **Chương 5 - Thực nghiệm và Đánh giá:** Đánh giá hệ thống trên môi trường triển hai thực tế với nhóm 5 máy chấm không đồng nhất. Nội dung đánh giá bao gồm hiệu năng xử lý khi nộp nhiều bài cùng lúc, khả năng phân phối tác vụ giữa các máy xử lý khi nộp nhiều bài cùng lúc, khả năng phân phối tác vụ giữa các máy xử lý, tải truy vấn API và ước lượng khả năng đáp ứng tổng thể của hệ thống.
- **Kết luận và hướng phát triển:** Tổng kết kết quả đạt được, trình bày hạn chế của hệ thống và đề xuất hướng nghiên cứu tiếp theo.

5. *Các bản vẽ, đồ thị (ghi rõ các loại và kích thước bản vẽ):*

Không có bản vẽ, đồ thị.

6. *Họ tên người hướng dẫn:* TS. Phạm Minh Tuấn

7. *Ngày giao nhiệm vụ đồ án:*/...../202.....

8. *Ngày hoàn thành đồ án:*/...../202.....

Đà Nẵng, ngày tháng năm 202

Trưởng Bộ môn

Người hướng dẫn

LỜI NÓI ĐẦU

Đồ án tốt nghiệp này là kết quả của quá trình học tập, rèn luyện và nghiên cứu trong suốt những năm học tại Khoa Công nghệ Thông tin, Trường Đại học Bách khoa - Đại học Đà Nẵng. Trong quá trình thực hiện đề tài, tôi đã nhận được rất nhiều sự quan tâm, hỗ trợ và động viên quý báu từ thầy cô, gia đình, bạn bè và những người đồng hành. Tôi xin được bày tỏ lòng biết ơn chân thành nhất đến tất cả mọi người.

Trước hết, tôi xin gửi lời cảm ơn sâu sắc đến TS. Phạm Minh Tuấn, giảng viên hướng dẫn trực tiếp của đồ án, cùng với đó là TS. Nguyễn Năng Hùng Vân, hai thầy đã luôn quan tâm, dành thời gian trao đổi và hỗ trợ tôi trong suốt quá trình thực hiện đề tài. Những góp ý của các thầy về kiến trúc hệ thống, cơ chế chấm bài phân tán và phương pháp đánh giá thực nghiệm đã giúp tôi nhìn nhận vấn đề một cách toàn diện hơn, đồng thời tránh được nhiều hạn chế trong quá trình thiết kế và triển khai hệ thống.

Tôi xin chân thành cảm ơn Ban Chủ nhiệm Khoa Công nghệ Thông tin cùng toàn thể quý thầy cô đã giảng dạy và truyền đạt cho tôi những kiến thức quý báu trong suốt quá trình học tập tại trường. Những kiến thức nền tảng về lập trình là hành trang quan trọng giúp tôi có đủ nền tảng để thực hiện và hoàn thành đồ án này.

Tôi cũng xin gửi lời cảm ơn đến các anh, chị và các bạn đã hỗ trợ thử nghiệm hệ thống, đóng góp ý kiến trong quá trình phát triển, đồng thời tham gia xây dựng mạng lưới máy chấm tình nguyện phục vụ cho các thực nghiệm của đề tài. Sự hỗ trợ nhiệt tình của mọi người đã giúp tôi có cơ hội đánh giá hệ thống trên môi trường gần với điều kiện vận hành thực tế, từ đó nâng cao giá trị và tính ứng dụng của các kết quả đạt được.

Bên cạnh đó, tôi cũng xin ghi nhận sự hỗ trợ của các công cụ Trí tuệ Nhân tạo trong quá trình thực hiện đồ án. Việc ứng dụng các trợ lý AI vào việc tra cứu tài liệu, chỉnh sửa văn phong học thuật và hỗ trợ tối ưu hóa luồng xử lý đã giúp tôi tiết kiệm thời gian và nâng cao hiệu suất làm việc một cách đáng kể.

Mặc dù đã nỗ lực hết sức trong quá trình thực hiện, do hạn chế về thời gian, kinh nghiệm và nguồn lực, đồ án khó tránh khỏi những thiếu sót nhất định. Tôi rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô và Hội đồng để có thể tiếp tục hoàn thiện hơn trong các nghiên cứu và công việc sau này.

Xin trân trọng cảm ơn!

CAM ĐOAN

Tôi xin cam đoan:

1. Toàn bộ nội dung được trình bày trong đồ án tốt nghiệp này là kết quả nghiên cứu, tìm hiểu và thực hiện của cá nhân tôi dưới sự hướng dẫn trực tiếp của TS. Phạm Minh Tuấn.
2. Các tài liệu, công trình nghiên cứu, số liệu và nguồn tham khảo được sử dụng trong đồ án đều được trích dẫn đầy đủ, rõ ràng và tuân thủ các quy định về đạo đức học thuật.
3. Đồ án không sao chép trái phép nội dung từ bất kỳ công trình nào khác. Các nội dung kế thừa, tham khảo hoặc trích dẫn đều được ghi rõ nguồn gốc theo đúng quy định.
4. Nếu có bất kỳ sai phạm nào liên quan đến bản quyền, trích dẫn hoặc tính trung thực học thuật, tôi xin hoàn toàn chịu trách nhiệm trước Hội đồng và các quy định của Nhà trường.

Sinh viên thực hiện

LỜI NÓI ĐẦU	i
CAM ĐOAN	ii
DANH MỤC BẢNG BIỂU	viii
DANH MỤC HÌNH ẢNH	ix
DANH SÁCH CHỮ VIẾT TẮT	x
MỞ ĐẦU	1
CHƯƠNG 1: TỔNG QUAN	4
1.1. Bối cảnh và động lực	4
1.1.1 Cuộc thi AI trong giáo dục và nghiên cứu	4
1.1.2 Hạn chế của các nền tảng hiện có	5
1.2. Các hệ thống liên quan	7
1.2.1 Nền tảng đánh giá trực tuyến (Online Judge) truyền thống	7
1.2.2 Các nền tảng thi đấu Học máy và Khoa học Dữ liệu	8
1.2.3 Hệ thống phân tán và lập lịch tác vụ	8
1.3. Đóng góp của đề tài	9
1.3.1 Yêu cầu đặc thù của cuộc thi AI - Mô hình 4-phase	9
1.3.2 Đóng góp chính của đề tài.....	10
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	13
2.1. Mô hình chấm bài trong hệ thống Online Judge	13
2.1.1 Online Judge truyền thống	13
2.1.2 Output-only Judging.....	13
2.1.3 Inference-based Judging.....	14
2.1.4 Mô hình đa phase trong cuộc thi AI	14
2.2. Hàng đợi tin nhắn và Redis Streams	15
2.2.1 Mô hình xử lý bất đồng bộ với hàng đợi tin nhắn.....	15
2.2.2 Redis Streams	15
2.3. Lưu trữ đối tượng và Presigned URL	17
2.3.1 Object Storage và giao thức S3-compatible.....	17
2.3.2 Presigned URL	17

2.4. Tính toán phân tán và lập lịch trên worker không đồng nhất	18
2.4.1 Volunteer Computing	18
2.4.2 Bài toán lập lịch trên worker không đồng nhất	19
2.4.3 Cơ chế Claim Lease và Heartbeat	19
2.5. Containerization và cô lập môi trường thực thi.....	20
2.5.1 Docker và cơ chế namespace/cgroup	20
2.5.2 Sandbox và bảo mật môi trường thực thi	21
2.6. Bảng xếp hạng và Materialized View	22
2.6.1 Materialized View trong cơ sở dữ liệu quan hệ	22
2.6.2 Window Function và xếp hạng trong SQL.....	22
2.7. Công nghệ và nền tảng phát triển	23
2.7.1 Ngôn ngữ Go và mô hình đồng thời Goroutine	23
2.7.2 PostgreSQL, JSONB và Window Function	23
2.7.3 React và mô hình Single Page Application	23
2.7.4 Python và hệ sinh thái Machine Learning.....	24
2.7.5 Tổng hợp lựa chọn công nghệ.....	24
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	26
3.1. Phân tích yêu cầu	26
3.1.1 Các vai trò người dùng	26
3.1.2 Yêu cầu chức năng đối với Admin.....	27
3.1.3 Yêu cầu chức năng đối với Contestant.....	28
3.1.4 Yêu cầu phi chức năng	28
3.1.5 Đặc tả Use Case.....	29
3.2. Sơ đồ kiến trúc tổng thể.....	33
3.3. Thiết kế Cơ sở Dữ liệu	35
3.3.1 ER Diagram tổng thể và nguyên tắc thiết kế.....	35
3.3.2 Các nhóm bảng theo domain nghiệp vụ	36
3.3.3 Các quyết định thiết kế chính.....	39
3.4. Thiết kế kiến trúc Backend	40

3.4.1	Phân lớp ứng dụng	40
3.4.2	Judging Contract	42
3.4.3	Redis Streams và cơ chế phân phối công việc	43
3.4.4	Leaderboard Materialized	44
3.4.5	Luồng nộp bài và chấm điểm	47
3.4.6	Sơ đồ hoạt động	49
3.5.	Thiết kế Hệ thống Máy chấm tình nguyện	52
3.5.1	WorkerProfile và benchmark thực đo	52
3.5.2	Mô hình năng lực Worker	53
3.5.3	Quy trình đăng ký và tham gia Worker Pool	53
3.5.4	Thuật toán ClaimNext - Chi tiết thiết kế	54
3.5.5	Cơ chế cấp phát tác vụ nguyên tử	56
3.5.6	Cơ chế Lease, Heartbeat và Fault Recovery	56
3.5.7	Vòng đời của Submission	57
3.5.8	Artifact Cache	58
3.5.9	Sandbox và Cô lập Mã Không Tin Cây	59
3.6.	Thiết kế API	61
3.6.1	Nguyên tắc thiết kế	61
3.6.2	Các nhóm API chính	62
3.6.3	Đặc tả các API nghiệp vụ trọng yếu	63
3.7.	Thiết kế Giao diện Người dùng	65
3.7.1	Routing và cấu trúc trang	65
3.7.2	Quản lý trạng thái giao diện	66
3.7.3	Phase Hub - Màn hình trung tâm của thí sinh	67
3.8.	Thiết kế Bảo mật	67
3.8.1	Xác thực và phân quyền	67
3.8.2	Giới hạn tần suất truy cập	68
3.8.3	Bảo vệ dữ liệu cuộc thi	68
3.8.4	Bảo vệ môi trường thực thi	69

3.8.5	Kiến trúc bảo mật nhiều lớp	69
CHƯƠNG 4: CÀI ĐẶT VÀ KIỂM THỬ		70
4.1.	Môi trường và Công cụ Phát triển	70
4.1.1	Ngôn ngữ lập trình và framework	70
4.1.2	Công cụ hỗ trợ phát triển.....	70
4.1.3	Cấu trúc thư mục dự án	71
4.2.	Cài đặt Backend	71
4.2.1	Luồng xử lý nộp bài	71
4.2.2	Cơ chế phân phối job	72
4.2.3	Cập nhật bảng xếp hạng	73
4.2.4	Xử lý sự cố và khôi phục	74
4.3.	Cài đặt Volunteer Agent	75
4.3.1	Thiết lập và đăng ký worker.....	75
4.3.2	Vòng lặp nhận và xử lý job	76
4.3.3	Thực thi phase final.....	77
4.4.	Giao diện Người dùng	78
4.5.	Kiểm thử Hệ thống	81
4.5.1	Phương pháp kiểm thử	81
4.5.2	Kiểm thử chức năng	81
4.5.3	Kiểm thử tích hợp và hiệu năng	84
4.6.	Triển khai Production	84
4.6.1	Hạ tầng server và dịch vụ.....	84
4.6.2	Triển khai Máy chấm tình nguyện	85
CHƯƠNG 5: THỰC NGHIỆM VÀ ĐÁNH GIÁ		87
5.1.	Mục tiêu thực nghiệm	87
5.2.	Thiết lập thực nghiệm	88
5.2.1	Contest thực nghiệm.....	88
5.2.2	Cấu hình worker pool.....	88
5.2.3	Thiết kế workload thực nghiệm	91

5.2.4	Phương pháp thu thập và tính toán số liệu	91
5.3.	Kiểm thử chức năng đầu cuối	92
5.4.	Hiệu năng xử lý submission	93
5.4.1	Kết quả batch tăng tải.....	93
5.4.2	Phân tích độ trễ vòng đời submission	94
5.5.	Đánh giá workload tính toán tổng hợp	96
5.6.	Phân phối job và mức sử dụng worker	97
5.7.	Đánh giá tải đọc API	99
5.8.	Ước lượng sức chứa bằng phân tích độ nhảy	100
5.9.	Thảo luận	103
5.9.1	Trả lời RQ1 - Tính đúng đắn đầu cuối	103
5.9.2	Trả lời RQ2 - Hiệu quả khai thác worker pool không đồng nhất.....	103
5.9.3	Trả lời RQ3 - Sức chứa và khả năng mở rộng	104
5.10.	Hạn chế của thực nghiệm.....	104
5.11.	Tổng kết	105
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....		107
TÀI LIỆU THAM KHẢO.....		110

DANH MỤC BẢNG BIỂU

Bảng 1.1: So sánh các nền tảng thi AI.....	6
Bảng 1.2: Đóng góp chính của đề tài	10
Bảng 2.1: Công nghệ sử dụng và lý do lựa chọn.....	24
Bảng 3.1: Yêu cầu phi chức năng và giải pháp kiến trúc tương ứng.....	29
Bảng 3.2: Đặc tả UC-01 - Nộp bài	29
Bảng 3.3: Đặc tả UC-02 - Đăng ký tham gia cuộc thi.....	30
Bảng 3.4: Đặc tả UC-03 - Đăng ký máy chấm tình nguyện.....	31
Bảng 3.5: Đặc tả UC-04 - Phê duyệt máy chấm tình nguyện	31
Bảng 3.6: Đặc tả UC-05 - Nhận job và chấm bài.....	32
Bảng 3.7: Trạng thái submission	58
Bảng 3.8: Các cơ chế giới hạn trong Docker Sandbox	60
Bảng 3.9: Đặc tả API khởi tạo nộp bài	63
Bảng 3.10: Đặc tả API xác nhận hoàn tất tải tệp	64
Bảng 3.11: Đặc tả API nhận tác vụ chấm bài	64
Bảng 3.12: Đặc tả API gửi kết quả chấm bài	64
Bảng 3.13: Đặc tả API truy vấn bảng xếp hạng (Leaderboard)	65
Bảng 3.14: Danh sách 18 trang và route tương ứng	66
Bảng 4.1: Kết quả kiểm thử chức năng	82
Bảng 4.2: Cấu hình hạ tầng production	84
Bảng 5.1: Cấu hình phân cứng của worker pool	89
Bảng 5.2: Cấu hình năng lực thực thi của worker pool	89
Bảng 5.3: Snapshot benchmark của worker pool	90
Bảng 5.4: Kết quả pilot workload.....	93
Bảng 5.5: So sánh các batch workload MNIST	93
Bảng 5.6: Độ trễ vòng đời submission trong batch 500	94
Bảng 5.7: Kết quả workload tính toán tổng hợp.....	97
Bảng 5.8: Phân phối job theo worker trong batch 500	98
Bảng 5.9: Kết quả kiểm thử tải đọc API.....	99
Bảng 5.10: Ước lượng sức chứa với 5 worker, 500 submission, 50% final.....	101
Bảng 5.11: Ước lượng sức chứa với 5 worker, 1000 submission, 50% final.....	102

DANH MỤC HÌNH ẢNH

Hình 2.1: Mô hình đánh giá đa phase trong cuộc thi AI	14
Hình 2.2: Luồng xử lý bất đồng bộ với Redis Streams và Consumer Group.....	16
Hình 2.3: Luồng upload tệp sử dụng Presigned URL	18
Hình 2.4: Cơ chế Claim Lease và Heartbeat trong hệ thống phân tán	20
Hình 3.1: Sơ đồ use case tổng quan.....	26
Hình 3.2: Kiến trúc tổng thể hệ thống BKDNAIOJ	33
Hình 3.3: ER Diagram tổng thể rút gọn của hệ thống	35
Hình 3.4: ER Diagram nhóm người dùng, đội thi và tham gia cuộc thi	36
Hình 3.5: ER Diagram nhóm cuộc thi, bài toán, phase và tài nguyên đánh giá.....	37
Hình 3.6: ER Diagram nhóm tham gia và nộp bài	38
Hình 3.7: ER Diagram nhóm leaderboard.....	38
Hình 3.8: ER Diagram nhóm worker phân tán.....	39
Hình 3.9: Kiến trúc phân lớp của Backend Go.....	41
Hình 3.10: Ba tầng leaderboard trong BKDNAIOJ	46
Hình 3.11: Sequence diagram đầy đủ từ nộp bài đến leaderboard update	48
Hình 3.12: Sơ đồ hoạt động - Luồng nộp bài của thí sinh.....	50
Hình 3.13: Sơ đồ hoạt động - Luồng thực thi chấm bài của Worker Agent	51
Hình 3.14: Quy trình thiết lập và đăng ký máy chấm tình nguyện	52
Hình 3.15: Minh hoạ thuật toán ClaimNext	55
Hình 3.16: State machine của submission trong BKDNAIOJ	57
Hình 4.1: Sequence diagram luồng nộp bài sử dụng Presigned URL	72
Hình 4.2: Hai goroutine nền chạy song song với HTTP server	75
Hình 4.3: Luồng sáu bước của setup wizard	76
Hình 4.4: Hai chế độ thực thi của phase final: Docker Sandbox và Trusted Native.	78
Hình 4.5: Phase Hub - tổng quan phase	79
Hình 4.6: Admin Workers - danh sách worker	79
Hình 4.7: Scheduler Dashboard - queue depth và phân bố trạng thái submission	80
Hình 4.8: Global Rankings - bảng xếp hạng tổng hợp với score breakdown	80
Hình 4.9: Admin Setup - wizard tạo cuộc thi nhiều bước	81
Hình 5.1: Độ trễ vòng đời submission trong batch 500	96
Hình 5.2: Ước lượng sức chứa theo hệ số thời gian final inference.....	102

DANH SÁCH CHỮ VIẾT TẮT

Viết tắt	Viết đầy đủ	Ý nghĩa
ACK	Acknowledgement	Xác nhận đã xử lý trong Redis Streams
ACME	Automatic Certificate Management Environment	Giao thức tự động cấp và gia hạn chứng chỉ TLS
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
BLEU	Bilingual Evaluation Understudy	Độ đo đánh giá chất lượng sinh văn bản hoặc dịch máy
BOINC	Berkeley Open Infrastructure for Network Computing	Hạ tầng tính toán phân tán mã nguồn mở
CLI	Command-Line Interface	Giao diện dòng lệnh
CPU	Central Processing Unit	Bộ xử lý trung tâm
CSV	Comma-Separated Values	Định dạng dữ liệu bảng phân tách bằng dấu phẩy
CTE	Common Table Expression	Biểu thức bảng chung trong SQL
CUDA	Compute Unified Device Architecture	Nền tảng tính toán song song trên GPU NVIDIA
DB	Database	Cơ sở dữ liệu
FIFO	First In, First Out	Vào trước, ra trước
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
HTTPS	HTTP Secure	HTTP có mã hóa bằng TLS
ICPC	International Collegiate Programming Contest	Cuộc thi lập trình sinh viên quốc tế
I/O	Input/Output	Hoạt động nhập/xuất dữ liệu

JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu nhẹ
JSONB	JSON Binary	Kiểu dữ liệu JSON nhị phân của PostgreSQL
JWT	JSON Web Token	Token xác thực dạng JSON có chữ ký số
K6	k6 Load Testing Tool	Công cụ kiểm thử tải hệ thống
ML	Machine Learning	Học máy
NVMe	Non-Volatile Memory Express	Chuẩn giao tiếp tốc độ cao cho ổ lưu trữ SSD
OJ	Online Judge	Hệ thống chấm bài trực tuyến
OLPAI	Olympic AI	Olympic Trí tuệ Nhân tạo (Sinh viên Việt Nam)
ORM	Object-Relational Mapping	Kỹ thuật ánh xạ đối tượng sang cơ sở dữ liệu quan hệ
P95	95th Percentile	Phân vị thứ 95
P99	99th Percentile	Phân vị thứ 99
PDF	Portable Document Format	Định dạng tài liệu di động
PNG	Portable Network Graphics	Định dạng ảnh nén không mất dữ liệu
PyPI	Python Package Index	Kho phân phối package Python công khai
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
REST	Representational State Transfer	Phong cách thiết kế API dựa trên tài nguyên
RQ	Research Question	Câu hỏi nghiên cứu hoặc câu hỏi đánh giá
S3	Simple Storage Service	Giao thức/lớp API lưu trữ object tương thích Amazon S3
SHA256	Secure Hash Algorithm 256-bit	Thuật toán băm mật mã 256-bit

SPA	Single Page Application	Ứng dụng web một trang
SQL	Structured Query Language	Ngôn ngữ truy vấn cơ sở dữ liệu quan hệ
SSD	Solid-State Drive	Ổ lưu trữ thể rắn
SSE	Server-Sent Events	Cơ chế đẩy sự kiện một chiều từ server đến client qua HTTP
TFLOPS	Tera Floating-Point Operations Per Second	Nghìn tỷ phép tính dấu phẩy động mỗi giây
TLS	Transport Layer Security	Giao thức bảo mật tầng truyền tải
TTL	Time To Live	Thời gian hiệu lực
UI	User Interface	Giao diện người dùng
URL	Uniform Resource Locator	Địa chỉ định vị tài nguyên
UUID	Universally Unique Identifier	Định danh duy nhất toàn cục
VRAM	Video Random Access Memory	Bộ nhớ video của GPU
VU	Virtual User	Người dùng ảo trong kiểm thử tải

MỞ ĐẦU

1. Tổng quan đề tài

Trong những năm gần đây, các cuộc thi về Trí tuệ Nhân tạo dành cho sinh viên phát triển nhanh cả về số lượng lẫn độ phức tạp. Khác với các kỳ thi lập trình truyền thống, một cuộc thi AI thường bao gồm nhiều tác vụ khác nhau, nhiều giai đoạn nộp bài, nhiều dạng hiện vật nộp lên hệ thống như file dự đoán, file ảnh, file zip, mã nguồn, checkpoint mô hình hoặc script suy luận. Việc chấm thi vì thế không chỉ dừng ở so sánh output đơn giản mà còn cần hỗ trợ nhiều độ đo khác nhau như Accuracy, F1-score, BLEU, Cosine Similarity, MAPE, PSNR hoặc các hàm đánh giá tổng hợp được ban tổ chức định nghĩa riêng. Bên cạnh đó, trong quá trình tổ chức thi thật, hệ thống không chỉ cần tiếp nhận bài nộp và trả kết quả, mà còn phải hỗ trợ toàn bộ luồng vận hành cuộc thi như quản lý đội thi, quản lý nhiều phase, giới hạn số lần nộp, hiển thị leaderboard public/private, gửi thông báo, trả lời clarification, xử lý khiếu nại kỹ thuật, chấm lại bài nộp và hậu kiểm kết quả ở các vòng cuối. Nếu thiếu các chức năng này, việc tổ chức thi sẽ dễ rời rạc, khó bảo đảm tính công bằng và khó sử dụng lâu dài trong môi trường trường đại học. Olympic Trí tuệ Nhân tạo Sinh viên Việt Nam là một ví dụ điển hình cho dạng cuộc thi AI nhiều tác vụ, nhiều phase và nhiều luật chấm. Tuy nhiên, hiện nay các hệ thống hỗ trợ cho loại cuộc thi này ở môi trường đại học còn chưa thật sự chuyên biệt; nhiều nền tảng chỉ phù hợp với online judge cho bài toán lập trình truyền thống hoặc chỉ hỗ trợ một loại workflow AI đơn giản. Do đó, việc xây dựng một nền tảng web chuyên biệt phục vụ tổ chức, chấm thi và ôn luyện cho các cuộc thi AI sinh viên là cần thiết, có giá trị thực tiễn cao và có thể phát triển thành một sản phẩm dùng lâu dài cho nhà trường. Đề tài được lựa chọn vì vừa đáp ứng nhu cầu thực tế trong đào tạo và tổ chức thi AI, vừa có chiều sâu kỹ thuật về phát triển web, hệ thống chấm thi tự động, quản lý bài nộp, thiết kế bảng xếp hạng, xử lý hàng đợi, bảo mật, giám sát hệ thống và khả năng mở rộng cho các loại bài toán AI khác nhau.

2. Mục đích thực hiện đề tài

Mục đích của đề tài là nghiên cứu, thiết kế và xây dựng một nền tảng phục vụ tổ chức cuộc thi AI có khả năng tự vận hành, hỗ trợ đầy đủ các nghiệp vụ chính của một cuộc thi, bao gồm quản lý cuộc thi, tiếp nhận bài nộp, chấm điểm tự động và công bố kết quả.

Thông qua đề tài, hệ thống hướng tới việc giải quyết các vấn đề kỹ thuật quan trọng trong các cuộc thi AI hiện nay như phân phối công việc trên tập hợp worker có cấu hình không đồng nhất, đảm bảo tính toàn vẹn của quá trình chấm bài khi worker gặp

sự cố, cũng như duy trì khả năng cập nhật bảng xếp hạng với độ trễ thấp trong điều kiện có nhiều bài nộp đồng thời.

3. Mục tiêu đề tài

Đề tài hướng tới các mục tiêu cụ thể sau:

- Thiết kế và xây dựng hệ thống backend phục vụ quản lý cuộc thi, tiếp nhận bài nộp và điều phối quá trình chấm bài tự động.
- Xây dựng volunteer agent cho phép các máy tính tình nguyện tham gia thực hiện chấm bài và suy luận mô hình.
- Hỗ trợ đồng thời hai hình thức đánh giá: nộp kết quả trực tiếp (output-only) và nộp chương trình suy luận (inference-based).
- Xây dựng cơ chế bảng xếp hạng cập nhật tự động và hỗ trợ nhiều giai đoạn đánh giá.
- Triển khai hệ thống trên môi trường thực tế và đánh giá hiệu năng, khả năng mở rộng cũng như tính ổn định của hệ thống.

4. Đối tượng và phạm vi đề tài

- **Đối tượng:** Đối tượng nghiên cứu của đề tài là nền tảng phần mềm hỗ trợ tổ chức các cuộc thi Trí tuệ nhân tạo với cơ chế chấm bài phân tán. Nội dung nghiên cứu tập trung vào kiến trúc hệ thống, cơ chế quản lý bài nộp, phân phối công việc trên máy chấm tình nguyện và duy trì bảng xếp hạng trong môi trường có nhiều người dùng đồng thời.
- **Phạm vi:** Hệ thống được xây dựng nhằm phục vụ các cuộc thi AI theo mô hình nhiều giai đoạn, hỗ trợ cả hình thức đánh giá dựa trên kết quả đầu ra và hình thức đánh giá thông qua quá trình suy luận mô hình. Đề tài tập trung vào việc thiết kế và triển khai nền tảng tổ chức cuộc thi, không đi sâu nghiên cứu các thuật toán AI cụ thể của từng bài toán. Đồng thời, đề tài không đề cập đến việc xây dựng bộ dữ liệu thi, thiết kế bài toán chuyên ngành hay phát triển các phương pháp học máy mới.

5. Phương pháp nghiên cứu

Đề tài được thực hiện theo các bước chính sau:

- Khảo sát các nền tảng tổ chức thi và đánh giá AI hiện có, từ đó xác định các yêu cầu và hạn chế cần giải quyết.
- Phân tích yêu cầu hệ thống và thiết kế kiến trúc tổng thể, cơ sở dữ liệu, giao diện và giao thức trao đổi giữa các thành phần.

- Triển khai hệ thống bằng các công nghệ phù hợp cho backend, frontend và volunteer agent.
- Tiến hành kiểm thử chức năng và thực nghiệm trên môi trường triển khai thực tế.
- Phân tích kết quả thực nghiệm để đánh giá tính đúng đắn, hiệu năng và khả năng mở rộng của hệ thống.

6. Cấu trúc đồ án tốt nghiệp

Đồ án được triển khai theo các chương mục chính như sau:

- Mở đầu: Trình bày bối cảnh nghiên cứu, lý do chọn đề tài, mục tiêu, đối tượng, phạm vi và phương pháp thực hiện.
- Chương 1 - Tổng quan: Phân tích hạn chế của các hệ thống liên quan, xác định đóng góp chính của đề tài và mô tả kiến trúc tổng thể của BKDNAIOJ.
- Chương 2 - Cơ sở lý thuyết: Trình bày các khái niệm nền tảng: mô hình cuộc thi bốn giai đoạn, hàng đợi phân tán với Redis Streams, cơ chế presigned URL, mô hình tính toán tình nguyện và bảng xếp hạng được tính sẵn.
- Chương 3 - Phân tích và Thiết kế hệ thống: Phân tích yêu cầu theo các tác nhân, đặc tả use case, thiết kế cơ sở dữ liệu, kiến trúc backend, hệ thống máy chấm tình nguyện, đặc tả API và giao diện người dùng.
- Chương 4 - Cài đặt và Kiểm thử: Chi tiết cài đặt các thành phần hệ thống, kết quả kiểm thử chức năng với 24 trường hợp kiểm thử và quy trình triển khai lên môi trường thực tế.
- Chương 5 - Thực nghiệm và Đánh giá: Đánh giá hệ thống trên môi trường triển khai thực tế với nhóm 5 máy chấm không đồng nhất. Nội dung đánh giá bao gồm hiệu năng xử lý khi nộp nhiều bài cùng lúc, khả năng phân phối tác vụ giữa các máy xử lý khi nộp nhiều bài cùng lúc, khả năng phân phối tác vụ giữa các máy xử lý, tải truy vấn API và ước lượng khả năng đáp ứng tổng thể của hệ thống.
- Kết luận và hướng phát triển: Tổng kết kết quả đạt được, trình bày hạn chế của hệ thống và đề xuất hướng nghiên cứu tiếp theo.

CHƯƠNG 1: TỔNG QUAN

1.1. Bối cảnh và động lực

1.1.1 Cuộc thi AI trong giáo dục và nghiên cứu

Trong thập kỷ qua, các cuộc thi trí tuệ nhân tạo và học máy đã khẳng định vai trò quan trọng trong đào tạo và đánh giá năng lực kỹ thuật tại các trường đại học và hội nghị khoa học lớn trên thế giới. Khác với bài kiểm tra lý thuyết hay dự án cá nhân, cuộc thi AI đặt người tham gia vào một bài toán thực tế với bộ dữ liệu chuẩn, ràng buộc thời gian và hệ thống đánh giá minh bạch - từ đó rèn luyện khả năng giải quyết vấn đề trong điều kiện cạnh tranh và thúc đẩy đổi mới sáng tạo.

Ở cấp độ quốc tế, các cuộc thi như NeurIPS Competition Track [1], Kaggle Competitions [2] và chuỗi thử thách ChaLearn [3] thu hút cộng đồng nghiên cứu rộng lớn, đồng thời cung cấp các benchmark chuẩn cho nhiều bài toán học máy quan trọng. Tại Việt Nam, các cuộc thi như VINIF AI Competition và AI4VN bắt đầu xuất hiện từ năm 2020, phản ánh nhu cầu ngày càng tăng trong việc đánh giá và phát triển nhân lực AI trong nước. Đặc biệt, sự ra đời của kỳ thi Olympic Trí tuệ Nhân tạo Sinh viên Việt Nam (OlpAI) lần thứ nhất vào tháng 12 năm 2025 - tổ chức song song cùng kỳ thi Olympic Tin học Sinh viên Việt Nam (OLP) và kỳ thi Lập trình sinh viên quốc tế ICPC - đã đánh dấu một bước ngoặt quan trọng. Không giống như OLP hay ICPC truyền thống (chỉ cần nộp mã nguồn C++/Python để chấm test case tĩnh), OlpAI đặt ra một tiêu chuẩn mới vô cùng khắt khe: các đội thi phải thực hiện toàn bộ quy trình AI liên tục trong khoảng thời gian giới hạn, từ xử lý dữ liệu, thiết kế mô hình đến lập trình trên môi trường tính toán có GPU. Thí sinh không chỉ giải quyết các bài toán về Thị giác máy tính hay Xử lý ngôn ngữ tự nhiên mà còn phải đảm bảo mô hình hoạt động thực tế trên tập dữ liệu ẩn của Ban tổ chức.

Ở cấp độ học thuật nội bộ, nhiều trường đại học kỹ thuật bắt đầu tổ chức các cuộc thi AI như một phương pháp đánh giá thay thế hoặc bổ sung cho thi cử truyền thống. Các cuộc thi này (bao gồm cả việc huấn luyện đội tuyển tham dự OlpAI) có đặc điểm riêng so với nền tảng quốc tế: quy mô nhỏ hơn (thường 20-200 đội), yêu cầu tùy chỉnh bài toán cao, ràng buộc bảo mật dữ liệu riêng của tổ chức và không thể phụ thuộc vào dịch vụ cloud thương mại có chi phí cao hay điều khoản sử dụng hạn chế. Bên cạnh đó, nhiều đơn vị sẵn có tài nguyên GPU trong phòng lab hoặc workstation của giảng viên - nguồn tài nguyên này hoàn toàn có thể tận dụng cho việc chấm bài nếu có hệ thống phân phối phù hợp.

Từ thực tế trên, nhu cầu về một nền tảng tổ chức cuộc thi AI có thể tự vận hành, hỗ trợ đầy đủ vòng đời từ đăng ký đến chấm điểm và xếp hạng, đồng thời tận dụng được

tài nguyên tính toán sẵn có của tổ chức - ngày càng trở nên cấp thiết trong môi trường học thuật.

1.1.2 Hạn chế của các nền tảng hiện có

Hiện nay, trên thế giới đã có nhiều nền tảng trực tuyến phục vụ các kỳ thi lập trình và trí tuệ nhân tạo. Tuy nhiên, khi đối chiếu với yêu cầu đặc thù của một kỳ thi AI quy mô nội bộ tại các cơ sở giáo dục - nơi đòi hỏi tính tự chủ về hạ tầng và khả năng tùy biến cao - các hệ thống này vẫn bộc lộ những hạn chế nhất định.

Kaggle [2] dù được biết đến là một trong những nền tảng thi đấu học máy lớn nhất thế giới, nhưng lại vận hành hoàn toàn dưới dạng dịch vụ đám mây độc quyền (proprietary cloud). Điều này đồng nghĩa với việc các tổ chức giáo dục không thể tự triển khai hệ thống trên hạ tầng nội bộ (self-host). Hệ quả là toàn bộ dữ liệu bài toán phải lưu trữ trên máy chủ của bên thứ ba. Quan trọng hơn, nền tảng này không có cơ chế nào cho phép tổ chức huy động và tận dụng các tài nguyên phần cứng có sẵn tại địa phương (như máy trạm trang bị GPU của khoa hay phòng thí nghiệm).

Các hệ thống chấm bài trực tuyến (Online Judge) truyền thống như DOMjudge [4] (chuyên dùng trong các kỳ thi ICPC) hay Codeforces [5] được thiết kế tối ưu riêng cho bộ môn lập trình thi đấu. Các nền tảng này hoạt động dựa trên cơ chế biên dịch mã nguồn (C++, Java, Python) và đối chiếu kết quả đầu ra với đáp án mẫu theo từng byte (byte-by-byte). Phương pháp đánh giá mang tính "tắt định" này hoàn toàn không phù hợp với bài toán AI, bởi kết quả của mô hình học máy cần được chấm điểm thông qua các độ đo thống kê chuyên biệt (metric) như F1-score, mAP, hay RMSE. Hơn thế nữa, các hệ thống này không hỗ trợ cơ chế chấm điểm qua quá trình suy luận của mô hình (inference-based judging), thiếu môi trường Python tùy chỉnh, và kiến trúc gốc không được thiết kế để chịu tải các tệp tin nộp bài có kích thước khổng lồ (lên tới hàng trăm Megabyte) đặc trưng của các checkpoint mô hình AI.

Các nền tảng thiết kế riêng cho lĩnh vực Data Science như AICrowd [7] và Codabench [6] mang lại sự linh hoạt đáng kể trong hình thức đánh giá. Mặc dù vậy, trong phạm vi khảo sát của đề tài, các nền tảng này vẫn phụ thuộc vào hạ tầng điện toán tập trung và chưa cung cấp mô hình máy chấm tình nguyện (máy chấm tình nguyện). Do đó, đơn vị tổ chức không thể phân tán khối lượng tính toán để tận dụng hệ thống máy tính GPU nhàn rỗi. Đồng thời, việc can thiệp sâu vào quy trình quản trị hoặc tinh chỉnh giao diện nhằm đáp ứng quy chuẩn riêng của từng trường đại học cũng gặp nhiều khó khăn về mặt triển khai.

Bảng 1.1 tổng hợp so sánh các nền tảng theo các tiêu chí phù hợp với yêu cầu tổ chức cuộc thi AI trong môi trường học thuật.

Bảng 1.1: So sánh các nền tảng thi AI

	Kaggle	DOMjudge	Alcrowd	Codabench	BKDNAIOJ
Self-hosted	✗	✓	✗	✓	✓
Volunteer judging	✗	✗	✗	✗	✓
Output-only judging	✓	✗	✓	✓	✓
Inference-based judging	Giới hạn	✗	✓	✓	✓
Custom judge script	Giới hạn	Giới hạn	Giới hạn	✓	✓
ICPC-style penalty	✗	✓	✗	✗	✓
Mô hình 4-phase	✗	✗	✗	✗	✓

(Ghi chú: “Giới hạn” - nền tảng có hỗ trợ tính năng này nhưng không đủ độ linh hoạt để đáp ứng yêu cầu triển khai thực tế cho mục đích học thuật tự chủ của các cơ sở giáo dục).

Kết quả đối chiếu từ Bảng 1.1 cho thấy một khoảng trống công nghệ rõ rệt trong phạm vi các nền tảng đã khảo sát. Cụ thể, chưa có một hệ thống nào đáp ứng đồng thời các tiêu chí cốt lõi bao gồm: khả năng tự triển khai (self-hosting), cơ chế chấm điểm phân tán dựa trên máy tính tình nguyện (volunteer distributed judging), mô hình thi đấu nhiều giai đoạn (tích hợp cả output-only và inference), cùng với giao thức kịch bản chấm điểm (judge script) tùy chỉnh đủ linh hoạt cho khâu vận hành nội bộ.

Đặc biệt, xét trên bình diện các giải pháp hiện hành, vẫn chưa có một hệ thống nào thiết kế cấu trúc kiến trúc tối ưu để phục vụ trọn vẹn quy trình tổ chức thi đấu, đánh giá năng lực và ôn luyện chuyên sâu cho kỳ thi Olympic Trí tuệ Nhân tạo Sinh viên Việt Nam. Việc giải quyết triệt để khoảng trống thực tiễn này chính là mục tiêu và động lực ra đời của nền tảng BKDNAIOJ.

1.2. Các hệ thống liên quan

1.2.1 Nền tảng đánh giá trực tuyến (Online Judge) truyền thống

Các hệ thống đánh giá trực tuyến (Online Judge - OJ) truyền thống như DOMjudge [4], ICPC Live Archive hay Codeforces [5] chủ yếu vận hành dựa trên một mô hình tiêu chuẩn: thí sinh nộp mã nguồn, hệ thống tiến hành biên dịch, thực thi độc lập, và đối chiếu kết quả đầu ra với đáp án chuẩn để trả về các trạng thái như Accepted (Chấp nhận) hoặc Wrong Answer (Sai kết quả). Đặc trưng cốt lõi của mô hình này là tính tất định tuyệt đối: một bộ dữ liệu đầu vào luôn sinh ra duy nhất một đầu ra tương ứng, do đó việc đối chiếu có thể được thực hiện triệt để thông qua các công cụ so sánh chuẩn xác đến từng byte (byte-by-byte diff).

Ưu điểm nổi bật của các hệ thống OJ truyền thống nằm ở độ tin cậy cao, khả năng triển khai tương đối đơn giản và tốc độ phản hồi cực kỳ nhanh nhạy đối với các bài toán thuật toán cổ điển. Điển hình như DOMjudge - hệ thống tiêu chuẩn của các kỳ thi Lập trình sinh viên quốc tế (ICPC) - sở hữu kiến trúc chấm điểm dạng trình nền (judge daemon) vận hành bên trong các môi trường cô lập nghiêm ngặt. Đây cũng chính là một nguyên lý kỹ thuật mang tính nền tảng mà hệ thống BKDNAIOJ đã kế thừa và phát triển tại tầng thực thi an toàn (sandbox).

Tuy nhiên, khi áp dụng trực tiếp vào đặc thù của các kỳ thi Trí tuệ Nhân tạo, các hệ thống OJ truyền thống bộc lộ ba điểm bất cập cốt lõi:

1. **Phương pháp đánh giá:** Kết quả dự đoán của các mô hình Học máy không mang tính chính xác tuyệt đối định dạng văn bản mà cần được đo lường thông qua các độ đo thống kê tùy chỉnh (custom metrics) như F1-score, mAP, RMSE hay BLEU, thay vì phương pháp đối chiếu tĩnh byte-by-byte.
2. **Quy mô dữ liệu nộp bài:** Dung lượng tệp tin trong một kỳ thi AI thường có kích thước khổng lồ (lên đến hàng trăm Megabyte đối với các tệp trọng số checkpoint hoặc tệp dự đoán trên tập dữ liệu quy mô lớn), vượt xa giới hạn xử lý thông thường của một lượt nộp mã nguồn thuần túy.
3. **Tính linh hoạt của logic chấm điểm:** Quá trình đánh giá bài toán AI đòi hỏi phải thực thi các kịch bản chấm điểm phức tạp bằng Python do Ban tổ chức quy định. Việc gán cứng (hardcode) các thuật toán chấm điểm này vào máy chủ hệ thống là không khả thi và thiếu tính mở rộng.

Để khắc phục triệt để các rào cản này, BKDNAIOJ đã áp dụng thiết kế tách biệt hoàn toàn phân hệ tính điểm thành các kịch bản đánh giá độc lập (judge script). Cụ thể, mỗi bài toán sẽ được gán với một tệp Python riêng biệt do Ban tổ chức cung cấp. Kịch

bản này có nhiệm vụ đọc dữ liệu từ thư mục chứa kết quả dự đoán của thí sinh và thư mục chứa đáp án thực tế (ground truth), sau đó tính toán và trả về điểm số dưới định dạng chuẩn JSON thông qua luồng xuất chuẩn (stdout). Kiến trúc này đảm bảo tính linh hoạt tối ưu, cho phép định nghĩa các hàm đánh giá chuyên biệt cho từng bài toán mà không yêu cầu bất kỳ sự can thiệp hay nâng cấp nào đối với hệ thống lõi.

1.2.2 Các nền tảng thi đấu Học máy và Khoa học Dữ liệu

Kaggle Competitions [2] hiện là một trong những nền tảng thi đấu Học máy quy mô lớn nhất thế giới. Nền tảng này chủ yếu vận hành theo mô hình chấm điểm đầu ra (output-only judging): thí sinh tải bộ dữ liệu về, huấn luyện và suy luận mô hình trên thiết bị cá nhân, sau đó nộp tệp kết quả (thường dưới định dạng CSV). Hệ thống sẽ tự động tính toán các độ đo (metric) và cập nhật bảng xếp hạng công khai theo thời gian thực. Ưu điểm của kiến trúc này là tính bảo mật cao do máy chủ không trực tiếp thực thi mã nguồn của thí sinh. Tuy nhiên, rủi ro lớn nhất là hệ thống không thể xác minh được việc thí sinh thực sự đã huấn luyện mô hình hay chỉ đơn thuần là thao túng kết quả dự đoán. Đây chính là động lực đòi hỏi các kỳ thi chuyên sâu phải bổ sung giai đoạn suy luận cuối cùng (final inference) nhằm minh bạch hóa và xác thực tính trung thực của giải pháp.

Các nền tảng như CodaLab Competitions [6] và AICrowd [7] mang đến giải pháp toàn diện hơn cho các thử thách Khoa học Dữ liệu bằng việc hỗ trợ cấu trúc thi đấu nhiều giai đoạn, cho phép nộp cả kết quả lẫn mã nguồn, cùng hệ thống bảng xếp hạng đa tiêu chí. Cơ chế chương trình chấm điểm (scoring program) của các nền tảng này có nhiều nét tương đồng với giao thức "judging contract" được đề xuất trong BKDNAIOJ. Mặc dù vậy, sự khác biệt mang tính quyết định nằm ở thiết kế cấp phát tài nguyên: trong khi CodaLab và AICrowd hoàn toàn phụ thuộc vào hạ tầng điện toán đám mây tập trung của nhà cung cấp, BKDNAIOJ lại hướng đến mô hình máy chấm tự nguyện (máy chấm tình nguyện) do chính tổ chức tự quản lý, mở ra khả năng tận dụng triệt để các hệ thống máy tính phân tán nội bộ.

1.2.3 Hệ thống phân tán và lập lịch tác vụ

BOINC (Berkeley Open Infrastructure for Network Computing) [8] là một trong những nền tảng tiên phong trong lĩnh vực **volunteer computing**, cho phép người dùng đóng góp tài nguyên tính toán nhàn rỗi cho các dự án khoa học quy mô lớn như SETI@home và Folding@home. BOINC giải quyết hiệu quả bài toán phân phối công việc trên tập hợp các nút tính toán không đồng nhất, đồng thời cung cấp các cơ chế chịu lỗi như phát hiện worker ngừng hoạt động và tái phân phối công việc khi kết quả không

được trả về đúng thời hạn. BKDNAIOJ vận dụng ý tưởng này bằng cách cho phép bất kỳ máy tính nào cài đặt *olpai-volunteer-agent* và được quản trị viên phê duyệt đều có thể tham gia quá trình chấm bài, qua đó giảm yêu cầu đầu tư hạ tầng chuyên dụng.

HTCondor [9] là hệ thống lập lịch phân tán dành cho các bài toán **high-throughput computing**, nổi bật với cơ chế *matchmaking*, trong đó tác vụ được gán cho các worker dựa trên tập yêu cầu về năng lực xử lý. Cơ chế này đặc biệt phù hợp với môi trường tài nguyên không đồng nhất, nơi một số tác vụ có thể yêu cầu GPU hoặc dung lượng bộ nhớ lớn. BKDNAIOJ tham khảo ý tưởng phân phối công việc dựa trên năng lực của worker, tuy nhiên áp dụng theo mô hình worker chủ động truy vấn và nhận tác vụ (*polling model*) thay vì bộ lập lịch chủ động phân phối. Ngoài ra, hệ thống sử dụng các **typed slots** để phân biệt giữa các tác vụ chấm kết quả thông thường và các tác vụ thực hiện final inference.

Redis Streams [10] là một giải pháp hàng đợi thông điệp (*message queue*) phù hợp với các kiến trúc producer-consumer yêu cầu độ trễ thấp. Redis Streams được lựa chọn trong BKDNAIOJ nhờ ba ưu điểm chính: (1) độ trễ thấp đối với các hệ thống có quy mô vừa và nhỏ; (2) hỗ trợ cơ chế *consumer groups*, cho phép phân phối tác vụ đồng thời tới nhiều worker; và (3) hỗ trợ thực thi các Lua script theo cơ chế nguyên tử (*atomic execution*) trên máy chủ Redis. Đặc điểm cuối cùng đặc biệt quan trọng trong việc hiện thực hóa cơ chế nhận tác vụ (*job claiming*), giúp tránh các điều kiện tranh chấp (*race conditions*) khi nhiều worker đồng thời cố gắng nhận cùng một tác vụ.

1.3. Đóng góp của đề tài

1.3.1 Yêu cầu đặc thù của cuộc thi AI - Mô hình 4-phase

Một cuộc thi AI hiện đại, điển hình như Olympic Trí tuệ Nhân tạo Sinh viên Việt Nam bao gồm nhiều giai đoạn đánh giá với mục tiêu và ràng buộc khác nhau. Việc chỉ sử dụng cơ chế đánh giá output-only không đủ để xác minh tính hợp lệ của mô hình mà thí sinh xây dựng, trong khi việc áp dụng hoàn toàn cơ chế inference-based lại làm gia tăng đáng kể chi phí tính toán và thời gian phản hồi của hệ thống. Do đó, BKDNAIOJ triển khai mô hình đánh giá gồm bốn giai đoạn giống cuộc thi chính thức, vừa đảm bảo tính công bằng của cuộc thi, vừa giúp thí sinh làm quen với quy trình đánh giá của cuộc thi Olympic chính thức.

Public Test và Private Test là hai giai đoạn đánh giá theo cơ chế *output-only*. Trong các giai đoạn này, thí sinh thực hiện quá trình huấn luyện và suy luận trên máy cá nhân, sau đó nộp tệp kết quả dự đoán để hệ thống chấm điểm. Điểm khác biệt giữa hai giai đoạn nằm ở bộ dữ liệu đánh giá: Public Test sử dụng tập dữ liệu mà thí sinh có thể

tiếp cận hoặc quan sát một phần, trong khi Private Test sử dụng tập dữ liệu bí mật nhằm hạn chế hiện tượng tối ưu hóa quá mức theo bảng xếp hạng. Cách tiếp cận này tương tự cơ chế phân tách Public Leaderboard và Private Leaderboard được sử dụng trong nhiều cuộc thi Machine Learning hiện nay.

Final Public và Final Private là các giai đoạn đánh giá theo cơ chế *inference-based*. Thay vì nộp trực tiếp kết quả dự đoán, thí sinh phải nộp toàn bộ mã nguồn và trọng số mô hình dưới dạng gói nén. Hệ thống sẽ phân phối tác vụ đến các máy chấm tình nguyện, nơi mô hình được giải nén, thực thi trong môi trường cô lập và sinh ra kết quả dự đoán. Kết quả này sau đó được sử dụng làm đầu vào cho quá trình chấm điểm. Cơ chế này cho phép xác minh khả năng tái lập của mô hình và đảm bảo rằng kết quả dự thi được sinh ra từ một pipeline suy luận hợp lệ. Tương tự các giai đoạn output-only, Final Public sử dụng tập dữ liệu công khai, trong khi Final Private sử dụng tập dữ liệu bí mật phục vụ việc xếp hạng cuối cùng.

Việc kết hợp đồng thời bốn loại giai đoạn đánh giá giúp giải quyết hai yêu cầu quan trọng trong tổ chức các cuộc thi AI. Thứ nhất, dữ liệu đánh giá bí mật chỉ được sử dụng ở các giai đoạn phù hợp nhằm hạn chế nguy cơ rò rỉ thông tin và hiện tượng overfitting theo leaderboard. Thứ hai, các giai đoạn inference-based cung cấp cơ chế xác minh độc lập đối với mô hình dự thi, đảm bảo rằng kết quả đạt được có thể được tái tạo trong môi trường thực thi của hệ thống. Nhờ đó, quá trình đánh giá vừa duy trì được tính hiệu quả của mô hình output-only, vừa đáp ứng được yêu cầu về tính minh bạch và độ tin cậy của kết quả cuối cùng.

1.3.2 Đóng góp chính của đề tài

Đề tài đề xuất và cài đặt năm đóng góp kỹ thuật chính, được tổng hợp trong Bảng 1.2.

Bảng 1.2: Đóng góp chính của đề tài

	Đóng góp chính	Mô tả
1	Hệ thống nền tảng BKDNAIOJ	Nền tảng tổ chức thi AI trực tuyến tự chủ (self-hosted), hỗ trợ toàn diện vòng đời cuộc thi: từ đăng ký, nộp bài, tự động chấm điểm đến cập nhật bảng xếp hạng theo thời gian thực.
2	Mô hình thi đấu 4 giai đoạn (4-phase contest)	Số hóa hoàn toàn quy trình thi khắt khe của Olympic AI qua 4 giai đoạn (<i>Public Test</i> , <i>Private Test</i> , <i>Final Public</i> , <i>Final Private</i>), kết

		hợp chặt chẽ giữa việc nộp kết quả (output) và nộp mã nguồn suy luận (inference) để chống gian lận.
3	Cơ chế chấm bài phân tán tự nguyện (<i>Volunteer distributed judging</i>)	Cho phép tận dụng các máy tính nhàn rỗi (như máy tính phòng Lab, workstation cá nhân) làm các node chấm bài. Hỗ trợ chạy an toàn qua Docker hoặc chạy trực tiếp để tận dụng tối đa sức mạnh GPU của tổ chức.
4	Điều phối tác vụ linh hoạt (<i>Capability-aware FIFO</i>)	Thuật toán phân bổ bài nộp thông minh giúp các máy chấm (worker) chủ động nhận việc dựa trên năng lực phần cứng thực tế và cấu hình xử lý (chấm output/inference), giúp tối ưu hóa tài nguyên và chống nghẽn cổ chai.
5	Chuẩn hóa giao thức chấm điểm (<i>Judging contract</i>)	Tách biệt hoàn toàn hàm tính điểm (metric) khỏi hệ thống lõi. Ban tổ chức chỉ cần viết kịch bản tính điểm (judge script) bằng Python một lần, hệ thống sẽ tự động phân phối và tái sử dụng trên mọi máy chấm.

Đóng góp 1 (Hệ thống BKDNAIOJ): Đề tài đã thiết kế và hiện thực hệ thống BKDNAIOJ, một nền tảng phục vụ tổ chức và đánh giá các cuộc thi Trí tuệ nhân tạo. Hệ thống tích hợp các thành phần quản lý cuộc thi, quản lý bài nộp, đánh giá kết quả, thực thi mô hình và điều phối tài nguyên tính toán trong một kiến trúc thống nhất. BKDNAIOJ đã được triển khai và sử dụng trong một số kỳ thi thực tế, qua đó bước đầu chứng minh tính ổn định và khả năng ứng dụng của hệ thống trong môi trường đào tạo và nghiên cứu.

Đóng góp 2 (Mô hình thi 4 giai đoạn): Giải quyết triệt để rào cản về quy trình tổ chức thi AI - một khoảng trống lớn mà các nền tảng hiện nay (đã phân tích ở Bảng 1.1) chưa thể đáp ứng toàn diện. BKDNAIOJ là nền tảng hiếm hoi hỗ trợ đồng thời khả năng tự triển khai, chấm điểm đầu ra (output-only), chấm điểm suy luận (inference judging) và quản lý linh hoạt các giai đoạn thi sát với thể thức khắt khe của Olympic AI.

Đóng góp 3 (Cơ chế chấm bài phân tán): Đề tài phát triển cơ chế chấm bài phân tán cho phép huy động các tài nguyên tính toán sẵn có từ nhiều nguồn khác nhau, bao gồm máy tính phòng thí nghiệm, máy trạm và máy cá nhân. Cách tiếp cận này giúp giảm sự phụ thuộc vào hạ tầng máy chủ chuyên dụng và tăng khả năng mở rộng của hệ

thống khi số lượng bài nộp tăng lên. Bên cạnh đó, việc đóng gói worker agent dưới dạng thư viện Python giúp đơn giản hóa quá trình triển khai và quản lý các nút chấm bài.

Đóng góp 4 (Điều phối Capability-aware FIFO): Đề tài đề xuất chiến lược điều phối Capability-aware FIFO nhằm phân phối tác vụ trên tập hợp các worker có cấu hình phần cứng không đồng nhất. Thuật toán kết hợp nguyên tắc FIFO với cơ chế kiểm tra năng lực xử lý của worker trước khi gán tác vụ, giúp đảm bảo tính tương thích giữa yêu cầu của bài chấm và tài nguyên thực thi. Hiệu quả của chiến lược được đánh giá thông qua các thực nghiệm và phân tích định lượng được trình bày trong Chương 5.

Đóng góp 5 (Giao thức Judging contract): Đề tài thiết kế giao thức Judging Contract nhằm tách biệt hoàn toàn logic đánh giá khỏi hệ thống lõi của nền tảng. Theo cơ chế này, mỗi bài toán được định nghĩa bởi một chương trình đánh giá độc lập, nhận dữ liệu đầu vào và kết quả dự đoán của thí sinh, sau đó trả về kết quả chấm điểm theo định dạng chuẩn JSON. Thiết kế này nâng cao tính mở rộng và khả năng tái sử dụng của hệ thống, đồng thời cho phép ban tổ chức dễ dàng phát triển và tích hợp các tiêu chí đánh giá mới mà không cần thay đổi kiến trúc cốt lõi.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Mô hình chấm bài trong hệ thống Online Judge

2.1.1 Online Judge truyền thống

Online Judge (OJ) là hệ thống tự động tiếp nhận, thực thi và đánh giá bài nộp của người tham gia. Trong các cuộc thi lập trình truyền thống, quy trình chấm bài thường gồm các bước: tiếp nhận mã nguồn, biên dịch, chạy chương trình trên bộ test case định sẵn và so sánh kết quả đầu ra với đáp án chuẩn. Dựa trên kết quả thực thi, hệ thống trả về các trạng thái như Accepted, Wrong Answer, Time Limit Exceeded hoặc Runtime Error [4].

Ưu điểm chính của mô hình OJ truyền thống là tính xác định cao. Với cùng mã nguồn, cùng bộ test và cùng môi trường thực thi, kết quả chấm có thể được tái lập rõ ràng, từ đó đảm bảo tính công bằng giữa các thí sinh. DOMjudge và Codeforces là hai hệ thống OJ tiêu biểu được sử dụng rộng rãi trong các cuộc thi lập trình [4][5].

Tuy nhiên, mô hình này không hoàn toàn phù hợp với các cuộc thi Trí tuệ nhân tạo. Trong bài toán AI, kết quả thường được đánh giá bằng các metric chuyên biệt như accuracy, F1-score, mean squared error hoặc các độ đo do ban tổ chức định nghĩa, thay vì so sánh trực tiếp từng ký tự đầu ra. Ngoài ra, bài nộp có thể bao gồm file dự đoán, checkpoint, mô hình đã huấn luyện hoặc mã nguồn suy luận với kích thước lớn hơn nhiều so với bài nộp lập trình thông thường. Vì vậy, các cuộc thi AI cần những mô hình chấm bài phù hợp hơn.

2.1.2 Output-only Judging

Output-only judging là mô hình chấm bài trong đó người tham gia tự huấn luyện hoặc suy luận trên môi trường của mình, sau đó nộp file kết quả dự đoán lên hệ thống. Hệ thống không thực thi mã nguồn hay mô hình của thí sinh, mà chỉ chạy chương trình đánh giá do ban tổ chức cung cấp để so sánh kết quả với dữ liệu chuẩn và tính điểm theo metric đã định nghĩa.

Mô hình này giúp giảm tải đáng kể cho hệ thống chấm bài vì không cần chạy mô hình của người tham gia. Đồng thời, yêu cầu cô lập môi trường thực thi cũng đơn giản hơn do hệ thống chủ yếu thực thi judge script của ban tổ chức. Output-only judging phù hợp với các giai đoạn cần phản hồi nhanh, giúp thí sinh kiểm tra định dạng bài nộp, đánh giá chất lượng mô hình và điều chỉnh phương pháp trong quá trình thi.

Hạn chế của mô hình này là hệ thống không kiểm chứng được quy trình tạo ra kết quả dự đoán. Do đó, một kết quả đạt điểm cao chưa chắc đảm bảo tính tái lập nếu không có bước kiểm tra lại bằng mô hình hoặc mã nguồn gốc.

2.1.3 Inference-based Judging

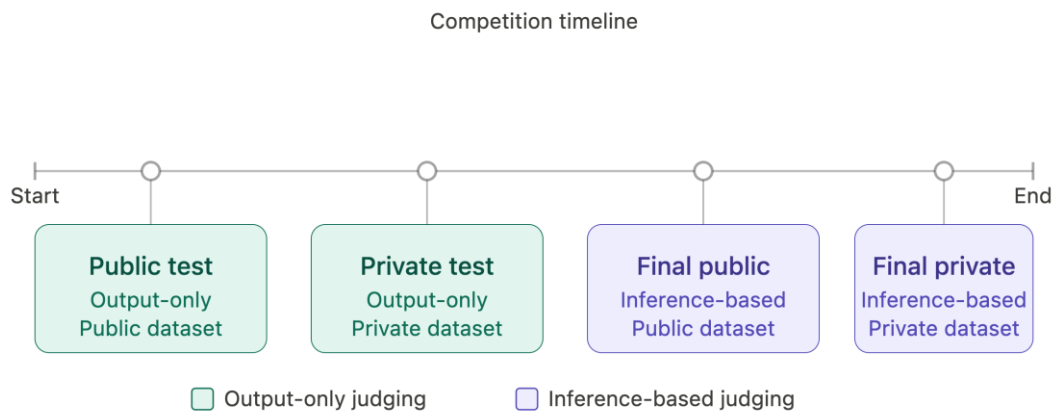
Inference-based judging là mô hình chấm bài trong đó người tham gia nộp mã nguồn, trọng số mô hình và các tệp cần thiết để hệ thống tự thực thi quá trình suy luận. Sau khi chương trình suy luận sinh ra kết quả dự đoán, hệ thống sử dụng judge script để tính điểm theo metric của bài toán.

Khác với output-only judging, mô hình này cho phép ban tổ chức kiểm chứng trực tiếp khả năng tái lập của lời giải, vì kết quả được sinh ra trong môi trường chấm chính thức. Cách tiếp cận này phù hợp với các giai đoạn đánh giá cuối cùng của cuộc thi AI, nơi tính hợp lệ và khả năng tái lập của mô hình có vai trò quan trọng.

Tuy nhiên, inference-based judging đặt ra nhiều thách thức kỹ thuật. Thời gian thực thi có thể khác nhau đáng kể giữa các mô hình; một số mô hình chỉ cần CPU, trong khi số khác yêu cầu GPU hoặc dung lượng bộ nhớ lớn. Ngoài ra, hệ thống phải thực thi mã nguồn do người dùng cung cấp, nên cần có cơ chế cô lập môi trường, giới hạn tài nguyên và kiểm soát lỗi để bảo vệ hạ tầng chấm bài.

2.1.4 Mô hình đa phase trong cuộc thi AI

Các cuộc thi AI hiện đại, điển hình như Olympic Trí tuệ Nhân tạo Sinh viên Việt Nam, thường được tổ chức theo nhiều phase nhằm cân bằng giữa phản hồi nhanh trong quá trình thi và đánh giá chính xác ở giai đoạn cuối. Thay vì chỉ sử dụng một cơ chế chấm duy nhất, hệ thống có thể kết hợp nhiều hình thức đánh giá khác nhau tùy theo mục tiêu của từng phase.



Hình 2.1: Mô hình đánh giá đa phase trong cuộc thi AI

Ở các phase public, người tham gia thường được phép nộp kết quả để nhận phản hồi nhanh trên một phần dữ liệu kiểm thử. Điều này giúp thí sinh phát hiện lỗi định dạng, kiểm tra hướng tiếp cận và cải thiện mô hình trong quá trình thi. Ngược lại, các phase private sử dụng tập dữ liệu được giữ bí mật nhằm đánh giá khách quan hơn khả năng tổng quát hóa của mô hình.

Bên cạnh đánh giá trực tiếp trên file kết quả, các phase final có thể yêu cầu người tham gia nộp mô hình hoặc chương trình suy luận để hệ thống tự sinh kết quả dự đoán trong môi trường chấm chính thức. Cách tổ chức này giúp ban tổ chức kiểm tra tính tái lập của lời giải, đồng thời giảm rủi ro phụ thuộc hoàn toàn vào tệp kết quả do thí sinh nộp.

Như vậy, mô hình đa phase giúp hệ thống chấm bài AI linh hoạt hơn so với Online Judge truyền thống. Mô hình này vừa hỗ trợ phản hồi nhanh cho thí sinh, vừa đảm bảo đánh giá nghiêm ngặt ở các giai đoạn quan trọng của cuộc thi.

2.2. Hàng đợi tin nhắn và Redis Streams

2.2.1 Mô hình xử lý bất đồng bộ với hàng đợi tin nhắn

Hàng đợi tin nhắn (message queue) là một thành phần trung gian cho phép các thành phần trong hệ thống giao tiếp với nhau theo cơ chế bất đồng bộ. Thay vì producer gọi trực tiếp consumer và phải chờ xử lý hoàn tất, producer chỉ cần đưa thông điệp hoặc tác vụ vào hàng đợi. Consumer sau đó có thể lấy dữ liệu từ hàng đợi và xử lý theo tốc độ của mình.

Cách tổ chức này giúp tách biệt tốc độ sinh tác vụ và tốc độ xử lý tác vụ. Khi số lượng yêu cầu tăng đột biến, các tác vụ có thể được lưu tạm trong hàng đợi thay vì bị từ chối hoặc làm quá tải thành phần xử lý phía sau. Vì vậy, message queue thường được sử dụng trong các hệ thống cần xử lý nền, phân phối công việc hoặc điều phối nhiều tiến trình xử lý song song [12].

Trong bài toán chấm bài phân tán, hàng đợi tin nhắn đóng vai trò lưu giữ các job chấm bài sau khi submission được tiếp nhận. API server chỉ cần ghi job vào hàng đợi, trong khi các worker sẽ lấy job ra xử lý khi còn tài nguyên khả dụng. Nhờ đó, hệ thống không yêu cầu worker phải sẵn sàng tại đúng thời điểm người dùng nộp bài, đồng thời vẫn đảm bảo các submission được xử lý tuần tự hoặc theo chính sách lập lịch phù hợp.

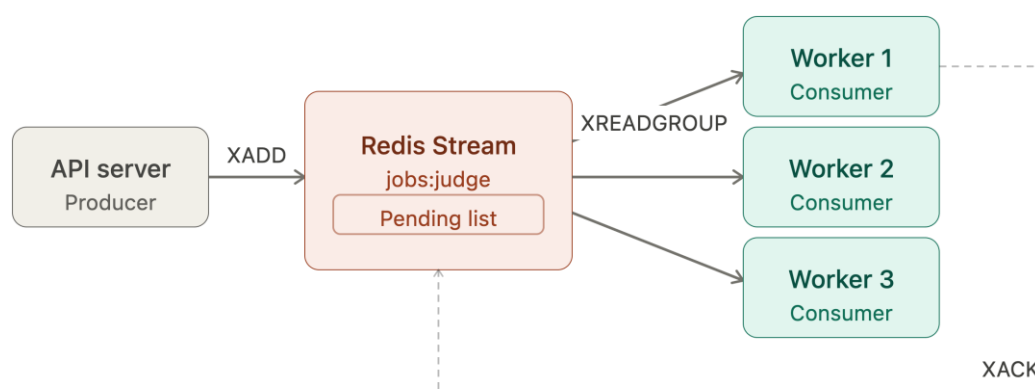
2.2.2 Redis Streams

Redis Streams là một cấu trúc dữ liệu của Redis được thiết kế để lưu trữ chuỗi thông điệp theo dạng log có thứ tự [10]. Mỗi entry trong stream được gán một ID tăng

dẫn theo thời gian, cho phép các consumer đọc dữ liệu theo thứ tự phát sinh. Khác với cơ chế publish/subscribe truyền thống, trong đó thông điệp có thể bị mất nếu không có consumer đang lắng nghe, Redis Streams cho phép thông điệp được lưu lại trong stream cho đến khi bị xóa hoặc bị loại bỏ theo chính sách giới hạn kích thước.

Một số thao tác cơ bản của Redis Streams bao gồm:

- **XADD**: thêm một entry mới vào cuối stream.
- **XRANGE**: đọc các entry trong một khoảng ID nhất định.
- **XDEL**: xóa entry khỏi stream theo ID.
- **XREADGROUP** và **XACK**: hỗ trợ mô hình consumer group, cho phép nhiều consumer phối hợp xử lý các entry và xác nhận sau khi xử lý hoàn tất.



Hình 2.2: Luồng xử lý bất đồng bộ với Redis Streams và Consumer Group

Consumer group là một cơ chế quan trọng của Redis Streams, cho phép nhiều consumer cùng tham gia xử lý một stream. Redis theo dõi các entry đã được giao nhưng chưa được xác nhận thông qua pending list. Cơ chế này hỗ trợ việc phát hiện các thông điệp chưa hoàn tất xử lý và tạo điều kiện cho việc xử lý lại khi consumer gặp lỗi.

Bên cạnh đó, Redis còn hỗ trợ thực thi Lua script như một đơn vị nguyên tử. Điều này giúp kết hợp nhiều thao tác kiểm tra và cập nhật dữ liệu thành một bước xử lý không bị gián đoạn. Trong các hệ thống có nhiều consumer hoạt động đồng thời, tính nguyên tử này có ý nghĩa quan trọng để hạn chế race condition và đảm bảo một thông điệp chỉ được một consumer claim thành công tại một thời điểm.

Nhờ đặc điểm lưu trữ có thứ tự, hỗ trợ consumer group, pending list và các thao tác nguyên tử, Redis Streams phù hợp cho nhiều bài toán xử lý bất đồng bộ như ghi log

sự kiện, phân phối tác vụ nền, xử lý job theo hàng đợi và xây dựng pipeline xử lý dữ liệu thời gian thực.

2.3. Lưu trữ đối tượng và Presigned URL

2.3.1 Object Storage và giao thức S3-compatible

Object storage là mô hình lưu trữ trong đó dữ liệu được quản lý dưới dạng các đối tượng (object). Mỗi object thường bao gồm dữ liệu, metadata và một khóa định danh duy nhất trong phạm vi một bucket. Khác với mô hình lưu trữ dạng hệ thống tệp truyền thống, object storage không phụ thuộc chặt chẽ vào cấu trúc thư mục phân cấp mà tổ chức dữ liệu thông qua các object key.

Mô hình này phù hợp với các tệp có kích thước lớn, dữ liệu ít cần chỉnh sửa từng phần và thường được truy cập thông qua giao thức HTTP. Do đó, object storage thường được sử dụng để lưu trữ dataset, tệp media, bản sao lưu, artifact, mô hình đã huấn luyện hoặc các tệp kết quả sinh ra từ hệ thống xử lý dữ liệu [13].

Amazon S3 là một trong những dịch vụ object storage phổ biến nhất. Bộ API của S3 đã trở thành một chuẩn thực tế trong lĩnh vực lưu trữ đối tượng. Các hệ thống S3-compatible triển khai giao diện tương thích với S3, cho phép ứng dụng thao tác với object storage thông qua các khái niệm như bucket, object key, upload, download và delete mà không phụ thuộc hoàn toàn vào một nhà cung cấp cụ thể.

2.3.2 Presigned URL

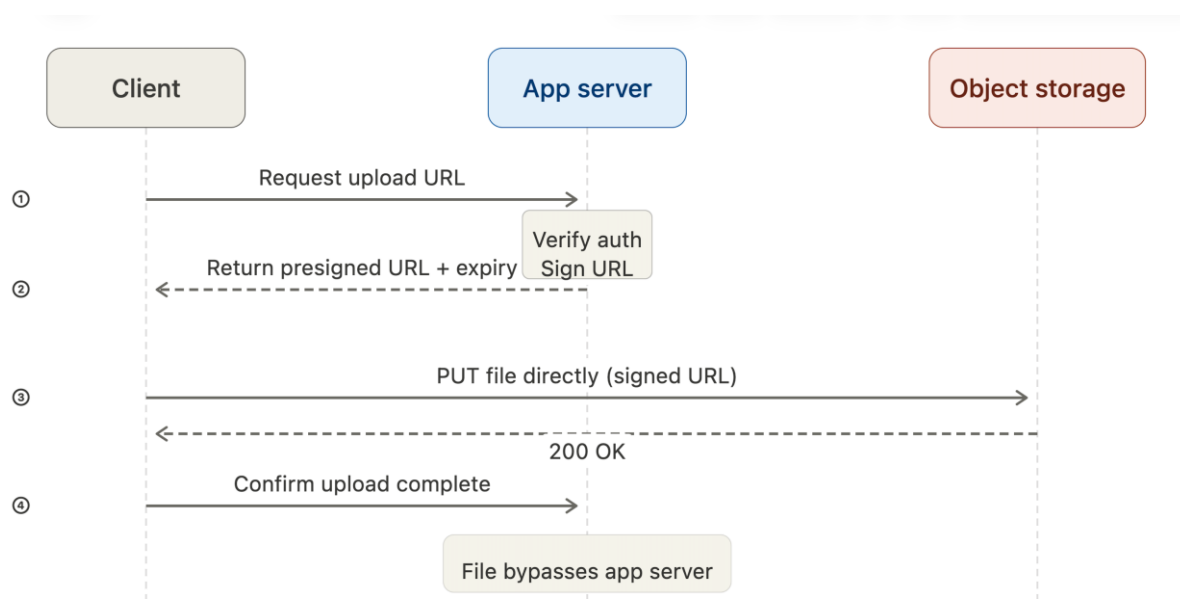
Presigned URL là cơ chế cho phép cấp quyền truy cập tạm thời đến một object cụ thể trong object storage mà không cần chia sẻ trực tiếp thông tin xác thực của hệ thống. Server sinh ra một URL có chứa chữ ký mật mã, phương thức truy cập được phép và thời gian hiệu lực. Client sau đó sử dụng URL này để thực hiện thao tác trực tiếp với object storage, chẳng hạn upload bằng phương thức PUT hoặc tải xuống bằng phương thức GET [13].

Cơ chế này đặc biệt hữu ích trong các hệ thống cần xử lý tệp dung lượng lớn. Nếu toàn bộ dữ liệu phải đi qua server ứng dụng, server sẽ tiêu tốn nhiều băng thông, bộ nhớ và thời gian xử lý cho việc truyền tải file. Với Presigned URL, client có thể giao tiếp trực tiếp với object storage, trong khi server ứng dụng chỉ chịu trách nhiệm kiểm tra quyền, sinh URL và quản lý trạng thái nghiệp vụ liên quan.

Ngoài việc giảm tải cho server, Presigned URL còn giúp giới hạn phạm vi truy cập. URL thường chỉ có hiệu lực trong một khoảng thời gian nhất định và chỉ cho phép thao tác trên đúng object key đã được chỉ định. Nhờ đó, client không cần biết thông tin

xác thực của hệ thống lưu trữ và cũng không có quyền truy cập tùy ý đến các object khác.

Một luồng upload sử dụng Presigned URL thường gồm hai bước. Trước tiên, client gửi yêu cầu đến server để khởi tạo phiên upload. Server kiểm tra quyền truy cập, xác định object key hợp lệ và sinh Presigned URL tương ứng. Sau đó, client sử dụng URL này để upload file trực tiếp lên object storage. Khi upload hoàn tất, client có thể gửi yêu cầu xác nhận về server để server kiểm tra object và cập nhật trạng thái xử lý tiếp theo.



Hình 2.3: Luồng upload tệp sử dụng Presigned URL

2.4. Tính toán phân tán và lập lịch trên worker không đồng nhất

2.4.1 Volunteer Computing

Volunteer computing là mô hình tính toán phân tán trong đó tài nguyên tính toán nhân rỗi từ nhiều thiết bị tự nguyện được huy động để thực hiện các tác vụ chung. Các thiết bị tham gia có thể là máy tính cá nhân, máy chủ nhỏ hoặc các môi trường tính toán không chuyên dụng. Nền tảng BOINC (Berkeley Open Infrastructure for Network Computing) là một ví dụ tiêu biểu, từng được sử dụng trong nhiều dự án khoa học quy mô lớn như SETI@home và Folding@home [8].

Đặc trưng quan trọng của mô hình volunteer computing là tính không đồng nhất và tính biến động của tài nguyên. Các worker tham gia có thể khác nhau về năng lực xử lý, dung lượng bộ nhớ, khả năng hỗ trợ GPU, hệ điều hành và chất lượng kết nối mạng. Ngoài ra, worker có thể tham gia hoặc rời khỏi hệ thống bất kỳ lúc nào. Vì vậy, hệ thống

sử dụng volunteer computing cần có cơ chế phân phối công việc phù hợp, đồng thời phải xử lý được trường hợp worker mất kết nối hoặc dừng hoạt động trong quá trình thực thi.

2.4.2 Bài toán lập lịch trên worker không đồng nhất

Lập lịch công việc là bài toán phân công các tác vụ cho các tài nguyên xử lý nhằm đạt được một hoặc nhiều mục tiêu nhất định, chẳng hạn như giảm thời gian chờ, tăng thông lượng, cân bằng tải hoặc tối ưu mức sử dụng tài nguyên [11]. Trong môi trường worker không đồng nhất, bài toán lập lịch không chỉ phụ thuộc vào thứ tự xuất hiện của tác vụ mà còn phải xét đến sự tương thích giữa yêu cầu thực thi của tác vụ và năng lực của worker.

Một số tác vụ có thể chỉ yêu cầu CPU thông thường, trong khi các tác vụ khác cần GPU, dung lượng bộ nhớ lớn hoặc môi trường thực thi đặc biệt. Nếu tác vụ được phân công cho worker không phù hợp, quá trình xử lý có thể thất bại, kéo dài thời gian thực thi hoặc làm giảm hiệu quả sử dụng tài nguyên toàn hệ thống.

Một hướng tiếp cận đơn giản là lập lịch theo thứ tự FIFO, trong đó tác vụ đến trước được xử lý trước. Phương pháp này dễ triển khai và đảm bảo tính công bằng theo thời gian đến, nhưng không xét đến năng lực khác nhau giữa các worker. Để khắc phục hạn chế này, có thể kết hợp FIFO với cơ chế kiểm tra năng lực, thường được gọi là **Capability-aware FIFO**. Theo cách tiếp cận này, hệ thống vẫn duyệt tác vụ theo thứ tự đến, nhưng chỉ cấp phát tác vụ cho worker nếu worker đáp ứng được các yêu cầu thực thi tương ứng.

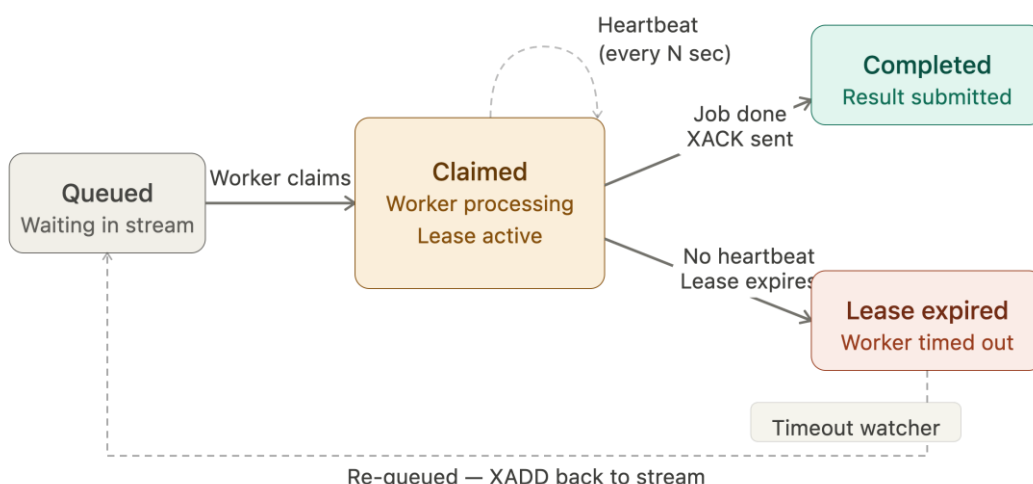
Bên cạnh đó, trong các hệ thống có worker biến động, mô hình **worker-driven scheduling** hay **pull model** thường được sử dụng. Thay vì server chủ động đẩy tác vụ đến một worker cụ thể, worker sẽ chủ động yêu cầu nhận việc khi còn tài nguyên khả dụng. Cách tổ chức này phù hợp với môi trường phân tán không ổn định, vì hệ thống không cần giả định trước worker nào sẽ luôn sẵn sàng tại một thời điểm nhất định.

2.4.3 Cơ chế Claim Lease và Heartbeat

Trong hệ thống phân tán, một worker có thể mất kết nối, bị dừng đột ngột hoặc gặp lỗi trong khi đang xử lý tác vụ. Nếu hệ thống không có cơ chế theo dõi trạng thái xử lý, tác vụ có thể bị kẹt ở trạng thái đang chạy và không được worker khác tiếp tục xử lý. Để giải quyết vấn đề này, nhiều hệ thống sử dụng cơ chế **claim lease** kết hợp với **heartbeat** [14].

Claim lease là cơ chế trong đó worker chỉ được quyền sở hữu một tác vụ trong một khoảng thời gian nhất định. Sau khi nhận tác vụ, worker cần gửi tín hiệu heartbeat định kỳ để thông báo rằng nó vẫn đang hoạt động và tiếp tục xử lý tác vụ. Nếu hệ thống không nhận được heartbeat trong thời gian hợp lệ, lease có thể được xem là hết hạn và tác vụ có thể được đưa trở lại hàng đợi để một worker khác xử lý.

Cơ chế này giúp hệ thống tránh mất tác vụ khi worker gặp sự cố. Về mặt đảm bảo xử lý, claim lease thường hướng đến mô hình **at-least-once delivery**, nghĩa là mỗi tác vụ sẽ được xử lý ít nhất một lần. Tuy nhiên, mô hình này cũng đặt ra khả năng một tác vụ có thể bị xử lý lại trong một số tình huống, chẳng hạn worker cũ hoàn thành muộn sau khi lease đã hết hạn. Vì vậy, hệ thống cần có thêm cơ chế nhận diện phiên xử lý hoặc kiểm tra tính hợp lệ của kết quả để tránh ghi nhận kết quả không còn phù hợp.



Hình 2.4: Cơ chế Claim Lease và Heartbeat trong hệ thống phân tán

Nhìn chung, claim lease và heartbeat là hai cơ chế quan trọng trong các hệ thống xử lý tác vụ phân tán. Chúng giúp tăng khả năng chịu lỗi, hỗ trợ phục hồi khi worker mất kết nối và đảm bảo tác vụ không bị bỏ sót trong quá trình vận hành.

2.5. Containerization và cô lập môi trường thực thi

2.5.1 Docker và cơ chế namespace/cgroup

Containerization là kỹ thuật đóng gói ứng dụng cùng các thư viện và phụ thuộc cần thiết vào một môi trường thực thi độc lập gọi là container. Docker là một nền tảng containerization phổ biến, giúp ứng dụng có thể chạy nhất quán trên nhiều môi trường khác nhau [15].

Về mặt kỹ thuật, Docker dựa trên hai cơ chế chính của Linux kernel là **namespace** và **cgroup**. Namespace được sử dụng để cô lập các không gian tài nguyên như tiến trình, mạng, hệ thống tệp và người dùng giữa container và host. Nhờ đó, tiến trình bên trong container hoạt động như trong một môi trường riêng biệt dù vẫn chia sẻ kernel với hệ điều hành host.

Trong khi đó, **cgroup** cho phép giới hạn và giám sát tài nguyên mà container được sử dụng, chẳng hạn CPU, bộ nhớ và số lượng tiến trình. Cơ chế này giúp ngăn một container chiếm dụng quá mức tài nguyên và ảnh hưởng đến các tiến trình khác trên cùng máy chủ.

So với máy ảo truyền thống, container nhẹ hơn vì không cần khởi động một hệ điều hành riêng cho mỗi môi trường thực thi. Tuy nhiên, do container vẫn chia sẻ kernel với host, mức độ an toàn phụ thuộc nhiều vào cấu hình cô lập và giới hạn tài nguyên được áp dụng.

2.5.2 Sandbox và bảo mật môi trường thực thi

Sandbox là cơ chế tạo ra một môi trường thực thi bị giới hạn nhằm giảm rủi ro khi chạy mã nguồn không tin cậy. Trong các hệ thống cho phép người dùng gửi chương trình để thực thi, sandbox giúp ngăn chương trình ảnh hưởng đến máy chủ, dữ liệu hệ thống hoặc các tác vụ khác.

Một sandbox hiệu quả cần đảm bảo các yêu cầu cơ bản như cô lập hệ thống tệp, kiểm soát truy cập mạng, giới hạn CPU và bộ nhớ, giới hạn số tiến trình con và áp dụng thời gian thực thi tối đa [16]. Các cơ chế này giúp hạn chế những hành vi không mong muốn như đọc dữ liệu ngoài phạm vi cho phép, tạo quá nhiều tiến trình hoặc chạy vô hạn.

Docker có thể được sử dụng để xây dựng sandbox nhờ khả năng kết hợp namespace, cgroup và các cấu hình giới hạn quyền. Tuy nhiên, Docker không tự động đảm bảo an toàn tuyệt đối nếu cấu hình không phù hợp. Vì vậy, khi dùng container để thực thi mã không tin cậy, cần giới hạn quyền truy cập, kiểm soát thư mục được mount, hạn chế mạng và đặt giới hạn tài nguyên rõ ràng.

Trong các hệ thống chấm bài tự động, sandbox giúp chương trình của người tham gia được chạy trong môi trường kiểm soát, vừa hỗ trợ tính công bằng của quá trình đánh giá, vừa bảo vệ hạ tầng chấm bài khỏi lỗi hoặc hành vi không mong muốn.

2.6. Bảng xếp hạng và Materialized View

2.6.1 Materialized View trong cơ sở dữ liệu quan hệ

Materialized view là cơ chế lưu trữ kết quả của một truy vấn đã được tính toán trước trong cơ sở dữ liệu quan hệ. Khác với view thông thường, vốn chỉ lưu định nghĩa truy vấn và thực thi lại mỗi khi được đọc, materialized view lưu kết quả dưới dạng dữ liệu vật lý. Nhờ đó, các truy vấn đọc có thể được thực hiện nhanh hơn, đặc biệt với các truy vấn tổng hợp phức tạp hoặc được truy cập thường xuyên [17].

Đánh đổi của materialized view là dữ liệu có thể không phản ánh ngay lập tức thay đổi mới nhất từ các bảng nguồn. Vì vậy, hệ thống cần có cơ chế làm mới phù hợp, chẳng hạn làm mới toàn bộ định kỳ, cập nhật tăng dần hoặc làm mới theo sự kiện khi dữ liệu nguồn thay đổi.

Trong các ứng dụng có tần suất đọc cao nhưng chấp nhận độ trễ cập nhật nhỏ, materialized view là giải pháp phù hợp để giảm chi phí tính toán tại thời điểm truy vấn. Bảng xếp hạng là một ví dụ điển hình, vì dữ liệu thường được đọc nhiều lần trong khi việc tính toán thứ hạng từ dữ liệu gốc có thể tốn kém.

2.6.2 Window Function và xếp hạng trong SQL

Window function là nhóm hàm SQL cho phép tính toán trên một tập các hàng liên quan mà không gộp chúng lại như GROUP BY [19]. Cơ chế này thường được sử dụng trong các bài toán cần tính thứ tự, chọn bản ghi đại diện hoặc xây dựng bảng xếp hạng.

Trong bài toán xếp hạng, ROW_NUMBER() được dùng để gán số thứ tự duy nhất cho từng hàng theo một tiêu chí sắp xếp xác định. Hàm này hữu ích khi cần chọn một bản ghi đại diện cho mỗi nhóm, chẳng hạn submission mới nhất hoặc submission có điểm cao nhất.

DENSE_RANK() được dùng để gán thứ hạng cho các hàng theo thứ tự sắp xếp. Các hàng có cùng giá trị xếp hạng sẽ nhận cùng một hạng, và hạng tiếp theo không bị bỏ trống. Cách đánh hạng này phù hợp với bảng xếp hạng trong các cuộc thi.

Ngoài ra, thao tác UPSERT dưới dạng INSERT ... ON CONFLICT DO UPDATE thường được sử dụng để duy trì các bảng kết quả đã tính toán trước. Cơ chế này cho phép thêm mới hoặc cập nhật bản ghi một cách nhất quán mà không cần xóa toàn bộ dữ liệu cũ trước khi ghi lại.

2.7. Công nghệ và nền tảng phát triển

2.7.1 Ngôn ngữ Go và mô hình đồng thời Goroutine

Go là ngôn ngữ lập trình biên dịch tĩnh do Google phát triển, phù hợp cho việc xây dựng các dịch vụ mạng, hệ thống backend và ứng dụng cần xử lý đồng thời cao [18]. Một đặc điểm nổi bật của Go là cơ chế đồng thời dựa trên **goroutine**.

Goroutine là đơn vị thực thi nhẹ được quản lý bởi Go runtime. So với luồng hệ điều hành truyền thống, goroutine có chi phí khởi tạo và quản lý thấp hơn, cho phép một tiến trình Go xử lý nhiều tác vụ đồng thời. Go runtime tự động lập lịch các goroutine lên các luồng hệ điều hành, giúp đơn giản hóa việc xây dựng các dịch vụ có nhiều tác vụ I/O như API server, xử lý request hoặc tác vụ nền.

2.7.2 PostgreSQL, JSONB và Window Function

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, hỗ trợ SQL chuẩn và nhiều tính năng nâng cao phục vụ các hệ thống ứng dụng hiện đại [19]. Bên cạnh các kiểu dữ liệu quan hệ truyền thống, PostgreSQL hỗ trợ kiểu dữ liệu **JSONB**, cho phép lưu trữ dữ liệu JSON ở dạng nhị phân và hỗ trợ truy vấn, đánh chỉ mục trên các trường bên trong.

JSONB phù hợp với các dữ liệu có cấu trúc linh hoạt, chẳng hạn cấu hình, metadata hoặc các thuộc tính có thể thay đổi giữa các bản ghi. Ngoài ra, PostgreSQL còn hỗ trợ **window function** và **Common Table Expression (CTE)**, giúp xây dựng các truy vấn phân tích, xếp hạng và tổng hợp dữ liệu phức tạp một cách rõ ràng hơn.

2.7.3 React và mô hình Single Page Application

React là thư viện JavaScript do Meta phát triển, được sử dụng để xây dựng giao diện người dùng theo mô hình component [20]. Mỗi component biểu diễn một phần giao diện độc lập, giúp mã nguồn frontend dễ tổ chức, tái sử dụng và bảo trì.

Single Page Application (SPA) là mô hình ứng dụng web trong đó trang được tải một lần, sau đó việc điều hướng và cập nhật giao diện được thực hiện phía client mà không cần tải lại toàn bộ trang. Mô hình này phù hợp với các hệ thống có nhiều trạng thái tương tác và cần cập nhật dữ liệu thường xuyên.

Trong các ứng dụng React hiện đại, các thư viện như **React Query** thường được sử dụng để quản lý trạng thái bất đồng bộ, cache dữ liệu, polling định kỳ và làm mới dữ liệu khi cần. Bên cạnh đó, **Server-Sent Events (SSE)** là cơ chế cho phép server gửi cập nhật một chiều đến client qua kết nối HTTP, phù hợp với các trường hợp cần hiển thị trạng thái thay đổi theo thời gian thực.

2.7.4 Python và hệ sinh thái Machine Learning

Python là ngôn ngữ lập trình được sử dụng rộng rãi trong lĩnh vực học máy và trí tuệ nhân tạo nhờ cú pháp đơn giản cùng hệ sinh thái thư viện phong phú [21]. Các thư viện như PyTorch, TensorFlow, scikit-learn, NumPy và Pandas hỗ trợ nhiều tác vụ từ xử lý dữ liệu, xây dựng mô hình học máy đến huấn luyện và suy luận mô hình học sâu.

Nhiều framework học sâu trong Python hỗ trợ tăng tốc tính toán bằng GPU thông qua CUDA, giúp quá trình huấn luyện và suy luận mô hình đạt hiệu năng cao hơn so với chỉ sử dụng CPU. Ngoài ra, **PyPI** là kho lưu trữ package Python phổ biến, cho phép phân phối và cài đặt phần mềm thông qua công cụ pip, giúp đơn giản hóa quá trình triển khai và quản lý phụ thuộc của các ứng dụng Python.

2.7.5 Tổng hợp lựa chọn công nghệ

Bảng 2.1: Công nghệ sử dụng và lý do lựa chọn

Thành phần	Công nghệ	Lý do lựa chọn
Backend API	Go 1.23, Echo	Goroutine hỗ trợ xử lý đồng thời hiệu quả; tính năng biên dịch tĩnh giúp phát hiện lỗi từ sớm.
Cơ sở dữ liệu	PostgreSQL 15	Hỗ trợ Window function, CTE, JSONB và UPSERT, đặc biệt tối ưu cho các bài toán truy vấn bảng xếp hạng (leaderboard) phức tạp.
Message queue	Redis Streams	Cung cấp hệ thống log có thứ tự và bền vững; các thao tác XRange/XDel đảm bảo tính toàn vẹn (atomic) thông qua Lua script.
Object storage	MinIO (S3-compatible)	Khả năng tự lưu trữ (self-hosted); hỗ trợ presigned URL và tối ưu cho việc lưu trữ, truyền tải các tệp tin có dung lượng lớn.
Frontend	React 19, TypeScript, Vite	Xây dựng ứng dụng SPA kết hợp React Query polling; TypeScript

		giúp kiểm tra kiểu tĩnh, hạn chế lỗi trong quá trình phát triển.
Volunteer agent	Python 3.11	Tương thích hoàn toàn với hệ sinh thái Học máy (Machine Learning); dễ dàng đóng gói và phân phối thông qua PyPI.
Reverse proxy	Caddy	Tự động cấp phát chứng chỉ HTTPS; cấu hình đơn giản và gọn nhẹ.
Containerization	Docker	Cô lập mã nguồn không tin cậy (sandbox) an toàn thông qua cơ chế namespace và cgroup.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu

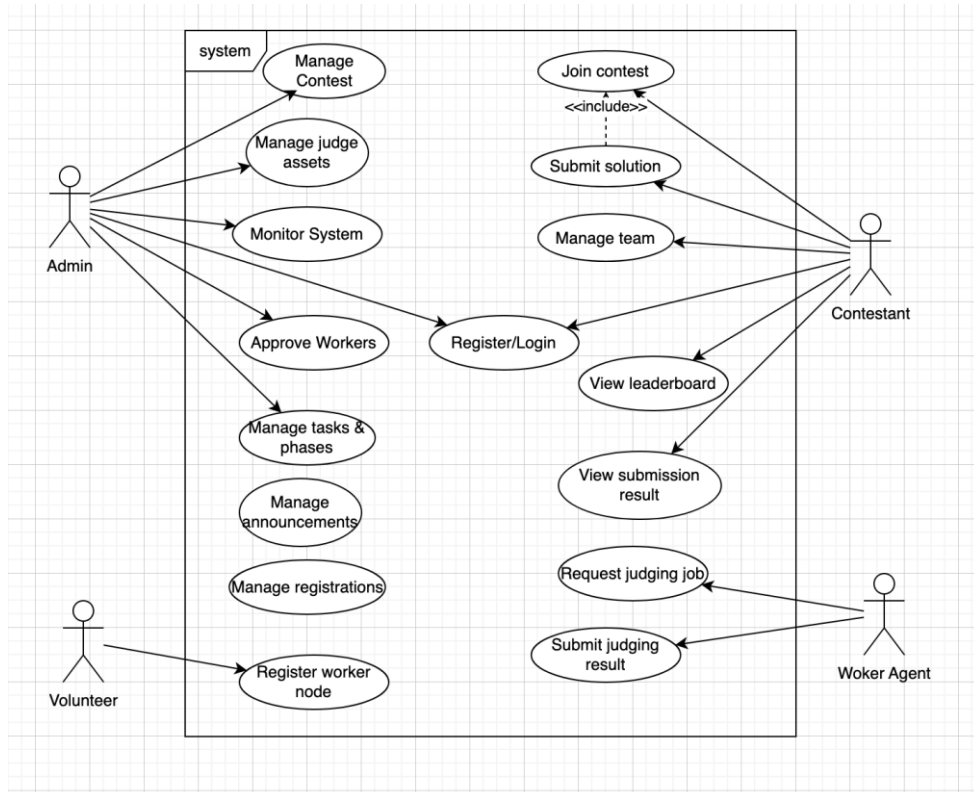
3.1.1 Các vai trò người dùng

BKDNAIOJ phục vụ ba nhóm tác nhân chính với các quyền hạn và trách nhiệm khác nhau trong hệ thống, được minh họa trong Hình 3.1.

Admin là nhóm người dùng quản trị, bao gồm ban tổ chức cuộc thi và quản trị viên hệ thống. Nhóm này chịu trách nhiệm cấu hình cuộc thi, quản lý tài nguyên chấm bài, theo dõi quá trình vận hành và điều phối mạng lưới máy chấm tình nguyện.

Contestant là người tham gia cuộc thi. Thí sinh có thể đăng ký tham gia, xem đề bài, nộp bài và theo dõi kết quả trong phạm vi các cuộc thi mà mình được phép truy cập.

Máy chấm tình nguyện là các tiến trình thực thi chấm bài được triển khai trên các máy tính tham gia hệ thống. Khác với hai nhóm trên, worker không tương tác thông qua giao diện người dùng mà giao tiếp với hệ thống thông qua tập API chuyên biệt nhằm nhận tác vụ, thực thi đánh giá và gửi kết quả.



Hình 3.1: Sơ đồ use case tổng quan

Việc tách biệt ba vai trò trên giúp hệ thống xây dựng cơ chế phân quyền rõ ràng, đảm bảo mỗi tác nhân chỉ có thể truy cập các chức năng phù hợp với trách nhiệm của mình.

3.1.2 Yêu cầu chức năng đối với Admin

Các chức năng của Admin được chia thành năm nhóm nghiệp vụ chính nhằm hỗ trợ toàn bộ quá trình tổ chức và vận hành cuộc thi.

Quản lý cuộc thi

Admin có quyền tạo mới, chỉnh sửa và cấu hình các cuộc thi trên hệ thống. Các thông tin cấu hình bao gồm thời gian diễn ra cuộc thi, hình thức tham gia (cá nhân hoặc đội), số lượng thành viên tối đa của mỗi đội và các quy định liên quan. Bên cạnh đó, Admin chịu trách nhiệm quản lý các bài toán thuộc cuộc thi và cấu hình các giai đoạn đánh giá tương ứng, bao gồm thời gian mở/đóng, giới hạn số lần nộp bài và các tham số phục vụ quá trình chấm điểm.

Quản lý tài nguyên đánh giá

Admin quản lý toàn bộ tài nguyên phục vụ quá trình đánh giá, bao gồm bộ dữ liệu kiểm thử, dữ liệu tham chiếu và các chương trình chấm điểm. Hệ thống hỗ trợ tổ chức các bộ dữ liệu theo từng giai đoạn đánh giá nhằm đáp ứng các yêu cầu khác nhau của cuộc thi. Đồng thời, hệ thống cung cấp cơ chế kiểm tra tính toàn vẹn của dữ liệu trước khi đưa vào sử dụng.

Quản lý đăng ký tham gia

Admin có thể theo dõi và xử lý các yêu cầu đăng ký tham gia cuộc thi, quản lý danh sách đội thi và thành viên, đồng thời thực hiện các thao tác quản trị đối với các trường hợp vi phạm quy định. Mọi thay đổi quan trọng đều được ghi nhận nhằm phục vụ công tác theo dõi và kiểm tra khi cần thiết.

Quản lý Máy chấm tình nguyện

Admin chịu trách nhiệm quản lý mạng lưới máy chấm tình nguyện tham gia quá trình chấm bài. Các chức năng bao gồm phê duyệt worker mới, theo dõi trạng thái hoạt động, giám sát các tác vụ đang được thực thi và cấu hình các chế độ vận hành phù hợp với từng worker. Ngoài ra, Admin có thể tạm ngừng hoặc thu hồi quyền tham gia của worker khi phát hiện sự cố hoặc vi phạm chính sách vận hành.

Quản lý thông tin và truyền thông

Admin có thể đăng tải các thông báo liên quan đến cuộc thi, cập nhật thông tin trên hệ thống và truyền đạt các nội dung cần thiết đến người tham gia. Chức

năng này hỗ trợ quá trình trao đổi thông tin giữa ban tổ chức và thí sinh trong suốt thời gian diễn ra cuộc thi.

3.1.3 Yêu cầu chức năng đối với Contestant

Contestant là nhóm người dùng chính của hệ thống, tương tác với nền tảng trong suốt quá trình đăng ký, tham gia cuộc thi và theo dõi kết quả. Các chức năng của Contestant được chia thành bốn nhóm chính.

Quản lý tài khoản

Thí sinh có thể đăng ký tài khoản, đăng nhập và thực hiện các chức năng quản lý thông tin cá nhân. Hệ thống hỗ trợ cơ chế khôi phục mật khẩu nhằm đảm bảo khả năng truy cập tài khoản khi cần thiết.

Tham gia cuộc thi

Thí sinh có thể xem thông tin các cuộc thi đang mở đăng ký và thực hiện đăng ký tham gia theo hình thức được ban tổ chức cho phép. Đối với các cuộc thi theo hình thức đội, hệ thống hỗ trợ tạo đội, quản lý thành viên và thực hiện các thao tác liên quan đến quá trình đăng ký trước khi cuộc thi bắt đầu.

Nộp bài

Thí sinh có thể truy cập đề bài, xem các tài nguyên được công bố và nộp bài theo định dạng quy định của từng bài toán. Hệ thống tiếp nhận bài nộp, kiểm tra các điều kiện hợp lệ và theo dõi trạng thái xử lý của từng submission trong suốt quá trình chấm bài.

Theo dõi kết

Thí sinh có thể xem lịch sử các lần nộp bài, theo dõi điểm số đạt được và truy cập các bảng xếp hạng liên quan đến cuộc thi. Hệ thống cung cấp thông tin về trạng thái xử lý, kết quả đánh giá và vị trí xếp hạng nhằm hỗ trợ người tham gia theo dõi tiến trình của mình trong suốt thời gian diễn ra cuộc thi.

3.1.4 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, BKDNAIOJ cần đáp ứng một số yêu cầu phi chức năng nhằm đảm bảo khả năng vận hành ổn định, an toàn và mở rộng trong môi trường tổ chức các cuộc thi AI. Các yêu cầu này cùng với định hướng giải pháp được trình bày trong Bảng 3.1.

Đặc biệt, hệ thống được thiết kế để hoạt động trong môi trường volunteer computing, nơi các worker có thể tham gia hoặc rời khỏi hệ thống bất kỳ lúc nào. Do

đó, khả năng chịu lỗi và phục hồi tác vụ là một yêu cầu quan trọng nhằm đảm bảo quá trình chấm bài không bị gián đoạn khi xảy ra sự cố trên các worker.

Bảng 3.1: Yêu cầu phi chức năng và giải pháp kiến trúc tương ứng

Tiêu chí	Yêu cầu	Giải pháp thiết kế
Hiệu năng	Hỗ trợ nhiều bài nộp đồng thời mà không làm suy giảm đáng kể khả năng phản hồi của hệ thống	Tách biệt luồng tải tệp, luồng xử lý nghiệp vụ và luồng chấm bài thông qua kiến trúc phân tán
Độ tin cậy	Đảm bảo tác vụ không bị mất khi worker gặp sự cố hoặc mất kết nối	Áp dụng cơ chế lease, heartbeat và phục hồi tác vụ tự động
Tính nhất quán	Đảm bảo mỗi submission chỉ được xử lý bởi một worker tại một thời điểm	Sử dụng cơ chế cấp phát tác vụ nguyên tử và kiểm soát trạng thái thực thi
Bảo mật	Bảo vệ hệ thống trước các truy cập trái phép và mã nguồn không tin cậy	Cơ chế xác thực, phân quyền và môi trường thực thi cô lập
Khả năng mở rộng	Cho phép mở rộng năng lực chấm bài bằng cách bổ sung worker mới	Mô hình worker phân tán và cơ chế đăng ký động
Khả năng vận hành	Hỗ trợ giám sát và quản lý hệ thống trong thời gian thực	Giao diện quản trị tập trung và các công cụ theo dõi trạng thái hệ thống

Các yêu cầu phi chức năng trên đóng vai trò định hướng cho các quyết định thiết kế của BKDNAIOJ. Trong đó, khả năng chịu lỗi, tính nhất quán và khả năng mở rộng là ba tiêu chí có ảnh hưởng lớn nhất đến kiến trúc tổng thể của hệ thống, đặc biệt trong bối cảnh sử dụng mô hình máy chấm tình nguyện để thực hiện các tác vụ chấm bài phân tán.

3.1.5 Đặc tả Use Case

Bảng 3.2: Đặc tả UC-01 - Nộp bài

Trường	Nội dung
Tên Use Case	Nộp bài

Mã Use Case	UC-01
Actor	Contestant
Mô tả	Thí sinh nộp bài cho một bài toán trong giai đoạn thi đang mở.
Trigger	Thí sinh chọn tệp và nhấn nút "Nộp bài".
Điều kiện tiên quyết	Thí sinh đã đăng nhập; đã đăng ký tham gia cuộc thi; phase đang mở nhận bài; chưa vượt quá số lượt nộp cho phép.
Điều kiện hậu	Submission được ghi nhận và chuyển sang trạng thái chờ chấm hoặc đang chấm.
Luồng cơ bản	1. Thí sinh truy cập Phase Hub. 2. Thí sinh chọn bài toán cần nộp. 3. Thí sinh chọn tệp theo yêu cầu của bài toán. 4. Hệ thống kiểm tra tính hợp lệ của yêu cầu nộp bài. 5. Hệ thống tiếp nhận submission. 6. Submission được đưa vào hàng đợi xử lý. 7. Giao diện hiển thị trạng thái submission.
Luồng thay thế	Đối với phase dạng output-only, hệ thống chấm trực tiếp trên file kết quả được nộp.
Luồng ngoại lệ	Phase đã đóng; vượt quá số lượt nộp; định dạng tệp không hợp lệ; kích thước tệp vượt giới hạn cho phép.

Bảng 3.3: Đặc tả UC-02 - Đăng ký tham gia cuộc thi

Trường	Nội dung
Tên Use Case	Đăng ký tham gia cuộc thi
Mã Use Case	UC-02
Actor	Contestant
Mô tả	Thí sinh đăng ký tham gia một cuộc thi dưới hình thức cá nhân hoặc đội.
Trigger	Thí sinh nhấn nút "Đăng ký tham gia".
Điều kiện tiên quyết	Thí sinh đã đăng nhập; cuộc thi đang trong thời gian nhận đăng ký.
Điều kiện hậu	Contest entry được tạo thành công.
Luồng cơ bản	1. Thí sinh truy cập trang chi tiết cuộc thi.

	2. Thí sinh chọn hình thức tham gia. 3. Hệ thống kiểm tra điều kiện đăng ký. 4. Hệ thống tạo contest entry. 5. Thí sinh được phép tham gia các phase của cuộc thi.
Luồng thay thế	Nếu cuộc thi tổ chức theo đội, thí sinh tạo đội hoặc tham gia đội trước khi đăng ký.
Luồng ngoại lệ	Đã đăng ký trước đó; hết thời gian đăng ký; không đáp ứng yêu cầu về đội thi.

Bảng 3.4: Đặc tả UC-03 - Đăng ký máy chấm tình nguyện

Trường	Nội dung
Tên Use Case	Đăng ký máy chấm tình nguyện
Mã Use Case	UC-03
Actor	Volunteer
Mô tả	Tình nguyện viên đăng ký máy tính tham gia hệ thống chấm bài phân tán.
Trigger	Người dùng chạy worker setup wizard và gửi yêu cầu đăng ký.
Điều kiện tiên quyết	Worker agent đã được cài đặt và cấu hình thành công.
Điều kiện hậu	Worker được ghi nhận trong hệ thống ở trạng thái chờ phê duyệt.
Luồng cơ bản	1. Tình nguyện viên cài đặt worker agent. 2. Worker thực hiện benchmark phần cứng. 3. Worker gửi thông tin cấu hình tới hệ thống. 4. Hệ thống tạo bản ghi worker ở trạng thái chờ phê duyệt.
Luồng thay thế	Worker có thể cập nhật thông tin cấu hình trước khi được phê duyệt.
Luồng ngoại lệ	Không thể kết nối tới hệ thống; thông tin đăng ký không hợp lệ.

Bảng 3.5: Đặc tả UC-04 - Phê duyệt máy chấm tình nguyện

Trường	Nội dung
Tên Use Case	Phê duyệt máy chấm tình nguyện

Mã Use Case	UC-04
Actor	Admin
Mô tả	Quản trị viên phê duyệt hoặc từ chối worker tham gia cụm chấm bài.
Trigger	Có worker mới ở trạng thái chờ phê duyệt.
Điều kiện tiên quyết	Worker đã đăng ký thành công.
Điều kiện hậu	Worker được phép hoặc không được phép tham gia nhận job.
Luồng cơ bản	1. Admin truy cập trang quản lý worker. 2. Hệ thống hiển thị danh sách worker chờ duyệt. 3. Admin xem thông tin cấu hình phần cứng. 4. Admin phê duyệt worker. 5. Worker được kích hoạt và có thể nhận job.
Luồng thay thế	Admin cấu hình thêm quyền hoặc giới hạn tài nguyên trước khi phê duyệt.
Luồng ngoại lệ	Admin từ chối worker; worker không còn khả dụng trong quá trình xét duyệt.

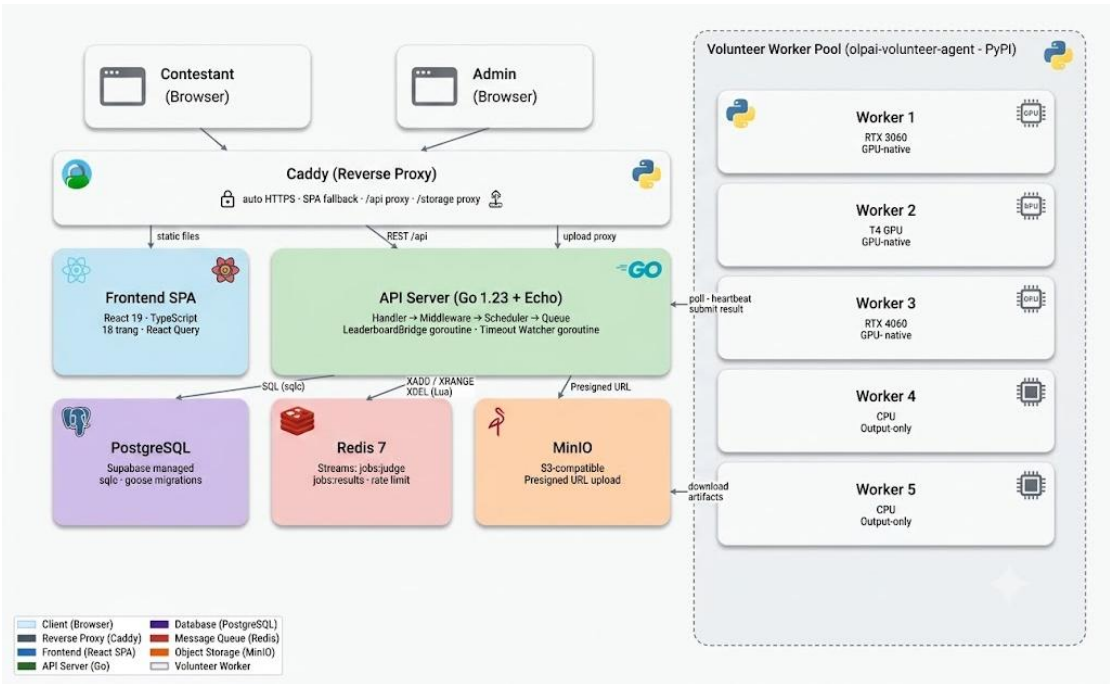
Bảng 3.6: Đặc tả UC-05 - Nhận job và chấm bài

Trường	Nội dung
Tên Use Case	Nhận job và chấm bài
Mã Use Case	UC-05
Actor	Máy chấm tình nguyện
Mô tả	Worker nhận tác vụ từ hệ thống, thực hiện chấm bài và gửi kết quả.
Trigger	Worker chủ động yêu cầu nhận việc.
Điều kiện tiên quyết	Worker đã được phê duyệt và còn khả năng nhận thêm job.
Điều kiện hậu	Submission được chấm thành công hoặc ghi nhận lỗi thực thi.
Luồng cơ bản	1. Worker yêu cầu nhận job. 2. Hệ thống phân công một job phù hợp. 3. Worker tải các tài nguyên cần thiết. 4. Worker thực thi quá trình chấm bài.

	5. Worker gửi kết quả về hệ thống. 6. Hệ thống cập nhật trạng thái submission và leaderboard.
Luồng thay thế	Không có job phù hợp để giao cho worker.
Luồng ngoại lệ	Lỗi thực thi; vượt quá thời gian cho phép; worker mất kết nối trong quá trình xử lý.

3.2. Sơ đồ kiến trúc tổng thể

Hệ thống BKDNAIOJ được thiết kế theo kiến trúc phân lớp (layered architecture), trong đó các thành phần được tách biệt theo chức năng và giao tiếp với nhau thông qua RESTful API và cơ chế truyền thông điệp sử dụng Redis Streams. Kiến trúc này giúp nâng cao tính mô-đun, khả năng mở rộng và khả năng bảo trì của hệ thống. Tổng quan kiến trúc được minh họa trong Hình 3.2.



Hình 3.2: Kiến trúc tổng thể hệ thống BKDNAIOJ

Tại lớp giao tiếp bên ngoài, toàn bộ yêu cầu từ người dùng, bao gồm thí sinh và quản trị viên, được tiếp nhận bởi Caddy. Thành phần này đóng vai trò là một reverse proxy, đồng thời cung cấp cơ chế quản lý chứng chỉ TLS phục vụ kết nối bảo mật. Dựa trên quy tắc định tuyến theo đường dẫn, Caddy phân phối các yêu cầu truy cập giao diện người dùng đến ứng dụng React (Single Page Application) và chuyển tiếp các yêu cầu API đến hệ thống xử lý nghiệp vụ phía máy chủ.

Lớp xử lý trung tâm (Core API) được phát triển bằng ngôn ngữ Go và chịu trách nhiệm thực hiện toàn bộ nghiệp vụ của hệ thống. Thành phần này đóng vai trò điều phối các hoạt động chính như quản lý cuộc thi, tiếp nhận bài nộp, phân phối tác vụ chấm điểm và cập nhật bảng xếp hạng. Để đáp ứng các yêu cầu lưu trữ khác nhau, Core API tương tác với ba hệ thống lưu trữ chuyên biệt:

- **PostgreSQL**: lưu trữ dữ liệu quan hệ của hệ thống, bao gồm thông tin người dùng, cấu hình cuộc thi, trạng thái bài nộp và lịch sử hoạt động.
- **MinIO**: cung cấp dịch vụ lưu trữ đối tượng (object storage) cho các tệp có dung lượng lớn như bài nộp của thí sinh, bộ dữ liệu đánh giá và các chương trình chấm điểm.
- **Redis Streams**: đảm nhiệm vai trò hàng đợi tác vụ phân tán (distributed job queue), hỗ trợ trao đổi thông tin giữa Core API và các worker.

Lớp thực thi đánh giá (Worker Pool) bao gồm tập hợp các máy tính phân tán tham gia quá trình chấm bài. Mỗi máy được cài đặt tác nhân nền *olpai-volunteer-agent*, có nhiệm vụ định kỳ gửi yêu cầu đến Core API để nhận tác vụ mới. Tùy theo yêu cầu của bài toán và cấu hình của worker, quá trình thực thi có thể được triển khai trong môi trường Docker nhằm tăng cường tính cô lập hoặc được thực hiện trực tiếp trên hệ điều hành trong trường hợp cần khai thác tối đa năng lực phần cứng, đặc biệt là GPU.

Sau khi hoàn thành tác vụ, worker gửi kết quả về Redis Streams thông qua luồng dữ liệu jobs. Các kết quả này được xử lý bất đồng bộ bởi thành phần *LeaderboardBridge*, từ đó cập nhật trạng thái bài nộp và bảng xếp hạng của cuộc thi.

Kiến trúc phân tán của BKDNAIOJ được xây dựng nhằm đáp ứng hai yêu cầu quan trọng của các hệ thống tổ chức cuộc thi AI.

- Thứ nhất, kiến trúc cho phép tách biệt hoàn toàn quá trình xử lý nghiệp vụ và quá trình thực thi mô hình. Các tác vụ tính toán có chi phí cao, đặc biệt là các bài toán suy luận sử dụng GPU, được chuyển giao cho cụm worker thay vì thực hiện trực tiếp trên máy chủ API. Cách tiếp cận này giúp giảm tải cho lớp xử lý trung tâm và tạo điều kiện mở rộng năng lực tính toán bằng cách bổ sung thêm worker khi cần thiết.
- Thứ hai, hệ thống được trang bị các cơ chế giám sát và phục hồi tác vụ nhằm tăng khả năng chịu lỗi. Thông qua cơ chế *lease* và *heartbeat*, trạng thái hoạt động của các worker được theo dõi liên tục trong suốt quá trình thực thi. Khi một worker mất kết nối hoặc không phản hồi trong khoảng thời gian quy định,

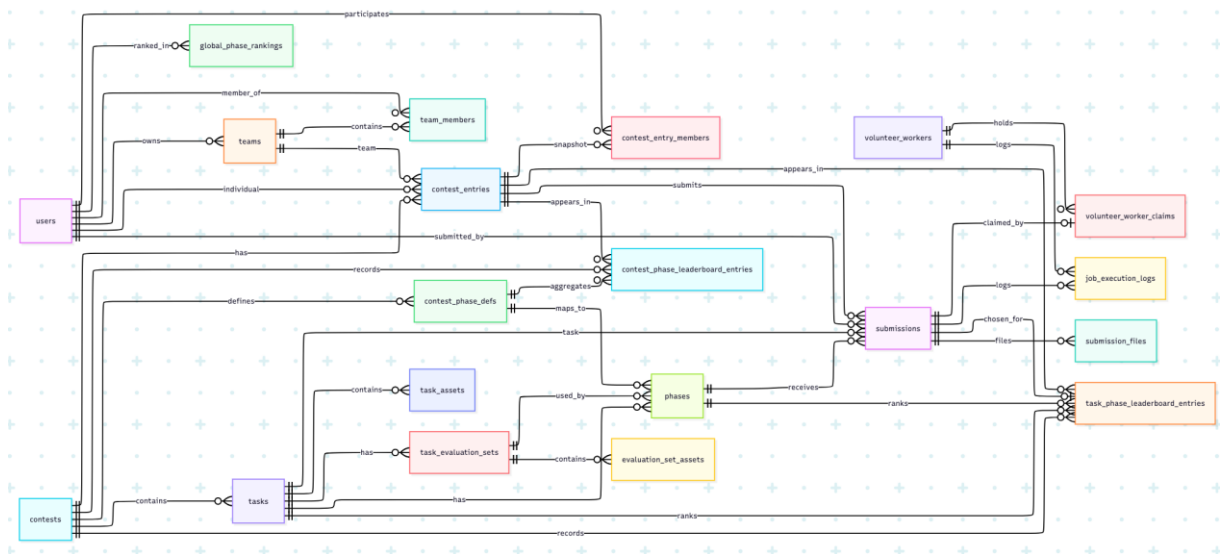
tác vụ đang xử lý có thể được thu hồi và đưa trở lại hàng đợi để phân phối cho một worker khác. Cơ chế này giúp hạn chế ảnh hưởng của các sự cố phần cứng hoặc mạng đến quá trình chấm bài.

3.3. Thiết kế Cơ sở Dữ liệu

3.3.1 ER Diagram tổng thể và nguyên tắc thiết kế

Cơ sở dữ liệu của BKDNAIOJ được thiết kế theo hướng phân tách các thực thể dựa trên từng domain nghiệp vụ của hệ thống. Các thực thể chính bao gồm người dùng, đội thi, cuộc thi, bài toán, phase đánh giá, submission, leaderboard và worker phân tán. Quan hệ giữa các thực thể được xây dựng nhằm hỗ trợ toàn bộ quy trình tổ chức và vận hành một cuộc thi AI, từ khâu đăng ký tham gia, nộp bài, đánh giá kết quả cho đến công bố bảng xếp hạng.

Do số lượng bảng và quan hệ tương đối lớn, Hình 3.3 chỉ trình bày ER Diagram tổng thể ở mức khái quát, tập trung vào các bảng cốt lõi và các quan hệ chính giữa chúng. Các nhóm bảng chi tiết sẽ được trình bày riêng theo từng domain nghiệp vụ ở các phần tiếp theo nhằm đảm bảo tính dễ đọc của sơ đồ.



Hình 3.3: ER Diagram tổng thể rút gọn của hệ thống

Một nguyên tắc quan trọng trong thiết kế cơ sở dữ liệu là đảm bảo tính toàn vẹn dữ liệu không chỉ ở tầng ứng dụng mà còn trực tiếp tại tầng cơ sở dữ liệu. Đối với các thực thể có quan hệ phụ thuộc theo ngữ cảnh cuộc thi, hệ thống sử dụng các ràng buộc khóa ngoại kết hợp (*composite foreign key*) để đảm bảo dữ liệu luôn thuộc đúng phạm vi contest tương ứng.

Ví dụ, một submission không chỉ tham chiếu đến một task thông qua `task_id`, mà còn phải đảm bảo task đó thuộc đúng cuộc thi của submission. Cách thiết kế này giúp

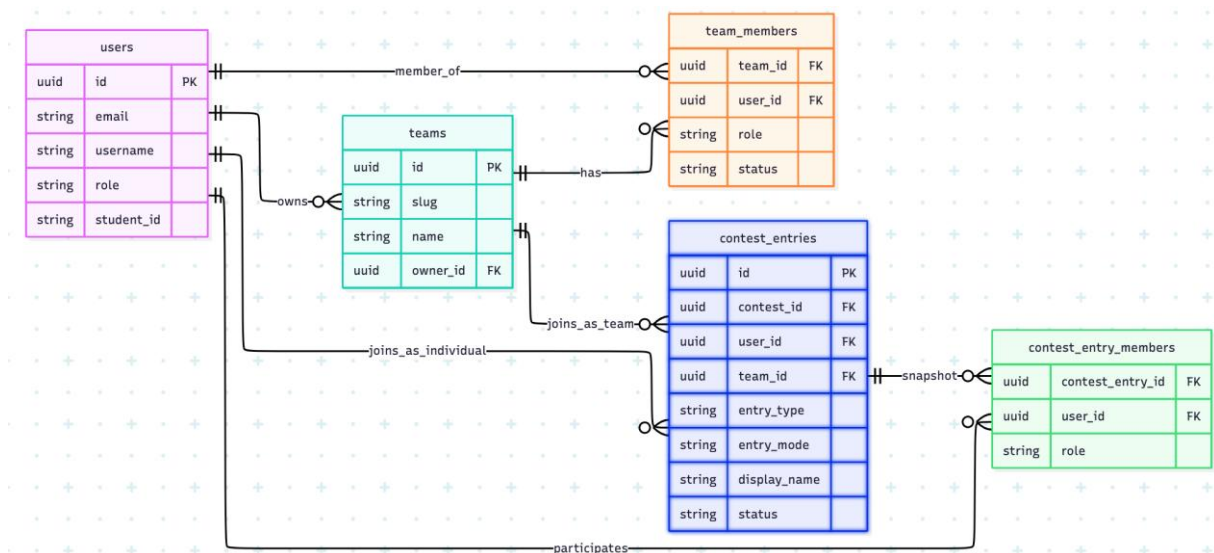
hạn chế các lỗi không nhất quán dữ liệu giữa nhiều cuộc thi khác nhau, đặc biệt trong bối cảnh hệ thống có thể vận hành đồng thời nhiều contest.

3.3.2 Các nhóm bảng theo domain nghiệp vụ

Nhóm người dùng và đội thi

Nhóm này gồm các bảng users, teams, team_members, contest_entries và contest_entry_members, chịu trách nhiệm quản lý thông tin tài khoản, đội thi và quá trình tham gia cuộc thi. Bảng users lưu trữ thông tin người dùng và vai trò tương ứng trong hệ thống. Bảng teams quản lý các đội thi, trong khi team_members biểu diễn quan hệ giữa người dùng và đội.

Bảng contest_entries đại diện cho một thực thể tham gia cuộc thi, có thể là cá nhân hoặc đội thi. Bảng contest_entry_members lưu thông tin thành viên của entry tại thời điểm đăng ký, giúp đảm bảo kết quả cuộc thi luôn gắn với đúng danh sách thành viên đã được xác nhận



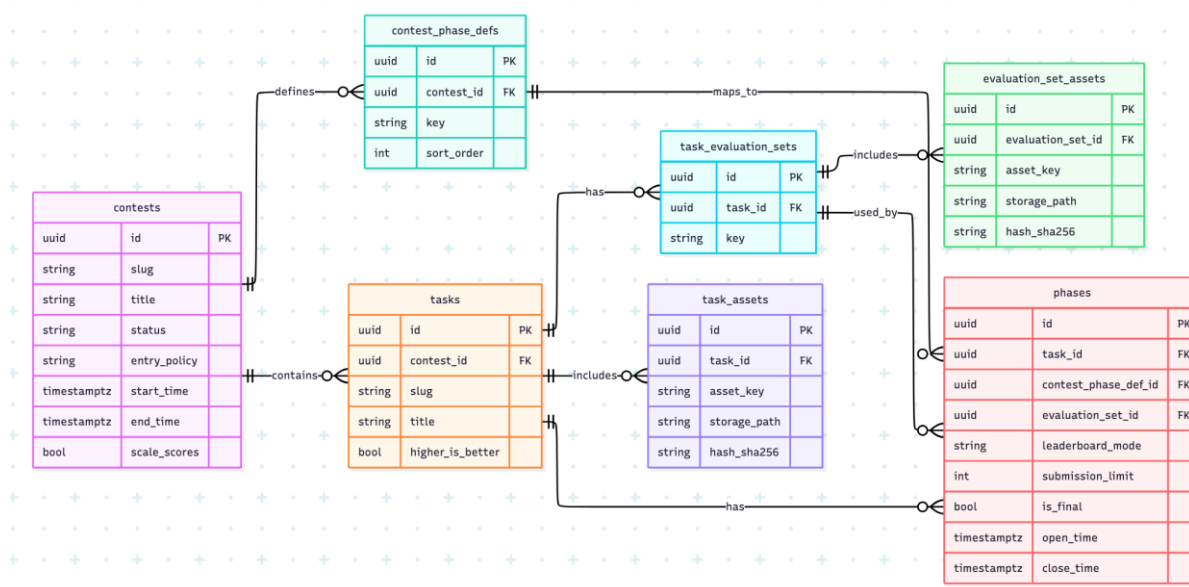
Hình 3.4: ER Diagram nhóm người dùng, đội thi và tham gia cuộc thi

Nhóm cuộc thi, bài toán và phase

Nhóm này là thành phần trung tâm của hệ thống, bao gồm các bảng contests, contest_phase_defs, tasks, task_evaluation_sets, evaluation_set_assets, task_assets và phases.

Bảng contests lưu trữ thông tin và cấu hình của từng cuộc thi. Mỗi cuộc thi bao gồm nhiều bài toán được quản lý bởi bảng tasks. Các loại phase sử dụng trong cuộc thi được định nghĩa tại bảng contest_phase_defs, trong khi bảng phases mô tả các giai đoạn đánh giá cụ thể của từng bài toán.

Các tài nguyên phục vụ quá trình chấm bài được quản lý thông qua `task_evaluation_sets`, `evaluation_set_assets` và `task_assets`. Thiết kế này cho phép tách biệt dữ liệu đánh giá với dữ liệu mô tả bài toán, đồng thời hỗ trợ tái sử dụng tài nguyên giữa các phase khác nhau.

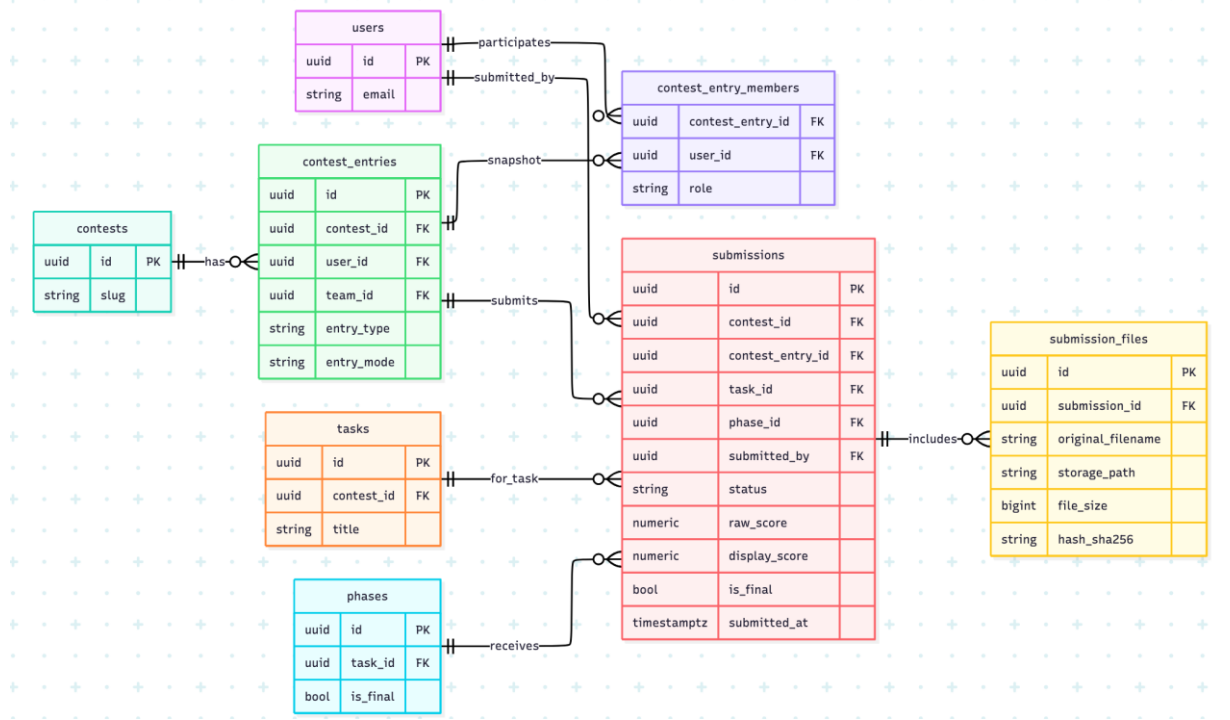


Hình 3.5: ER Diagram nhóm cuộc thi, bài toán, phase và tài nguyên đánh giá

Nhóm tham gia và nộp bài

Nhóm này bao gồm các bảng submissions và submission_files, mô hình hóa quá trình nộp bài của thí sinh. Mỗi submission được liên kết với một entry tham gia, một bài toán và một phase đánh giá cụ thể.

Bảng submissions lưu thông tin về trạng thái xử lý, điểm số và thời điểm nộp bài. Bảng submission_files quản lý các tệp được người dùng tải lên, bao gồm thông tin lưu trữ, kích thước tệp và dữ liệu kiểm tra toàn vẹn. Việc tách riêng hai bảng giúp hệ thống hỗ trợ linh hoạt các bài toán yêu cầu nhiều tệp đầu vào.

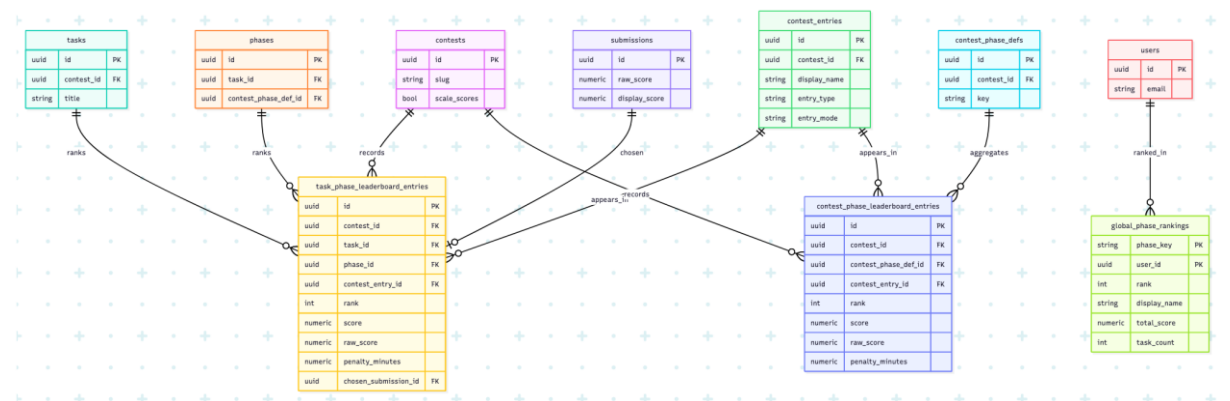


Hình 3.6: ER Diagram nhóm tham gia và nộp bài

Nhóm leaderboard

Nhóm leaderboard bao gồm các bảng `task_phase_leaderboard_entries`, `contest_phase_leaderboard_entries` và `global_phase_rankings`. Các bảng này lưu trữ dữ liệu xếp hạng đã được tính toán sẵn nhằm tối ưu hiệu năng truy vấn.

Trong đó, `task_phase_leaderboard_entries` lưu kết quả theo từng bài toán và phase cụ thể; `contest_phase_leaderboard_entries` tổng hợp kết quả ở phạm vi cuộc thi; còn `global_phase_rankings` lưu dữ liệu xếp hạng ở phạm vi toàn hệ thống. Thiết kế này giúp giảm chi phí tính toán khi hiển thị bảng xếp hạng cho người dùng.

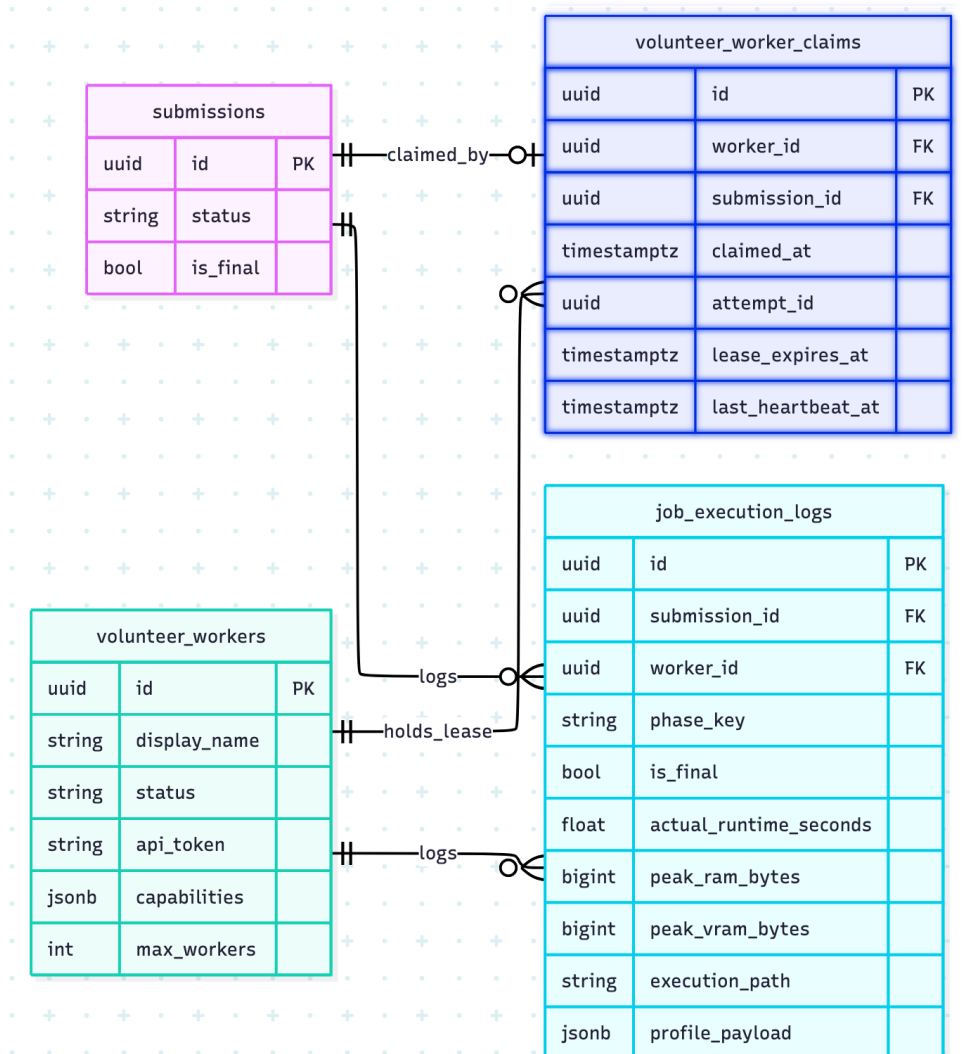


Hình 3.7: ER Diagram nhóm leaderboard

Nhóm worker phân tán

Nhóm này phục vụ cơ chế đánh giá bài làm trên hạ tầng worker phân tán, bao gồm các bảng `volunteer_workers`, `volunteer_worker_claims` và `job_execution_logs`.

Bảng `volunteer_workers` lưu thông tin các worker đã đăng ký tham gia hệ thống. Bảng `volunteer_worker_claims` theo dõi các tác vụ đang được worker tiếp nhận xử lý, hỗ trợ cơ chế lease nhằm tránh việc một submission được xử lý đồng thời bởi nhiều worker. Bảng `job_execution_logs` lưu lịch sử thực thi của các tác vụ chấm bài, bao gồm thời gian thực thi và thông tin sử dụng tài nguyên.



Hình 3.8: ER Diagram nhóm worker phân tán

3.3.3 Các quyết định thiết kế chính

Trong quá trình xây dựng mô hình dữ liệu, một số quyết định thiết kế quan trọng đã được đưa ra nhằm đáp ứng các yêu cầu về tính nhất quán, khả năng mở rộng và hiệu năng của hệ thống.

❖ Sử dụng Contest Entry làm thực thể trung tâm

Thay vì gắn trực tiếp kết quả cuộc thi với đội thi (*team*), hệ thống sử dụng thực thể *contest entry* làm đơn vị tham gia và xếp hạng. Mỗi *contest entry* đại diện cho một lần tham gia cụ thể của cá nhân hoặc đội thi trong một cuộc thi.

Cách tiếp cận này cho phép lưu trữ độc lập thành phần tham gia tại thời điểm đăng ký thông qua bảng thành viên của *entry*. Nhờ đó, các thay đổi về thành viên đội sau khi cuộc thi kết thúc không ảnh hưởng đến dữ liệu lịch sử và kết quả đã được ghi nhận trước đó. Thiết kế này đặc biệt phù hợp với các cuộc thi được tổ chức định kỳ, nơi thành phần đội có thể thay đổi giữa các mùa giải.

❖ **Materialized Leaderboard**

Bảng xếp hạng là thành phần có tần suất truy cập rất cao trong suốt thời gian diễn ra cuộc thi. Một phương án đơn giản là tính toán thứ hạng trực tiếp từ dữ liệu *submission* tại thời điểm truy vấn. Tuy nhiên, khi số lượng *submission* tăng lên, chi phí tổng hợp dữ liệu và tính toán thứ hạng cũng tăng theo, làm ảnh hưởng đến thời gian phản hồi của hệ thống.

Để giải quyết vấn đề này, BKDNAIOJ sử dụng cơ chế *materialized leaderboard*, trong đó dữ liệu xếp hạng được lưu trữ sẵn ở các bảng chuyên biệt và được cập nhật bất đồng bộ sau khi có kết quả mới.

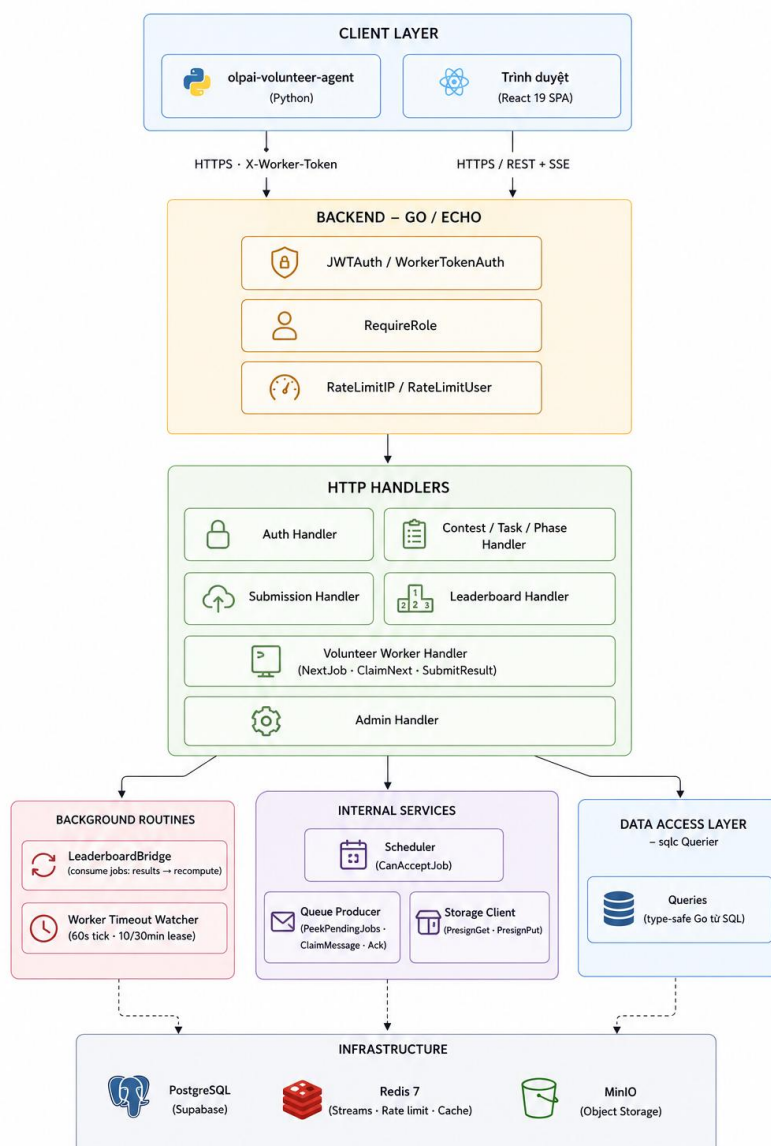
Cách tiếp cận này tách biệt hoàn toàn luồng cập nhật dữ liệu và luồng truy vấn hiển thị. Việc tính toán thứ hạng được thực hiện ở nền, trong khi các truy vấn từ người dùng chỉ cần đọc dữ liệu đã được tổng hợp trước. Nhờ đó, thời gian phản hồi của bảng xếp hạng gần như không phụ thuộc vào số lượng *submission* hoặc độ phức tạp của thuật toán xếp hạng.

Hai quyết định thiết kế trên phản ánh định hướng chung của BKDNAIOJ: ưu tiên tính nhất quán của dữ liệu, khả năng mở rộng trong dài hạn và hiệu năng vận hành khi số lượng người dùng và *submission* tăng lên.

3.4. Thiết kế kiến trúc Backend

3.4.1 Phân lớp ứng dụng

Backend của BKDNAIOJ được xây dựng bằng ngôn ngữ Go và tổ chức theo kiến trúc phân lớp như minh họa trong Hình 3.9.



Hình 3.9: Kiến trúc phân lớp của Backend Go

Lớp ngoài cùng là lớp tiếp nhận yêu cầu HTTP. Lớp này chịu trách nhiệm nhận request từ client, kiểm tra dữ liệu đầu vào, điều phối luồng xử lý và trả về phản hồi dưới dạng JSON. Các logic nghiệp vụ phức tạp được tách khỏi handler và tổ chức trong các module chuyên biệt nhằm tăng tính rõ ràng và khả năng bảo trì của mã nguồn.

Trước khi request được chuyển đến handler, hệ thống áp dụng một chuỗi middleware để xử lý các yêu cầu chung như xác thực người dùng, phân quyền truy cập và giới hạn tần suất request. Cách tổ chức này giúp tách các chức năng cắt ngang khỏi logic nghiệp vụ chính, đồng thời đảm bảo các API được bảo vệ nhất quán.

Lớp truy cập dữ liệu bao gồm các thành phần giao tiếp với PostgreSQL, MinIO và Redis. Đối với PostgreSQL, hệ thống sử dụng các truy vấn SQL được định nghĩa

trước và sinh mã truy cập dữ liệu tương ứng bằng sqlc. Cách tiếp cận này giúp duy trì tính rõ ràng của SQL, đồng thời đảm bảo an toàn kiểu dữ liệu ở mức mã nguồn Go. MinIO được sử dụng cho các thao tác lưu trữ tệp, trong khi Redis đảm nhiệm vai trò hàng đợi và truyền tải sự kiện giữa các thành phần.

Bên cạnh HTTP server, backend còn triển khai các tiến trình nền phục vụ các tác vụ bất đồng bộ. Thành phần cập nhật leaderboard xử lý kết quả chấm bài và cập nhật các bảng xếp hạng. Thành phần giám sát worker định kỳ kiểm tra các tác vụ quá hạn, thu hồi các claim không còn hợp lệ và phục hồi các submission bị gián đoạn.

Việc tách các tiến trình nền khỏi luồng xử lý request giúp các tác vụ có chi phí xử lý cao hoặc có tính chu kỳ không làm ảnh hưởng trực tiếp đến thời gian phản hồi của API. Đồng thời, cách tổ chức này giúp các cơ chế như cập nhật bảng xếp hạng và phục hồi lỗi có thể hoạt động độc lập với luồng tương tác của người dùng.

Các thành phần backend được thiết kế theo hướng không lưu trạng thái nghiệp vụ trong bộ nhớ tiến trình. Trạng thái chính của hệ thống được lưu trữ trong PostgreSQL và Redis. Thiết kế này tạo điều kiện cho việc mở rộng hệ thống theo chiều ngang trong trường hợp cần triển khai nhiều instance API, đồng thời giảm rủi ro mất trạng thái khi một tiến trình backend được khởi động lại.

3.4.2 Judging Contract

Judging Contract là giao thức chuẩn hóa quá trình giao tiếp giữa hệ thống chấm bài và chương trình đánh giá của từng bài toán. Mục tiêu của cơ chế này là tách biệt hoàn toàn logic đánh giá khỏi hệ thống lõi, cho phép mỗi bài toán định nghĩa tiêu chí chấm điểm riêng mà không cần thay đổi kiến trúc hoặc mã nguồn của nền tảng.

Trong BKDNAIOJ, Judging Contract được xây dựng dựa trên hai thành phần chính: cấu trúc dữ liệu đầu vào chuẩn hóa và giao thức trả kết quả thống nhất.

❖ Cấu trúc thư mục thực thi

Trong quá trình chấm bài, hệ thống chuẩn bị một môi trường làm việc với cấu trúc thư mục cố định để các chương trình đánh giá có thể truy cập dữ liệu một cách nhất quán:

- **submission/**: chứa bài nộp của thí sinh sau khi được tải xuống và giải nén.
- **assets/**: chứa các tài nguyên do ban tổ chức cung cấp, bao gồm dữ liệu đầu vào, dữ liệu tham chiếu (*ground truth*) và chương trình đánh giá.
- **generated/**: chứa kết quả dự đoán được sinh ra bởi chương trình suy luận trong các phase inference-based.
- **output/**: thư mục tạm phục vụ quá trình thực thi.

- **context.json**: chứa thông tin ngữ cảnh của cuộc thi, bài toán, phase và submission.

Đối với các phase output-only, chương trình đánh giá đọc trực tiếp dữ liệu từ thư mục **submission/**. Đối với các phase inference-based, hệ thống thực thi chương trình suy luận trước để tạo kết quả trong thư mục **generated/**, sau đó chuyển kết quả này cho chương trình đánh giá. Cách tiếp cận này giúp tách biệt rõ ràng giữa quá trình suy luận và quá trình tính điểm.

❖ *Giao thức trả kết quả*

Sau khi hoàn tất quá trình đánh giá, chương trình chấm điểm trả về kết quả dưới dạng một đối tượng JSON theo định dạng chuẩn:

```
{"status": "success", "raw_score": 0.847, "display_score": 84.7}
```

Trong đó:

- **status** biểu thị trạng thái thực thi của chương trình đánh giá.
- **raw_score** là điểm số gốc được sinh ra từ thước đo đánh giá của bài toán.
- **display_score** là điểm số được sử dụng cho việc xếp hạng và hiển thị trên giao diện người dùng.

Hệ thống đọc đối tượng JSON được xuất ra từ chương trình đánh giá để cập nhật kết quả của submission. Việc sử dụng một định dạng đầu ra thống nhất giúp các chương trình chấm điểm có thể được phát triển, kiểm thử và triển khai độc lập với hệ thống lõi, đồng thời nâng cao khả năng mở rộng của nền tảng khi bổ sung các loại bài toán và tiêu chí đánh giá mới.

Xét dưới góc độ kiến trúc phần mềm, Judging Contract cho phép mỗi bài toán được xem như một plugin đánh giá độc lập. Hệ thống lõi không cần biết chi tiết metric hay logic chấm điểm của từng bài toán mà chỉ cần đảm bảo tuân thủ giao thức đầu vào và đầu ra đã được định nghĩa. Thiết kế này giúp giảm sự phụ thuộc giữa các thành phần và tăng khả năng mở rộng của nền tảng.

3.4.3 *Redis Streams và cơ chế phân phối công việc*

Trong các hệ thống xử lý phân tán, hàng đợi thông điệp (*message queue*) đóng vai trò trung gian giữa thành phần sinh tác vụ và thành phần thực thi tác vụ. Việc sử dụng hàng đợi giúp tách biệt các thành phần của hệ thống, đồng thời hỗ trợ mở rộng quy mô và tăng khả năng chịu lỗi.

BKDNAIOJ sử dụng Redis Streams [10] làm cơ chế truyền tải tác vụ giữa Core API và các worker. Redis Streams cung cấp mô hình lưu trữ theo dòng sự kiện (*event*

stream), cho phép nhiều tiến trình cùng ghi nhận và xử lý dữ liệu theo cơ chế bất đồng bộ.

Trong hệ thống, hai luồng dữ liệu chính được sử dụng:

- **Job Stream:** lưu trữ các tác vụ chấm bài đang chờ được xử lý.
- **Result Stream:** lưu trữ kết quả được gửi về từ các worker sau khi hoàn thành tác vụ.

Thông qua cơ chế này, việc tiếp nhận bài nộp, thực thi đánh giá và cập nhật bảng xếp hạng được tách thành các quy trình độc lập nhưng vẫn duy trì khả năng phối hợp thông qua các sự kiện được ghi nhận trên Redis Streams.

3.4.4 *Leaderboard Materialized*

❖ *Bài toán cân bằng giữa tính cập nhật và hiệu năng truy vấn*

Bảng xếp hạng là một thành phần quan trọng của hệ thống thi đấu AI, có nhiệm vụ phản ánh kết quả mới nhất của các thí sinh trong quá trình diễn ra cuộc thi. Do đó, hệ thống cần đồng thời đáp ứng hai yêu cầu: cập nhật kết quả trong thời gian ngắn sau khi quá trình chấm bài hoàn tất và duy trì thời gian phản hồi thấp đối với các truy vấn hiển thị bảng xếp hạng.

Một cách tiếp cận trực tiếp là tính toán lại bảng xếp hạng từ toàn bộ lịch sử submission sau mỗi lần xuất hiện kết quả mới. Tuy nhiên, phương pháp này làm phát sinh chi phí xử lý tăng dần theo số lượng submission và trở nên kém hiệu quả khi cuộc thi có nhiều người tham gia hoặc tần suất nộp bài cao.

Ngược lại, nếu thực hiện tính toán thứ hạng tại thời điểm truy vấn (*on-the-fly ranking*), mỗi yêu cầu hiển thị bảng xếp hạng sẽ phải tổng hợp dữ liệu từ nhiều bảng và thực hiện các phép tính xếp hạng động. Cách tiếp cận này làm tăng độ phức tạp của truy vấn và ảnh hưởng đến thời gian phản hồi của hệ thống.

Để cân bằng giữa khả năng cập nhật và hiệu năng truy vấn, BKDNAIOJ sử dụng cơ chế **materialized leaderboard**, trong đó kết quả xếp hạng được lưu trữ sẵn trong các bảng chuyên biệt và được cập nhật bất đồng bộ sau khi quá trình chấm bài hoàn thành. Nhờ đó, việc hiển thị bảng xếp hạng chỉ yêu cầu truy xuất dữ liệu đã được tính toán trước thay vì thực hiện tính toán lại toàn bộ tại thời điểm truy vấn.

❖ *Chiến lược cập nhật: Incremental và Full Recompute*

BKDNAIOJ sử dụng hai chiến lược cập nhật bảng xếp hạng nhằm cân bằng giữa chi phí tính toán và tính nhất quán của dữ liệu.

Cập nhật gia tăng (Incremental Update) được sử dụng trong các trường hợp kết quả mới chỉ ảnh hưởng đến một số lượng nhỏ bản ghi. Khi một submission mới được

chấm xong, hệ thống so sánh kết quả vừa nhận với kết quả tốt nhất đang được lưu trữ của thí sinh trong cùng phase. Nếu thay đổi không ảnh hưởng đến các tham số toàn cục của bảng xếp hạng, hệ thống chỉ cập nhật các bản ghi liên quan thay vì tính toán lại toàn bộ. Cách tiếp cận này giúp giảm đáng kể chi phí xử lý trong điều kiện vận hành thông thường.

Tính toán lại toàn bộ (Full Recompute) được sử dụng khi xuất hiện các thay đổi có khả năng ảnh hưởng đến toàn bộ thứ hạng, chẳng hạn như việc xuất hiện một điểm số cao nhất mới hoặc khi cấu hình bảng xếp hạng được điều chỉnh bởi quản trị viên. Trong trường hợp này, hệ thống thực hiện tính toán lại toàn bộ bảng xếp hạng dựa trên dữ liệu hiện có nhằm đảm bảo tính nhất quán của kết quả.

Để tránh ảnh hưởng đến luồng xử lý chính của hệ thống, các tác vụ cập nhật bảng xếp hạng được thực hiện bất đồng bộ thông qua thành phần LeaderboardBridge. Nhờ đó, quá trình chấm bài và quá trình cập nhật bảng xếp hạng được tách biệt, giúp duy trì khả năng phản hồi của hệ thống ngay cả khi số lượng submission tăng cao.

Sau khi cập nhật bảng xếp hạng ở cấp độ bài toán (*task-phase leaderboard*), hệ thống tiếp tục tổng hợp kết quả lên các cấp độ cao hơn, bao gồm bảng xếp hạng theo phase và bảng xếp hạng tổng thể của cuộc thi. Quá trình cập nhật được thực hiện theo hướng từ dưới lên (*bottom-up aggregation*), đảm bảo tính nhất quán giữa các tầng dữ liệu xếp hạng trong toàn hệ thống.

❖ **ICPC-style Penalty**

Nhiều cuộc thi lập trình và thuật toán, điển hình là ICPC, sử dụng cơ chế **penalty time** như một tiêu chí phụ để phân biệt thứ hạng giữa các đội có cùng điểm số. Ý tưởng cơ bản của cơ chế này là khuyến khích thí sinh đạt kết quả tốt trong thời gian ngắn hơn, đồng thời cung cấp một quy tắc phá vỡ thế hòa khi các đội có cùng thành tích.

BKDNAIOJ áp dụng một cơ chế penalty time, trong đó thời gian phạt được tính dựa trên thời điểm nộp bài của thí sinh:

$$\text{penalty_minutes} = (\text{submitted_at} - \text{contest.start_time}) / 60 \text{ giây}$$

Trong đó:

- **submitted_at** là thời điểm bài nộp được hệ thống ghi nhận.
- **contest_start_time** là thời điểm bắt đầu cuộc thi.

Việc sử dụng thời điểm nộp bài thay vì thời điểm hoàn thành chấm bài giúp tách biệt kết quả xếp hạng khỏi hiệu năng của hạ tầng thực thi. Nhờ đó, thời gian xử lý của worker hoặc độ dài hàng đợi chấm bài không ảnh hưởng đến thứ hạng của thí sinh.

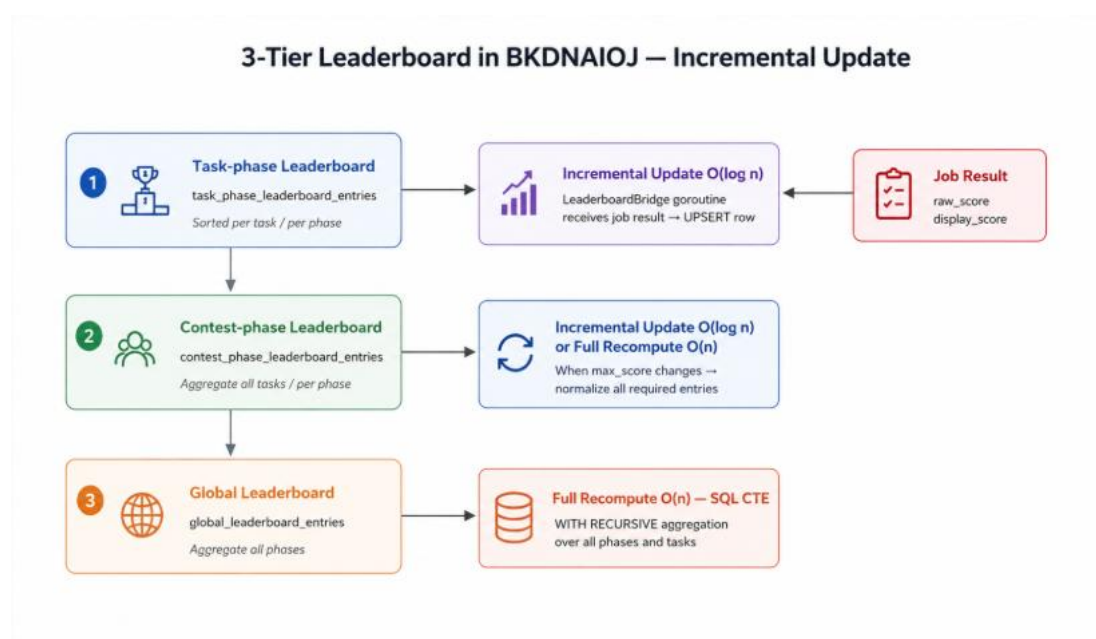
Thứ tự xếp hạng được xác định theo hai tiêu chí:

1. Điểm số cao hơn được xếp hạng cao hơn.
2. Nếu điểm số bằng nhau, penalty time nhỏ hơn được ưu tiên.

Cách tiếp cận này vừa duy trì tính công bằng của quá trình đánh giá, vừa đảm bảo khả năng phân biệt thứ hạng trong các trường hợp có cùng điểm số.

❖ Mô hình Leaderboard phân tầng

Để hỗ trợ nhiều phạm vi đánh giá khác nhau, BKDNAIOJ tổ chức dữ liệu xếp hạng theo mô hình phân tầng gồm ba cấp độ



Hình 3.10: Ba tầng leaderboard trong BKDNAIOJ

➤ Tầng 1 - Task-phase Leaderboard

Đây là tầng xếp hạng ở mức chi tiết nhất, phản ánh kết quả của một thí sinh hoặc đội thi đối với một bài toán cụ thể trong một giai đoạn thi cụ thể. Mỗi bản ghi tại tầng này lưu trữ kết quả tốt nhất của người tham gia trên bài toán tương ứng và là nguồn dữ liệu đầu vào cho các tầng tổng hợp phía trên.

➤ Tầng 2 - Contest-phase Leaderboard

Tầng thứ hai tổng hợp kết quả từ nhiều bài toán trong cùng một giai đoạn của cuộc thi. Điểm số tại tầng này được tính từ các kết quả ở tầng Task-phase theo quy tắc được cấu hình bởi ban tổ chức, chẳng hạn như tổng điểm hoặc điểm trung bình. Đây là bảng xếp hạng chính mà thí sinh thường sử dụng để theo dõi vị trí của mình trong cuộc thi.

➤ *Tầng 3 - Global Ranking*

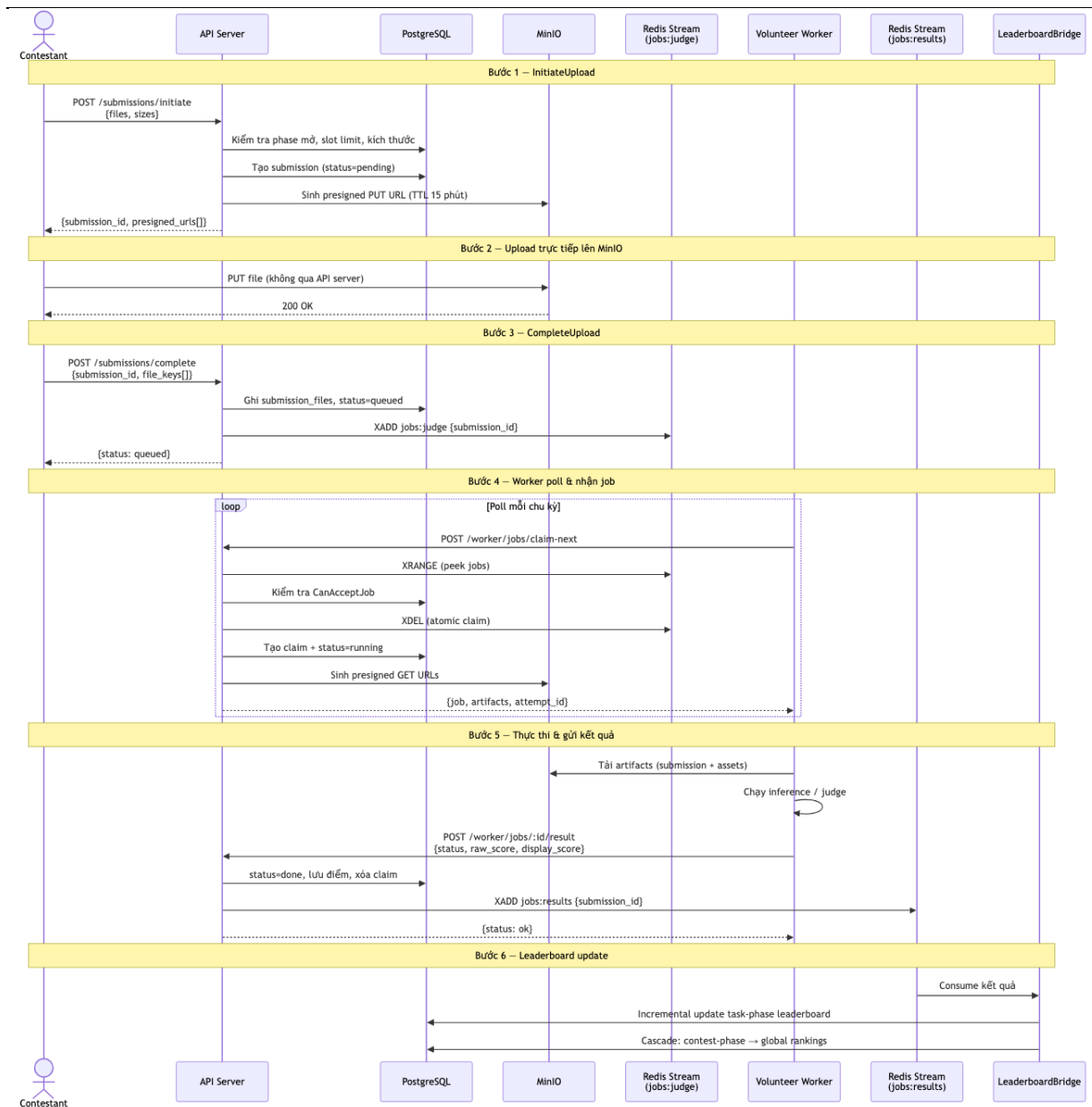
Tầng cao nhất tổng hợp kết quả của người dùng trên phạm vi toàn hệ thống. Thay vì chỉ giới hạn trong một cuộc thi cụ thể, bảng xếp hạng này cho phép so sánh thành tích của người tham gia giữa nhiều cuộc thi khác nhau, từ đó cung cấp một góc nhìn tổng thể về năng lực và mức độ hoạt động của người dùng trên nền tảng.

Ba tầng được cập nhật tuần tự theo thứ tự task-phase → contest-phase → global ranking. Khi một submission mới được chấm xong, hệ thống trước tiên cập nhật kết quả ở tầng Task-phase. Những thay đổi phát sinh sau đó được lan truyền lên tầng Contest-phase và cuối cùng là Global Ranking. Cách tiếp cận này đảm bảo rằng dữ liệu tổng hợp luôn được xây dựng từ các kết quả chi tiết nhất, đồng thời giúp duy trì tính nhất quán giữa các tầng xếp hạng.

Mô hình phân tầng còn cho phép mỗi cấp độ leaderboard được tối ưu hóa độc lập theo đặc điểm truy cập của mình. Các bảng xếp hạng có tần suất cập nhật cao được tối ưu cho thao tác ghi, trong khi các bảng xếp hạng tổng hợp được tối ưu cho truy vấn đọc và hiển thị. Nhờ đó, hệ thống có thể đáp ứng đồng thời yêu cầu cập nhật liên tục và khả năng truy xuất dữ liệu với độ trễ thấp.

3.4.5 *Luồng nộp bài và chấm điểm*

Toàn bộ quy trình từ lúc thí sinh bấm “Nộp bài” đến khi bảng xếp hạng cập nhật trải qua sáu bước, được mô tả chi tiết trong Hình 3.4.



Hình 3.11: Sequence diagram đầy đủ từ nộp bài đến leaderboard update

Bước 1 - Khởi tạo quá trình nộp bài. Thí sinh gửi yêu cầu nộp bài đến hệ thống, kèm theo các thông tin mô tả tệp như tên tệp và kích thước dữ liệu. API thực hiện kiểm tra các điều kiện hợp lệ, bao gồm trạng thái của giai đoạn thi, số lượt nộp còn lại của thí sinh và giới hạn kích thước tệp. Nếu yêu cầu hợp lệ, hệ thống tạo một bản ghi submission với trạng thái ban đầu là *pending* và sinh ra một presigned URL phục vụ quá trình tải tệp lên hệ thống lưu trữ.

Bước 2 - Tải tệp lên hệ thống lưu trữ. Thí sinh sử dụng presigned URL để tải trực tiếp tệp bài nộp lên MiniIO. Cách tiếp cận này giúp tách biệt luồng truyền dữ liệu dung lượng lớn khỏi máy chủ API, qua đó giảm tải cho lớp xử lý nghiệp vụ và nâng cao khả năng mở rộng của hệ thống.

Bước 3 - Xác nhận hoàn tất tải lên. Sau khi quá trình tải tệp hoàn tất, ứng dụng khách gửi yêu cầu xác nhận đến API. Hệ thống kiểm tra lại thông tin của tệp đã được lưu trữ, cập nhật cơ sở dữ liệu và chuyển trạng thái submission sang *queued*. Đồng thời, một tác vụ chấm bài tương ứng được đưa vào hàng đợi Redis Streams để chờ xử lý.

Bước 4 - Phân phối tác vụ cho worker. Các máy chấm tình nguyện định kỳ gửi yêu cầu nhận tác vụ đến hệ thống. Khi phát hiện worker có tài nguyên khả dụng và đáp ứng các yêu cầu thực thi của tác vụ, hệ thống sẽ gán bài chấm tương ứng cho worker. Sau khi tác vụ được xác nhận, worker nhận các thông tin cần thiết để truy cập dữ liệu và bắt đầu quá trình thực thi.

Bước 5 - Thực thi và ghi nhận kết quả. Worker tải các tài nguyên cần thiết từ hệ thống lưu trữ, bao gồm bài nộp của thí sinh, bộ dữ liệu đánh giá và chương trình chấm điểm. Tùy thuộc vào loại phase, worker thực hiện suy luận mô hình hoặc đánh giá trực tiếp trên tệp kết quả. Sau khi hoàn tất, kết quả chấm được gửi về hệ thống, lưu vào cơ sở dữ liệu và phát sinh sự kiện cập nhật kết quả.

Bước 6 - Cập nhật bảng xếp hạng. Thành phần LeaderboardBridge tiếp nhận các sự kiện kết quả và thực hiện cập nhật các bảng xếp hạng liên quan. Quá trình này bao gồm việc cập nhật điểm số của từng bài toán trong từng giai đoạn, tổng hợp kết quả theo giai đoạn thi và tính toán thứ hạng chung của cuộc thi. Các phép cập nhật được thực hiện theo hướng gia tăng nhằm giảm chi phí xử lý và đảm bảo khả năng phản hồi theo thời gian thực.

Quy trình trên được thiết kế nhằm đảm bảo ba yêu cầu quan trọng của hệ thống. Thứ nhất, tính nhất quán của dữ liệu được duy trì thông qua việc chỉ khởi tạo tác vụ chấm bài sau khi quá trình tải tệp được xác nhận thành công. Thứ hai, khả năng chịu lỗi được hỗ trợ bởi cơ chế giám sát worker, cho phép thu hồi và phân phối lại tác vụ khi xảy ra sự cố trong quá trình thực thi. Thứ ba, các thành phần API, hệ thống lưu trữ và cụm worker được tách biệt rõ ràng về trách nhiệm, góp phần nâng cao khả năng mở rộng và bảo trì của toàn bộ hệ thống.

3.4.6 Sơ đồ hoạt động

Sơ đồ hoạt động luồng nộp bài (Hình 3.12) mô tả quy trình xử lý một submission từ thời điểm thí sinh khởi tạo yêu cầu nộp bài cho đến khi bài nộp được đưa vào hàng đợi chấm. Sơ đồ được tổ chức theo ba swim lane tương ứng với Contestant, API Server và hệ thống lưu trữ (MinIO/Redis).

Quy trình bắt đầu khi thí sinh lựa chọn tệp và gửi yêu cầu nộp bài. API Server thực hiện các kiểm tra nghiệp vụ như thời gian mở của phase, quyền tham gia cuộc thi, giới hạn số lần nộp và tính hợp lệ của tệp. Nếu các điều kiện được thỏa mãn, hệ thống cấp quyền upload và cho phép client tải trực tiếp dữ liệu lên object storage. Sau khi quá trình upload hoàn tất, API xác nhận thông tin tệp, tạo submission và đưa tác vụ vào hàng đợi chấm bài.

Hai điểm quyết định quan trọng trong luồng này là bước kiểm tra điều kiện trước khi cấp quyền upload và bước xác nhận sau khi upload hoàn tất. Thiết kế hai giai đoạn giúp tránh việc tạo submission không hợp lệ, đồng thời đảm bảo chỉ những tệp đã được tải lên thành công mới được đưa vào quy trình chấm bài.



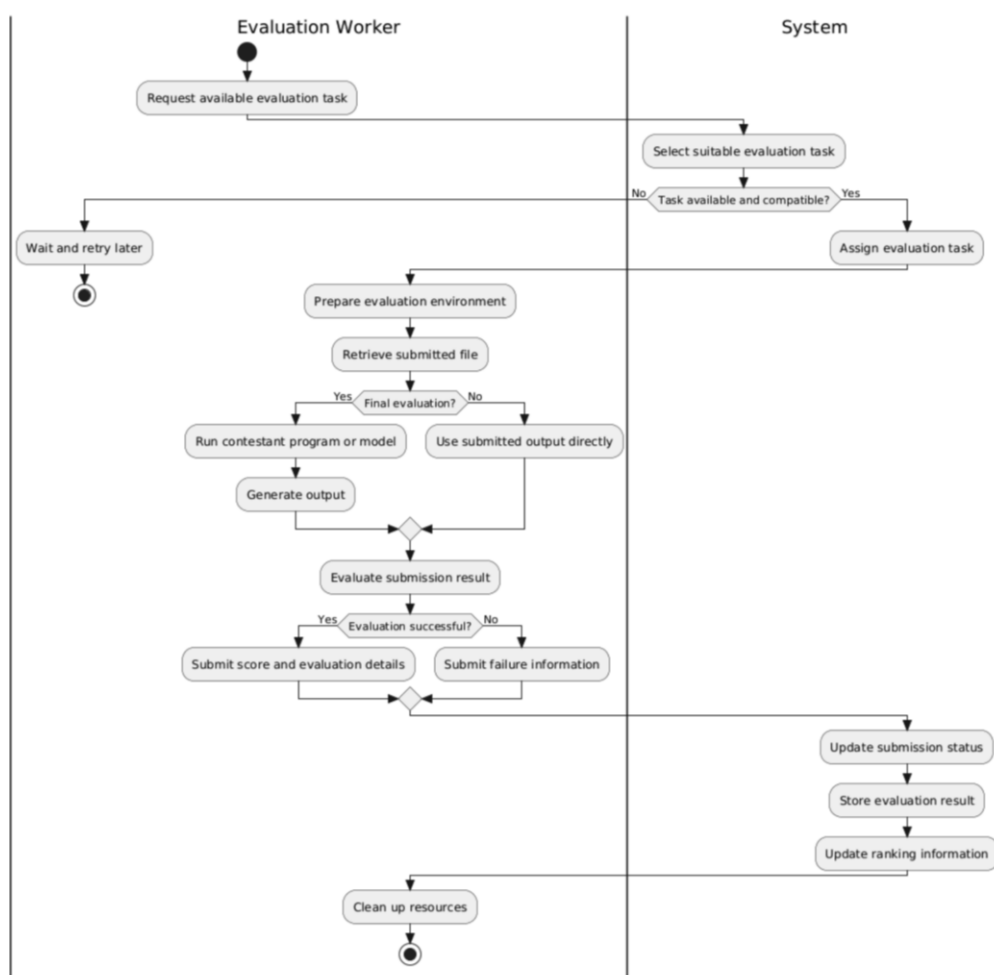
Hình 3.12: Sơ đồ hoạt động - Luồng nộp bài của thí sinh

Sơ đồ hoạt động luồng chấm bài (Hình 3.13) mô tả vòng đời của một job kể từ khi worker gửi yêu cầu nhận việc đến khi kết quả được ghi nhận vào hệ thống. Sơ đồ được tổ chức theo các swim lane tương ứng với Worker Agent, API Server và hệ thống lưu trữ.

Quy trình bắt đầu khi worker chủ động gửi yêu cầu nhận job. Hệ thống thực hiện kiểm tra tính tương thích giữa worker và các job đang chờ, sau đó phân công một tác vụ phù hợp. Worker tải các artifact cần thiết, chuẩn bị môi trường thực thi và tiến hành chấm bài.

Trong quá trình xử lý xuất hiện ba nhánh quyết định chính. Nhánh thứ nhất xác định worker có đủ điều kiện nhận job hay không. Nhánh thứ hai phân biệt giữa phase output-only và phase final inference, từ đó quyết định có cần thực thi mô hình của thí sinh hay không. Nhánh thứ ba xử lý kết quả thực thi, bao gồm hai trường hợp thành công hoặc thất bại do lỗi runtime, timeout hoặc lỗi đánh giá.

Sau khi quá trình chấm hoàn tất, worker gửi kết quả về API Server. Kết quả được lưu vào cơ sở dữ liệu và chuyển tiếp tới cơ chế cập nhật leaderboard bất đồng bộ.

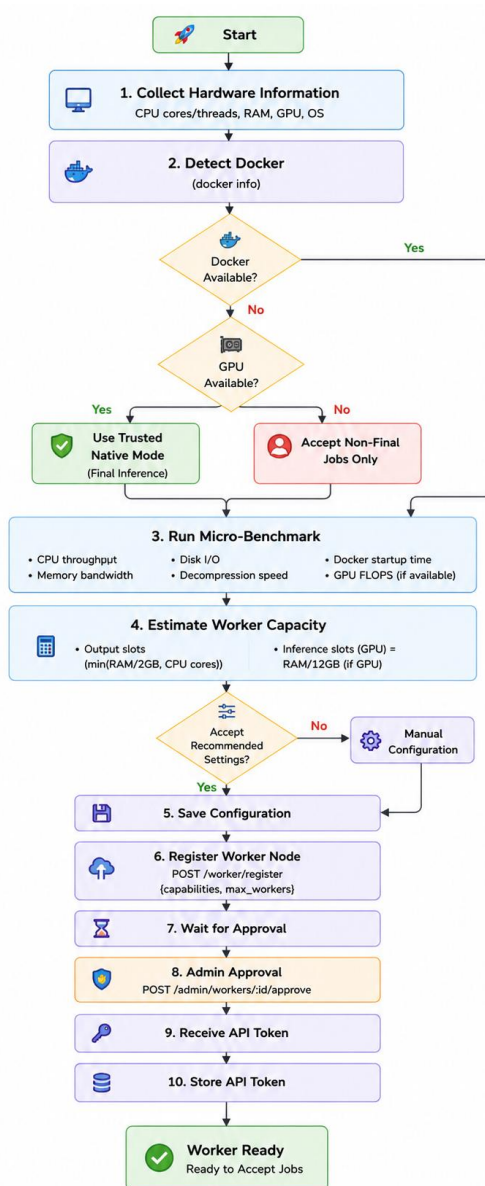


Hình 3.13: Sơ đồ hoạt động - Luồng thực thi chấm bài của Worker Agent

3.5. Thiết kế Hệ thống Máy chấm tình nguyện

3.5.1 WorkerProfile và benchmark thực đo

BKDNAIOJ sử dụng cơ chế đánh giá năng lực phần cứng tự động khi một máy chấm tình nguyện tham gia hệ thống lần đầu. Trong quá trình cài đặt, công cụ setup của olpai-volunteer-agent thực hiện một bộ micro-benchmark nhằm thu thập các thông số phản ánh năng lực thực tế của máy thay vì dựa trên khai báo thủ công của người dùng.



Hình 3.14: Quy trình thiết lập và đăng ký máy chấm tình nguyện

Quá trình benchmark đo lường một số nhóm tài nguyên chính bao gồm năng lực xử lý CPU, băng thông bộ nhớ, hiệu năng lưu trữ, khả năng khởi động môi trường thực

thi và năng lực GPU (nếu có). Kết quả được tổng hợp thành một hồ sơ năng lực (*WorkerProfile*) và gửi về hệ thống trong quá trình đăng ký worker.

Thông tin này được lưu trong trường capabilities của bảng `volunteer_workers` và được sử dụng trong quá trình phân phối tác vụ. Một *WorkerProfile* điển hình bao gồm:

- Thông tin CPU và bộ nhớ.
- Hiệu năng đọc/ghi lưu trữ.
- Khả năng hỗ trợ Docker.
- Tổng dung lượng RAM và dung lượng lưu trữ khả dụng.
- Số lượng slot xử lý đồng thời.
- Thông tin GPU và VRAM (nếu có).
- Các quyền vận hành được cấp bởi quản trị viên.

Việc sử dụng dữ liệu benchmark thực đo giúp hệ thống phản ánh chính xác khả năng của từng worker, đồng thời giảm phụ thuộc vào các thông tin cấu hình do người dùng tự khai báo.

3.5.2 Mô hình năng lực Worker

Mỗi worker được mô hình hóa thông qua một tập thuộc tính mô tả tài nguyên và khả năng thực thi. Trên cơ sở đó, hệ thống có thể phân biệt các worker CPU thông thường với các worker có GPU hoặc các worker được phép thực thi ở chế độ native.

Bên cạnh các thông số phần cứng, mỗi worker còn được cấu hình số lượng slot xử lý đồng thời. Hệ thống phân tách slot dành cho các tác vụ output-only và slot dành cho các tác vụ inference nhằm hạn chế tình trạng các tác vụ nhẹ chiếm toàn bộ tài nguyên và ảnh hưởng đến các tác vụ suy luận có yêu cầu cao hơn.

Thiết kế này cho phép hệ thống duy trì một mô hình tài nguyên thống nhất cho toàn bộ worker pool, đồng thời tạo nền tảng cho cơ chế phân phối tác vụ được trình bày ở phần tiếp theo.

3.5.3 Quy trình đăng ký và tham gia Worker Pool

Sau khi hoàn thành benchmark, worker gửi yêu cầu đăng ký đến hệ thống kèm theo *WorkerProfile* và các thông tin định danh cần thiết. Worker chỉ được phép tham gia xử lý tác vụ sau khi được quản trị viên phê duyệt.

Khi đã được kích hoạt, worker trở thành một thành viên của worker pool và có thể nhận các tác vụ chấm bài từ hệ thống. Mọi trạng thái hoạt động, lịch sử thực thi và thông tin cấu hình đều được lưu trữ tập trung để phục vụ giám sát và điều phối.

3.5.4 Thuật toán ClaimNext - Chi tiết thiết kế

Để hiện thực cơ chế lập lịch Capability-aware FIFO, BKDNAIOJ sử dụng thuật toán **ClaimNext** nhằm lựa chọn tác vụ phù hợp cho mỗi worker tại thời điểm gửi yêu cầu nhận việc. Khác với cơ chế FIFO thuần túy, thuật toán không mặc định gán tác vụ đầu tiên trong hàng đợi cho worker, mà thực hiện kiểm tra mức độ tương thích giữa yêu cầu của tác vụ và năng lực của worker trước khi đưa ra quyết định phân phối.

Thuật toán ClaimNext gồm bốn giai đoạn chính.

Bước 1 - Kiểm tra khả năng tiếp nhận tác vụ

Trước khi thực hiện tìm kiếm trong hàng đợi, hệ thống kiểm tra số lượng tác vụ đang được xử lý bởi worker. Nếu số lượng tác vụ đang hoạt động đã đạt tới giới hạn đồng thời được cấu hình cho worker, worker sẽ không yêu cầu nhận việc. Bước này giúp đảm bảo rằng worker không nhận vượt quá khả năng xử lý của mình.

Bước 2 - Đọc cửa sổ tác vụ từ hàng đợi

Hệ thống đọc một tập con các tác vụ đang chờ xử lý từ hàng đợi theo thứ tự FIFO. Quá trình này chỉ tạo một ảnh chụp (*snapshot*) của hàng đợi tại thời điểm hiện tại và không làm thay đổi trạng thái của các tác vụ.

Việc sử dụng cửa sổ giới hạn thay vì quét toàn bộ hàng đợi giúp kiểm soát chi phí tìm kiếm trong trường hợp số lượng submission lớn, đồng thời vẫn duy trì được tính ưu tiên theo thứ tự thời gian.

Bước 3 - Kiểm tra tính tương thích

Các tác vụ trong cửa sổ được duyệt tuần tự theo thứ tự FIFO. Đối với mỗi tác vụ, hệ thống đánh giá mức độ tương thích giữa yêu cầu của tác vụ và hồ sơ năng lực của worker.

Một tác vụ được xem là tương thích nếu các điều kiện cần thiết đều được thỏa mãn, bao gồm:

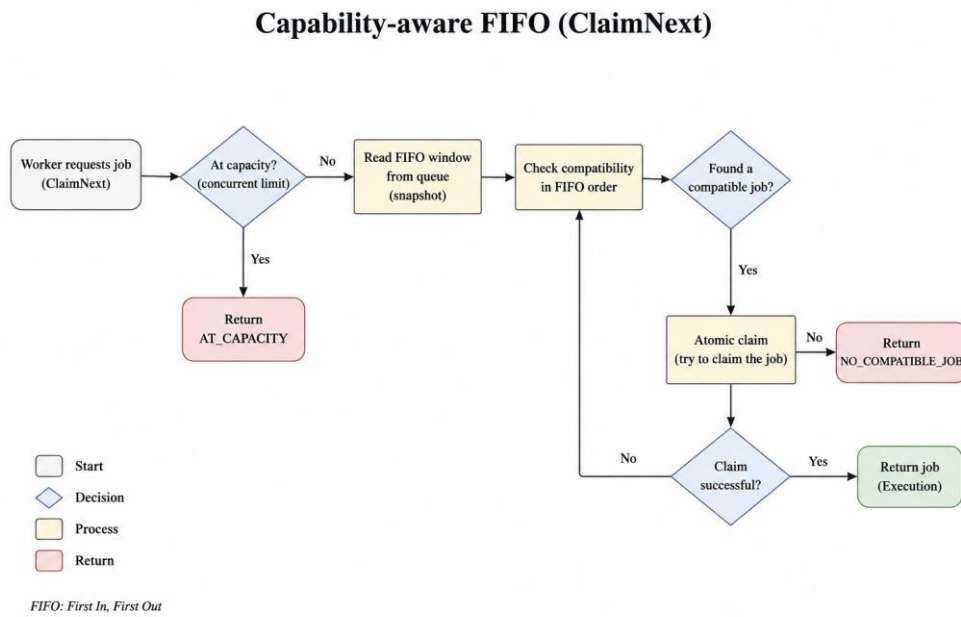
- Worker hỗ trợ môi trường thực thi yêu cầu của tác vụ.
- Số lượng slot còn trống của worker đáp ứng loại tác vụ tương ứng.
- Tài nguyên phần cứng khả dụng đáp ứng yêu cầu tối thiểu của tác vụ.
- Các ràng buộc đặc thù khác (nếu có) được thỏa mãn.

Tác vụ đầu tiên thỏa mãn toàn bộ các điều kiện trên sẽ được lựa chọn để cấp phát.

Bước 4 - Xác nhận quyền sở hữu tác vụ

Sau khi lựa chọn được tác vụ phù hợp, hệ thống thực hiện thao tác cấp phát theo cơ chế nguyên tử (*atomic claim*) nhằm đảm bảo rằng chỉ một worker duy nhất có thể nhận tác vụ đó. Nếu việc cấp phát thành công, hệ thống cập nhật trạng thái submission sang trạng thái thực thi và trả về các thông tin cần thiết để worker bắt đầu xử lý.

Trong trường hợp tác vụ đã được một worker khác nhận trước đó, hệ thống tiếp tục tìm kiếm tác vụ tương thích tiếp theo trong cửa sổ hiện tại.



Hình 3.15: Minh hoạ thuật toán ClaimNext

Thuật toán trên kết hợp hai nguyên tắc quan trọng. Thứ nhất, thứ tự FIFO được duy trì nhằm đảm bảo các tác vụ đến trước có cơ hội được xử lý trước. Thứ hai, việc kiểm tra khả năng tương thích giúp tránh tình trạng phân phối tác vụ cho các worker không đủ năng lực thực thi. Nhờ đó, hệ thống đạt được sự cân bằng giữa tính công bằng của hàng đợi và hiệu quả sử dụng tài nguyên trong môi trường volunteer computing không đồng nhất.

Việc giới hạn phạm vi tìm kiếm trong một cửa sổ tác vụ cố định tạo ra một sự đánh đổi giữa tính công bằng tuyệt đối và hiệu quả vận hành. Mặc dù một số tác vụ ở đầu hàng đợi có thể tạm thời bị bỏ qua nếu không tương thích với worker hiện tại, cơ chế này giúp giảm hiện tượng nhàn rỗi tài nguyên và tăng khả năng khai thác các worker đang sẵn sàng xử lý.

3.5.5 Cơ chế cấp phát tác vụ nguyên tử

Một thách thức phổ biến trong các hệ thống worker phân tán là hiện tượng tranh chấp tài nguyên (*race condition*) khi nhiều worker đồng thời cố gắng nhận cùng một tác vụ. Nếu không có cơ chế đồng bộ phù hợp, một tác vụ có thể bị cấp phát cho nhiều worker hoặc bị mất khỏi hàng đợi.

Để giải quyết vấn đề này, BKDNAIOJ sử dụng cơ chế thực thi nguyên tử (*atomic execution*) của Lua Script trên Redis. Toàn bộ quá trình lựa chọn và xác nhận quyền sở hữu một tác vụ được thực hiện trong một phiên thực thi duy nhất trên máy chủ Redis. Nhờ đó, việc kiểm tra trạng thái tác vụ và cập nhật hàng đợi được đảm bảo nhất quán, ngay cả khi nhiều worker gửi yêu cầu đồng thời.

Bên cạnh đó, hệ thống không áp dụng cơ chế phân phối tuần tự đơn thuần mà cho phép lựa chọn tác vụ dựa trên mức độ tương thích giữa yêu cầu của tác vụ và năng lực của worker. Cách tiếp cận này đặc biệt phù hợp với môi trường có tài nguyên không đồng nhất, nơi các worker sở hữu năng lực xử lý khác nhau.

Sự kết hợp giữa Redis Streams, cơ chế cấp phát nguyên tử và mô hình worker tự nguyện tạo thành nền tảng cho hệ thống chấm bài phân tán của BKDNAIOJ. Thiết kế này cho phép tận dụng hiệu quả các nguồn tài nguyên sẵn có, đồng thời duy trì tính nhất quán và khả năng mở rộng của hệ thống khi số lượng bài nộp hoặc số lượng worker tăng lên.

3.5.6 Cơ chế Lease, Heartbeat và Fault Recovery

Trong các hệ thống xử lý phân tán, một thách thức phổ biến là hiện tượng worker ngừng hoạt động trong khi đang thực hiện tác vụ. Nguyên nhân có thể xuất phát từ lỗi phần cứng, mất kết nối mạng, lỗi phần mềm hoặc quá trình thực thi bị treo. Nếu không có cơ chế giám sát và phục hồi phù hợp, các tác vụ đang được xử lý có thể không hoàn thành và không được phân phối lại cho các worker khác.

Để giải quyết vấn đề này, BKDNAIOJ áp dụng cơ chế **lease-based ownership**, trong đó quyền xử lý một tác vụ chỉ có hiệu lực trong một khoảng thời gian xác định. Khi một worker nhận được tác vụ, hệ thống tạo một bản ghi claim tương ứng với thời hạn lease ban đầu. Trong suốt quá trình thực thi, worker phải gửi tín hiệu **heartbeat** định kỳ để gia hạn lease và xác nhận rằng tác vụ vẫn đang được xử lý.

Nếu heartbeat không được nhận trong khoảng thời gian quy định, lease sẽ hết hiệu lực và tác vụ được xem là không còn thuộc quyền xử lý của worker đó. Khi đó, hệ

thống có thể tiến hành các bước phục hồi nhằm đưa tác vụ trở lại hàng đợi để được xử lý bởi một worker khác.

Để thực hiện cơ chế này, BKDNAIOJ triển khai một tiến trình giám sát chạy nền (*worker timeout watcher*) với hai nhiệm vụ chính.

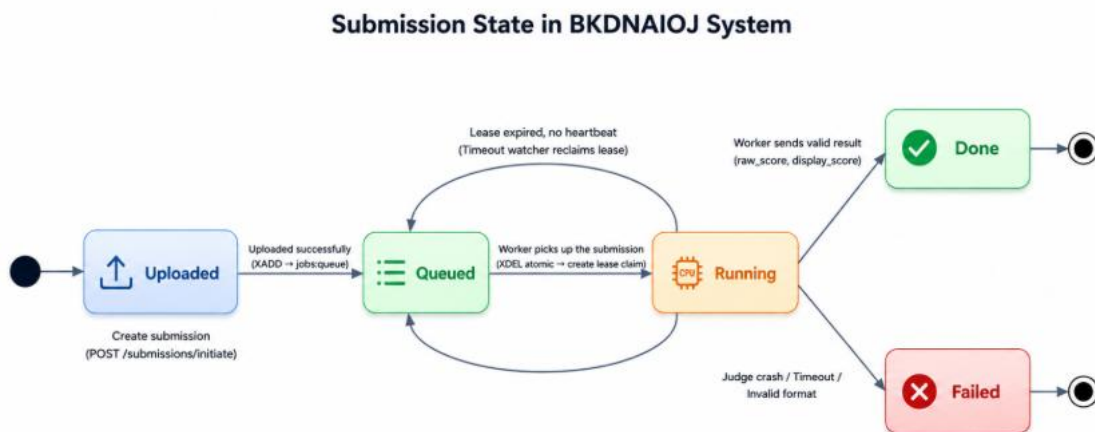
Thứ nhất, phát hiện và thu hồi các claim hết hạn (*stale claim cleanup*). Tiến trình giám sát định kỳ kiểm tra các claim đã vượt quá thời gian lease cho phép. Khi phát hiện một claim không còn hợp lệ, hệ thống thu hồi quyền xử lý của worker tương ứng, chuyển trạng thái submission về hàng đợi chờ xử lý và đưa tác vụ trở lại cơ chế phân phối công việc.

Thứ hai, phát hiện các submission không nhất quán (*orphan recovery*). Đây là trường hợp submission đang ở trạng thái thực thi nhưng không còn claim hợp lệ tương ứng, có thể xuất hiện do lỗi đồng bộ giữa các thành phần của hệ thống. Khi phát hiện tình huống này, hệ thống thực hiện phục hồi bằng cách đưa submission trở lại trạng thái chờ xử lý để tiếp tục vòng đời bình thường.

Nhờ sự kết hợp giữa lease, heartbeat và cơ chế phục hồi tự động, hệ thống có thể hạn chế ảnh hưởng của các sự cố xảy ra trên worker và nâng cao độ tin cậy của quá trình chấm bài trong môi trường phân tán.

3.5.7 Vòng đời của Submission

Hình 2.1 mô tả vòng đời của một submission trong hệ thống BKDNAIOJ.



Hình 3.16: State machine của submission trong BKDNAIOJ

Các trạng thái chính của submission được trình bày trong Bảng 3.7.

Bảng 3.7: Trạng thái submission

Trạng thái	Ý nghĩa
uploaded	Tệp bài nộp đã được tải lên hệ thống lưu trữ và đang chờ xác nhận
queued	Bài nộp đã được xác nhận và đang chờ worker tiếp nhận
running	Worker đang thực hiện tác vụ chấm bài
done	Quá trình chấm bài hoàn tất thành công và đã có kết quả
failed	Quá trình xử lý kết thúc với lỗi xác định

Một submission thường trải qua chuỗi trạng thái:

uploaded → queued → running → done

Trong trường hợp xảy ra lỗi hoặc worker mất khả năng xử lý, submission có thể quay trở lại trạng thái **queued** để được phân phối lại cho worker khác:

running → queued

Ngoài ra, nếu quá trình thực thi kết thúc với lỗi xác định như vượt quá thời gian cho phép, lỗi định dạng dữ liệu hoặc lỗi trong chương trình đánh giá, submission sẽ chuyển sang trạng thái:

running → failed

Việc mô hình hóa submission dưới dạng state machine giúp xác định rõ các trạng thái hợp lệ và các điều kiện chuyển trạng thái tương ứng. Cách tiếp cận này hỗ trợ quá trình thiết kế, kiểm thử và vận hành hệ thống, đồng thời tạo cơ sở cho việc xây dựng các cơ chế giám sát và phục hồi lỗi trong môi trường xử lý phân tán.

3.5.8 *Artifact Cache*

Để giảm băng thông mạng và thời gian chuẩn bị môi trường thực thi, worker duy trì một cơ chế cache artifact trên bộ nhớ lưu trữ cục bộ. Ý tưởng chính là tránh tải lại các tài nguyên dùng chung nhiều lần giữa các submission khác nhau.

Trong thực tế, các bộ dữ liệu đánh giá và judge script thường được sử dụng lặp lại cho toàn bộ submissions của cùng một phase. Một phase có thể nhận hàng trăm submission nhưng chỉ tham chiếu đến cùng một tập dataset và cùng một chương trình

đánh giá. Ngược lại, file submission của thí sinh luôn thay đổi theo từng lần nộp và do đó không được lưu cache.

Mỗi artifact được định danh bởi bộ khóa gồm loại tài nguyên, mã định danh của tài nguyên và giá trị băm SHA256 tương ứng. Khi cần sử dụng một artifact, worker kiểm tra cache cục bộ trước khi thực hiện tải về từ object storage. Nếu tệp đã tồn tại và giá trị băm khớp với thông tin do máy chủ cung cấp, worker sử dụng trực tiếp bản sao trong cache.

Trong trường hợp nhiều luồng thực thi đồng thời yêu cầu cùng một artifact, hệ thống áp dụng cơ chế khóa theo từng artifact. Chỉ một luồng thực hiện việc tải xuống, trong khi các luồng còn lại chờ quá trình này hoàn tất rồi sử dụng chung kết quả. Cách tiếp cận này giúp tránh các lần tải lặp không cần thiết và giảm tải cho hệ thống lưu trữ.

Để đảm bảo tính nhất quán, artifact được tải xuống một vị trí tạm thời, sau đó kiểm tra giá trị băm SHA256. Chỉ khi quá trình xác minh thành công, tệp mới được chuyển sang vị trí cache chính thức bằng thao tác thay thế nguyên tử (*atomic rename*). Nhờ đó, các luồng khác chỉ có thể quan sát và sử dụng artifact sau khi tệp đã được tải xuống đầy đủ và xác minh thành công, tránh tình trạng đọc phải dữ liệu chưa hoàn chỉnh.

3.5.9 Sandbox và Cô lập Mã Không Tin Cậy

❖ Yêu cầu cô lập môi trường thực thi

Trong các hệ thống đánh giá dựa trên thực thi mô hình (*inference-based judging*), mã nguồn do thí sinh cung cấp cần được xem là mã không tin cậy (*untrusted code*). Việc thực thi trực tiếp các chương trình này trên hạ tầng của hệ thống có thể làm phát sinh nhiều rủi ro về bảo mật, tính công bằng và độ ổn định của môi trường đánh giá.

Các rủi ro có thể xuất hiện bao gồm truy cập tài nguyên bên ngoài phạm vi cho phép, sử dụng dữ liệu không được cung cấp trong cuộc thi, tiêu thụ quá mức tài nguyên tính toán hoặc ảnh hưởng đến các tác vụ đang chạy đồng thời trên cùng một worker. Do đó, hệ thống cần đảm bảo cơ chế cô lập môi trường thực thi trên nhiều khía cạnh khác nhau.

Trong BKDNAIOJ, yêu cầu cô lập được xem xét trên bốn chiều chính:

- **Cô lập mạng (Network Isolation):** hạn chế hoặc vô hiệu hóa khả năng giao tiếp với các hệ thống bên ngoài.
- **Cô lập hệ thống tệp (Filesystem Isolation):** giới hạn phạm vi truy cập dữ liệu trong thư mục làm việc được cấp phát.
- **Cô lập tiến trình (Process Isolation):** kiểm soát số lượng tiến trình hoặc luồng có thể được tạo ra trong quá trình thực thi.

- **Cô lập tài nguyên (Resource Isolation):** giới hạn mức sử dụng CPU, bộ nhớ và thời gian thực thi của chương trình.

Các cơ chế trên giúp đảm bảo rằng việc thực thi mô hình của một thí sinh không ảnh hưởng đến tính công bằng của cuộc thi cũng như sự ổn định của hệ thống.

❖ **Cô lập dựa trên Container**

BKDNAIOJ sử dụng Docker container như cơ chế cô lập chính đối với các phase đánh giá dựa trên suy luận mô hình. Mỗi submission được thực thi trong một container độc lập được khởi tạo riêng cho tác vụ tương ứng. Sau khi quá trình thực thi hoàn tất hoặc vượt quá thời gian cho phép, container sẽ được hủy bỏ và toàn bộ dữ liệu tạm thời được giải phóng.

Các ràng buộc được áp dụng trên container nhằm giới hạn phạm vi hoạt động của chương trình và kiểm soát việc sử dụng tài nguyên hệ thống.

Bảng 3.8: Các cơ chế giới hạn trong Docker Sandbox

Cơ chế	Mục đích
Vô hiệu hóa kết nối mạng	Ngăn truy cập các dịch vụ và tài nguyên bên ngoài
Giới hạn số lượng tiến trình	Hạn chế việc tạo quá nhiều tiến trình hoặc luồng
Giới hạn bộ nhớ	Kiểm soát mức sử dụng RAM của chương trình
Giới hạn CPU	Đảm bảo khả năng chia sẻ tài nguyên giữa các tác vụ
Không gian làm việc riêng	Tách biệt dữ liệu giữa các submission
Tự động hủy container sau khi kết thúc	Ngăn việc duy trì trạng thái giữa các lần thực thi
Giới hạn thời gian thực thi	Phát hiện và dừng các tác vụ kéo dài bất thường

Môi trường thực thi được xây dựng trên image Docker chuẩn của hệ thống, trong đó đã tích hợp các thư viện Machine Learning phổ biến như PyTorch, TensorFlow và scikit-learn. Điều này giúp đảm bảo tính nhất quán của môi trường đánh giá giữa các worker và giảm sự phụ thuộc vào cấu hình cục bộ của từng máy.

❖ **Chế độ Trusted Native Execution**

Bên cạnh cơ chế Docker Sandbox, BKDNAIOJ còn hỗ trợ chế độ thực thi trực tiếp trên hệ điều hành (*Trusted Native Execution*). Chế độ này được thiết kế cho các môi trường mà việc sử dụng container có thể làm giảm khả năng khai thác phần cứng chuyên dụng, đặc biệt là các hệ thống GPU đã được cấu hình sẵn cho các thư viện tính toán tăng tốc.

Trong chế độ này, chương trình suy luận được thực thi trực tiếp trên môi trường Python của worker thay vì trong container. Cách tiếp cận này giúp tận dụng tối đa tài nguyên phần cứng và giảm chi phí khởi tạo môi trường thực thi cho mỗi submission.

Tuy nhiên, việc thực thi trực tiếp đồng nghĩa với việc mức độ cô lập thấp hơn so với Docker Sandbox. Vì vậy, Trusted Native Execution chỉ được sử dụng trong các môi trường mà ban tổ chức có khả năng kiểm soát hạ tầng và đánh giá được mức độ tin cậy của worker.

Trong BKDNAIOJ, chế độ này được kích hoạt thông qua cấu hình quản trị và thường được áp dụng cho các worker chuyên dụng thuộc hệ thống của đơn vị tổ chức. Việc hỗ trợ đồng thời hai chế độ thực thi giúp hệ thống linh hoạt hơn trong quá trình triển khai, cho phép cân bằng giữa yêu cầu bảo mật và hiệu năng xử lý tùy theo điều kiện thực tế.

3.6. Thiết kế API

3.6.1 Nguyên tắc thiết kế

BKDNAIOJ sử dụng kiến trúc RESTful API làm lớp giao tiếp thống nhất giữa các thành phần của hệ thống. Toàn bộ giao diện người dùng, máy chấm tình nguyện và các dịch vụ nền đều tương tác với hệ thống thông qua các API HTTP.

Các tài nguyên được tổ chức theo mô hình tài nguyên (resource-oriented), trong đó mỗi đối tượng nghiệp vụ như cuộc thi, phase, submission hay worker được biểu diễn bằng một endpoint riêng. Các thao tác tạo mới, truy vấn, cập nhật và xóa được ánh xạ tương ứng với các HTTP method chuẩn.

API được chia thành ba nhóm chính:

- **Contestant API:** phục vụ đăng nhập, đăng ký cuộc thi, nộp bài và xem kết quả.
- **Admin API:** phục vụ quản lý cuộc thi, tài nguyên chấm bài và máy chấm tình nguyện.
- **Worker API:** phục vụ nhận job, gửi heartbeat và báo cáo kết quả chấm bài.

Mọi phản hồi từ hệ thống đều sử dụng định dạng JSON thống nhất nhằm đơn giản hóa việc tích hợp giữa frontend, worker agent và backend.

3.6.2 Các nhóm API chính

❖ API quản lý submission

Nhóm API submission hỗ trợ toàn bộ vòng đời của một bài nộp, từ khởi tạo yêu cầu upload, xác nhận hoàn tất upload đến theo dõi trạng thái chấm bài.

Do kích thước submission có thể rất lớn, đặc biệt trong các phase final inference, hệ thống áp dụng cơ chế upload hai bước. API chỉ chịu trách nhiệm xác thực và quản lý metadata, trong khi dữ liệu thực tế được tải trực tiếp lên object storage thông qua các URL được cấp tạm thời.

Các API trong nhóm này bao gồm:

- Khởi tạo upload submission.
- Xác nhận upload hoàn tất.
- Truy vấn trạng thái submission.
- Xem lịch sử submission.

❖ API điều phối worker

Nhóm API worker phục vụ hệ thống volunteer computing. Các API này cho phép worker đăng ký tham gia cụm chấm bài, nhận tác vụ mới, gửi tín hiệu heartbeat và báo cáo kết quả sau khi hoàn thành.

Đây là lớp giao tiếp duy nhất giữa worker agent và hệ thống trung tâm, đóng vai trò kết nối giữa hàng đợi chấm bài và các máy thực thi phân tán.

Các API chính gồm:

- Đăng ký worker.
- Nhận job.
- Gia hạn trạng thái hoạt động.
- Gửi kết quả chấm bài.

❖ API leaderboard

Nhóm API leaderboard cung cấp dữ liệu xếp hạng cho thí sinh và quản trị viên.

Dữ liệu trả về được lấy từ các bảng leaderboard đã được tính toán trước nhằm đảm bảo thời gian phản hồi thấp ngay cả khi số lượng submission lớn. API hỗ trợ phân trang, tìm kiếm và đánh dấu vị trí của entry hiện tại trên bảng xếp hạng.

Các API chính gồm:

- Xem leaderboard của task-phase.
- Xem leaderboard của contest-phase.
- Xem bảng xếp hạng tổng hợp.

❖ **API quản trị**

Nhóm API quản trị hỗ trợ toàn bộ nghiệp vụ vận hành cuộc thi.

Thông qua các API này, quản trị viên có thể tạo cuộc thi, cấu hình phase, quản lý tài nguyên chấm bài, phê duyệt worker và theo dõi trạng thái hệ thống theo thời gian thực.

Các API chính gồm:

- Quản lý contest.
- Quản lý task và phase.
- Quản lý asset.
- Quản lý máy chấm tình nguyện.
- Theo dõi submission và scheduler

3.6.3 **Đặc tả các API nghiệp vụ trọng yếu**

Bảng 3.9: Đặc tả API khởi tạo nộp bài

Trường	Nội dung
Mô tả	Khởi tạo một bản ghi submission mới và cấp phát URL tạm thời (Presigned URL) để client tải tệp trực tiếp lên hệ thống lưu trữ (Object Storage).
Endpoint	/api/submissions/initiate
Phương thức	POST
Xác thực	Bearer JWT
Request	Thông tin phase và danh sách các tệp dữ liệu cần nộp (bao gồm tên và kích thước).
Response	Mã định danh submission (ID) cùng danh sách Presigned URL tương ứng cho từng tệp.
Kết quả	Submission được khởi tạo thành công và lưu ở trạng thái chờ tải tệp.

Bảng 3.10: Đặc tả API xác nhận hoàn tất tải tệp

Trường	Nội dung
Mô tả	Xác nhận quá trình tải tệp từ client lên lưu trữ đã hoàn tất, đồng thời đưa tác vụ vào hàng đợi để chờ chấm điểm.
Endpoint	/api/submissions/{id}/complete
Phương thức	POST
Xác thực	Bearer JWT
Request	Danh sách các tệp đã tải lên cùng thông tin kiểm tra tính toàn vẹn (ví dụ: Checksum, ETag).
Response	Trạng thái cập nhật mới nhất của submission.
Kết quả	Submission được chuyển sang trạng thái chờ xử lý (queued) và sẵn sàng được phân phối cho Worker.

Bảng 3.11: Đặc tả API nhận tác vụ chấm bài

Trường	Nội dung
Mô tả	Máy chấm tình nguyện chủ động gọi (poll) để yêu cầu nhận tác vụ chấm bài mới từ hệ thống phân phối.
Endpoint	/api/workers/jobs
Phương thức	GET
Xác thực	Worker Token
Response	Thông tin chi tiết của job, cấu hình môi trường thực thi và đường dẫn tải các tài nguyên (artifacts) cần thiết.
Lưu ý	Hệ thống có thể trả về phản hồi rỗng nếu không có job phù hợp với cấu hình của Worker, hoặc Worker đã đạt giới hạn xử lý đồng thời.
Kết quả	Worker nhận được một tác vụ tương thích và bắt đầu quá trình thực thi chấm điểm.

Bảng 3.12: Đặc tả API gửi kết quả chấm bài

Trường	Nội dung
Mô tả	Worker gửi trả kết quả đo lường và đánh giá sau khi hoàn thành quá trình thực thi chấm bài.
Endpoint	/api/workers/jobs/{id}/result
Phương thức	POST

Xác thực	Worker Token
Request	Điểm số đạt được, trạng thái thực thi (Thành công/Lỗi) và các số liệu thống kê tài nguyên (thời gian chạy, mức tiêu thụ RAM/VRAM).
Response	Trạng thái cập nhật cuối cùng của submission.
Kết quả	Điểm số được hệ thống ghi nhận thành công và tự động kích hoạt tiến trình cập nhật lại bảng xếp hạng.

Bảng 3.13: Đặc tả API truy vấn bảng xếp hạng (Leaderboard)

Trường	Nội dung
Mô tả	Trích xuất và trả về dữ liệu bảng xếp hạng hiện tại của một phase cụ thể trong cuộc thi.
Endpoint	/api/contests/{contestId}/phases/{phaseKey}/leaderboard
Phương thức	GET
Xác thực	Bearer JWT
Query Parameters	Các tham số phân trang (limit, offset) và tùy chọn điều hướng dữ liệu.
Response	Danh sách các bài nộp tốt nhất kèm thứ hạng, điểm số chi tiết và thông tin tổng hợp của thí sinh/đội thi.
Kết quả	Giao diện client nhận được dữ liệu để kết xuất (render) bảng xếp hạng thời gian thực.

3.7. Thiết kế Giao diện Người dùng

3.7.1 Routing và cấu trúc trang

Frontend của BKDNAIOJ được xây dựng theo mô hình Single Page Application (SPA) trên nền tảng React. Việc điều hướng giữa các chức năng được thực hiện phía client thông qua cơ chế routing, giúp giảm số lần tải lại trang và cải thiện trải nghiệm người dùng trong quá trình thi.

Hệ thống gồm 18 trang chức năng, được chia thành ba nhóm chính: các trang công khai dành cho người dùng chưa đăng nhập, các trang dành cho thí sinh và các trang quản trị dành cho Admin. Các trang quản trị được bảo vệ thông qua cơ chế phân quyền, đảm bảo chỉ những người dùng có quyền phù hợp mới có thể truy cập.

Bảng 3.14: Danh sách 18 trang và route tương ứng

	Trang	Route	Truy cập
1	Trang chủ	/	Đã đăng nhập
2	Đăng nhập	/login	Công khai
3	Đăng ký	/register	Công khai
4	Quên mật khẩu	/forgot-password	Công khai
5	Reset mật khẩu	/reset-password	Công khai (link email)
6	Danh sách cuộc thi	/contests	Đã đăng nhập
7	Chi tiết cuộc thi	/contests/:contestId	Đã đăng nhập
8	Phase Hub	/contests/:contestId/phases/:phaseKey	Contestant đã đăng ký
9	Danh sách bài	/problems	Đã đăng nhập
10	Bảng xếp hạng	/rankings	Đã đăng nhập
11	Quản lý đội	/teams	Đã đăng nhập
12	Bảng tin	/newsfeed	Đã đăng nhập
13	Tài liệu	/docs	Công khai
14	Tạo cuộc thi	/admin/contests/new	Admin
15	Cài đặt cuộc thi	/admin/contests/:contestId/setup	Admin
16	Quản lý người dùng	/admin/users	Admin
17	Quản lý workers	/admin/workers	Admin
18	Scheduler dashboard	/admin/scheduler	Admin

Cách tổ chức này giúp tách biệt rõ ràng các nhóm chức năng theo vai trò người dùng, đồng thời hỗ trợ mở rộng hệ thống trong tương lai mà không ảnh hưởng đến cấu trúc điều hướng hiện có.

3.7.2 Quản lý trạng thái giao diện

Frontend phân chia trạng thái ứng dụng thành ba nhóm với trách nhiệm riêng biệt nhằm giảm sự phụ thuộc giữa các thành phần và nâng cao khả năng bảo trì.

Server State bao gồm các dữ liệu được lấy từ hệ thống backend như thông tin cuộc thi, submission, leaderboard và trạng thái worker. Các dữ liệu này được lưu đệm

cục bộ và đồng bộ định kỳ với máy chủ nhằm giảm số lượng yêu cầu mạng không cần thiết trong khi vẫn đảm bảo tính cập nhật của dữ liệu.

Authentication State lưu trữ thông tin xác thực của người dùng đang đăng nhập, bao gồm thông tin định danh và quyền hạn truy cập. Trạng thái này được duy trì xuyên suốt phiên làm việc và được sử dụng để kiểm soát quyền truy cập đến các trang chức năng khác nhau.

UI State bao gồm các trạng thái cục bộ của giao diện như cửa sổ hội thoại, tab đang được chọn hoặc trạng thái hiển thị của các thành phần giao diện. Các trạng thái này chỉ tồn tại trong phạm vi từng màn hình và không cần chia sẻ trên toàn bộ ứng dụng.

Việc tách biệt ba loại trạng thái giúp frontend dễ quản lý hơn, đồng thời hạn chế các lần cập nhật giao diện không cần thiết khi dữ liệu thay đổi.

3.7.3 Phase Hub - Màn hình trung tâm của thí sinh

Phase Hub là trang thí sinh sử dụng nhiều nhất trong thời gian thi, tổ chức theo bốn tab trong cùng ngữ cảnh của một phase. Thiết kế một trang đơn thay vì điều hướng đa trang cho phép thí sinh theo dõi toàn bộ trạng thái của phase hiện tại mà không mất ngữ cảnh.

Giao diện được chia thành bốn nhóm chức năng chính:

- **Overview:** hiển thị thông tin tổng quan của phase, bao gồm thời gian diễn ra, giới hạn nộp bài và các quy định liên quan.
- **Problems:** hiển thị danh sách bài toán thuộc phase cùng khu vực thực hiện nộp bài.
- **Submission History:** cung cấp lịch sử các lần nộp bài và trạng thái xử lý tương ứng.
- **Standings:** hiển thị bảng xếp hạng của phase.

Thiết kế tập trung này cho phép thí sinh theo dõi tiến trình thi đấu, thực hiện nộp bài và quan sát kết quả trong cùng một giao diện, hạn chế việc chuyển đổi qua lại giữa nhiều trang khác nhau trong thời gian thi. Chi tiết hình ảnh giao diện được trình bày trong chương 4.

3.8. Thiết kế Bảo mật

3.8.1 Xác thực và phân quyền

BKDNAIOJ áp dụng cơ chế xác thực dựa trên token nhằm hỗ trợ mô hình ứng dụng web hiện đại và tách biệt hoàn toàn frontend với backend. Sau khi đăng nhập thành

công, người dùng được cấp một token xác thực và sử dụng token này trong các yêu cầu tiếp theo đến hệ thống.

Hệ thống phân chia người dùng thành nhiều nhóm quyền khác nhau, bao gồm thí sinh (contestant), quản trị viên (admin) và các vai trò phục vụ công tác tổ chức cuộc thi. Mỗi nhóm chỉ được phép truy cập các chức năng phù hợp với phạm vi trách nhiệm của mình. Việc kiểm soát truy cập được thực hiện tập trung thông qua các lớp middleware xác thực và phân quyền trước khi yêu cầu được chuyển tới các thành phần xử lý nghiệp vụ.

Bên cạnh người dùng thông thường, hệ thống còn hỗ trợ một cơ chế xác thực độc lập dành cho máy chấm tình nguyện. Các worker không sử dụng phiên đăng nhập của người dùng mà được cấp định danh xác thực riêng khi tham gia hệ thống. Thiết kế này giúp tách biệt hoàn toàn hai nhóm đối tượng có bản chất khác nhau: người dùng tương tác qua giao diện web và các tiến trình tự động thực hiện chấm bài.

Việc phân tách xác thực theo từng nhóm đối tượng giúp giảm nguy cơ nhầm lẫn quyền hạn, đồng thời đơn giản hóa việc quản lý bảo mật trong quá trình vận hành.

3.8.2 Giới hạn tần suất truy cập

Ngoài xác thực và phân quyền, hệ thống còn áp dụng cơ chế giới hạn tần suất truy cập nhằm giảm thiểu nguy cơ lạm dụng tài nguyên và các cuộc tấn công tự động.

Đối với các chức năng công khai như đăng nhập, đăng ký tài khoản hoặc khôi phục mật khẩu, việc giới hạn được thực hiện dựa trên địa chỉ truy cập nhằm hạn chế các hành vi dò mật khẩu hoặc tạo tài khoản hàng loạt. Đối với các chức năng yêu cầu xác thực, đặc biệt là chức năng nộp bài, việc giới hạn được thực hiện theo từng người dùng nhằm ngăn chặn việc sử dụng công cụ tự động tạo ra số lượng lớn yêu cầu trong thời gian ngắn.

Cơ chế này hoạt động độc lập với các quy tắc nghiệp vụ của cuộc thi. Nhờ đó, hệ thống vừa có thể kiểm soát hành vi bất thường ở tầng hạ tầng, vừa duy trì các giới hạn chuyên biệt của từng cuộc thi như số lượt nộp bài tối đa hoặc thời gian mở của phase.

3.8.3 Bảo vệ dữ liệu cuộc thi

Các tài nguyên phục vụ chấm bài như submission, dataset, judge script và các tệp hỗ trợ khác được lưu trữ tập trung trên hệ thống object storage thay vì truyền trực tiếp qua máy chủ ứng dụng.

Quyền truy cập tới các tài nguyên này được cấp phát tạm thời theo từng tác vụ cụ thể. Thí sinh chỉ được phép truy cập các tài nguyên công khai liên quan đến cuộc thi, trong khi các dữ liệu đánh giá bí mật chỉ được cấp quyền cho các worker đang thực hiện nhiệm vụ chấm bài tương ứng.

Để đảm bảo tính toàn vẹn dữ liệu trong quá trình truyền tải và thực thi, mỗi artifact được gán một giá trị băm kiểm tra. Worker có trách nhiệm xác minh lại tính toàn vẹn của tệp trước khi sử dụng. Cơ chế này giúp phát hiện các trường hợp dữ liệu bị hỏng hoặc không khớp với phiên bản do ban tổ chức cung cấp.

3.8.4 Bảo vệ môi trường thực thi

Một trong những yêu cầu quan trọng nhất của hệ thống là xử lý an toàn mã nguồn và mô hình do thí sinh cung cấp. Đây là các thành phần không đáng tin cậy và có thể gây ảnh hưởng tới hạ tầng chấm bài nếu không được kiểm soát phù hợp.

BKDNAIOJ áp dụng cơ chế cô lập môi trường thực thi đối với các phase yêu cầu chạy chương trình của thí sinh. Các worker thực hiện chấm bài trong một môi trường tách biệt với hệ điều hành chính, hạn chế khả năng truy cập tài nguyên ngoài phạm vi được cấp phép. Đồng thời, hệ thống áp dụng các giới hạn về tài nguyên thực thi nhằm tránh tình trạng một submission tiêu thụ quá mức CPU, bộ nhớ hoặc các tài nguyên dùng chung khác.

Trong một số trường hợp đặc biệt, hệ thống hỗ trợ chế độ thực thi trực tiếp trên máy chủ worker nhằm tận dụng tối đa năng lực phần cứng, đặc biệt là GPU. Tuy nhiên, chế độ này chỉ được cấp quyền cho các worker đáng tin cậy và được quản trị viên phê duyệt trước khi sử dụng.

3.8.5 Kiến trúc bảo mật nhiều lớp

Các cơ chế xác thực, phân quyền, giới hạn tần suất truy cập, kiểm soát quyền truy cập dữ liệu và cô lập môi trường thực thi được thiết kế như các lớp bảo vệ độc lập nhưng bổ trợ lẫn nhau.

Cách tiếp cận nhiều lớp này giúp hệ thống không phụ thuộc vào một cơ chế bảo vệ duy nhất. Ngay cả khi một lớp phòng vệ gặp sự cố hoặc bị vượt qua, các lớp còn lại vẫn tiếp tục giảm thiểu rủi ro và bảo vệ tài nguyên của cuộc thi cũng như hạ tầng chấm bài phân tán.

CHƯƠNG 4: CÀI ĐẶT VÀ KIỂM THỬ

4.1. Môi trường và Công cụ Phát triển

4.1.1 Ngôn ngữ lập trình và framework

BKDNAIOJ được phát triển theo mô hình đa ngôn ngữ, trong đó mỗi thành phần của hệ thống sử dụng công nghệ phù hợp với vai trò và yêu cầu kỹ thuật tương ứng.

Backend API được xây dựng bằng Go 1.23 kết hợp framework Echo. Phần backend chịu trách nhiệm xử lý toàn bộ nghiệp vụ của hệ thống, bao gồm quản lý cuộc thi, submission, leaderboard, worker và các tác vụ nền. Mã nguồn được tổ chức thành các module riêng biệt cho API, truy cập dữ liệu, xử lý leaderboard và điều phối worker.

Frontend được xây dựng bằng React 19 kết hợp TypeScript và Vite. React được sử dụng để phát triển giao diện dạng Single Page Application (SPA), trong khi TypeScript hỗ trợ kiểm tra kiểu dữ liệu trong quá trình phát triển. Vite đảm nhiệm quá trình build và phát triển cục bộ với khả năng cập nhật giao diện theo thời gian thực.

Máy chấm tình nguyện được phát triển bằng Python 3.11. Thành phần này thực hiện các nhiệm vụ tải artifact, thực thi quá trình inference, chạy judge script và gửi kết quả trở lại hệ thống. Worker được đóng gói dưới dạng package Python và có thể cài đặt trực tiếp thông qua trình quản lý gói pip.

Ngoài các ngôn ngữ lập trình chính, hệ thống còn sử dụng Docker để hỗ trợ triển khai môi trường phát triển và thực thi sandbox đối với các submission yêu cầu chạy mã nguồn của thí sinh.

4.1.2 Công cụ hỗ trợ phát triển

Trong quá trình phát triển BKDNAIOJ, một số công cụ hỗ trợ được sử dụng nhằm đơn giản hóa việc quản lý mã nguồn, cơ sở dữ liệu và kiểm thử.

Đối với tầng truy cập dữ liệu, dự án sử dụng **sqlc** để sinh mã Go từ các truy vấn SQL được định nghĩa sẵn. Thay vì sử dụng ORM, toàn bộ câu lệnh truy vấn được viết trực tiếp bằng SQL và được sqlc chuyển đổi thành các interface, struct và phương thức tương ứng trong Go. Cách tiếp cận này giúp duy trì tính rõ ràng của SQL đồng thời đảm bảo tính nhất quán giữa schema cơ sở dữ liệu và mã nguồn ứng dụng.

Quản lý thay đổi cơ sở dữ liệu được thực hiện bằng **goose**. Mỗi thay đổi schema được lưu dưới dạng một migration độc lập và được quản lý thông qua hệ thống quản lý phiên bản mã nguồn. Nhờ đó, môi trường phát triển và môi trường triển khai có thể được đồng bộ hóa dễ dàng, đồng thời hỗ trợ việc tái tạo hoặc khôi phục cơ sở dữ liệu tại các mốc phát triển khác nhau.

Đối với volunteer agent, hệ thống sử dụng **pytest** để xây dựng bộ kiểm thử tự động. Các thành phần như API giả lập, artifact cache và quy trình thực thi job được kiểm thử độc lập nhằm đảm bảo tính ổn định trước khi phát hành phiên bản mới của agent.

Bên cạnh đó, Git và GitHub được sử dụng để quản lý phiên bản mã nguồn, trong khi Docker Compose hỗ trợ khởi tạo nhanh môi trường phát triển bao gồm PostgreSQL, Redis và MinIO.

4.1.3 Cấu trúc thư mục dự án

Mã nguồn BKDNAIOJ được tổ chức thành ba thành phần chính tương ứng với ba hệ thống độc lập trong kiến trúc tổng thể.

```
backend/  
frontend/  
volunteer-agent/
```

Thư mục **backend** chứa mã nguồn của API server được phát triển bằng Go. Mã nguồn được tổ chức theo các lớp chức năng như middleware, handler, service và tầng truy cập dữ liệu. Ngoài các API phục vụ frontend, backend còn chứa các thành phần nền như LeaderboardBridge và Worker Timeout Watcher.

Thư mục **frontend** chứa ứng dụng React phục vụ giao diện người dùng. Mã nguồn được tổ chức theo từng miền nghiệp vụ như xác thực, cuộc thi, bảng xếp hạng và quản trị hệ thống nhằm thuận tiện cho việc mở rộng và bảo trì.

Thư mục **volunteer-agent** là một package Python độc lập, được phân phối dưới tên gọi olpai-volunteer-agent. Thành phần này bao gồm các module thực hiện benchmark phần cứng, artifact cache, nhận job, thực thi inference và gửi kết quả về hệ thống.

Ba thành phần được phát triển và kiểm thử độc lập, giao tiếp với nhau thông qua HTTP API và judging contract đã được định nghĩa trước. Cách tổ chức này giúp mỗi thành phần có thể được triển khai, nâng cấp hoặc bảo trì mà không ảnh hưởng trực tiếp tới các thành phần còn lại.

4.2. Cài đặt Backend

4.2.1 Luồng xử lý nộp bài

Luồng nộp bài được hiện thực theo mô hình upload hai bước nhằm tránh truyền trực tiếp các tệp dung lượng lớn qua API Server. Trong triển khai hiện tại, phase output-only cho phép tệp tối đa 50 MB, trong khi phase final inference cho phép tới 500 MB.

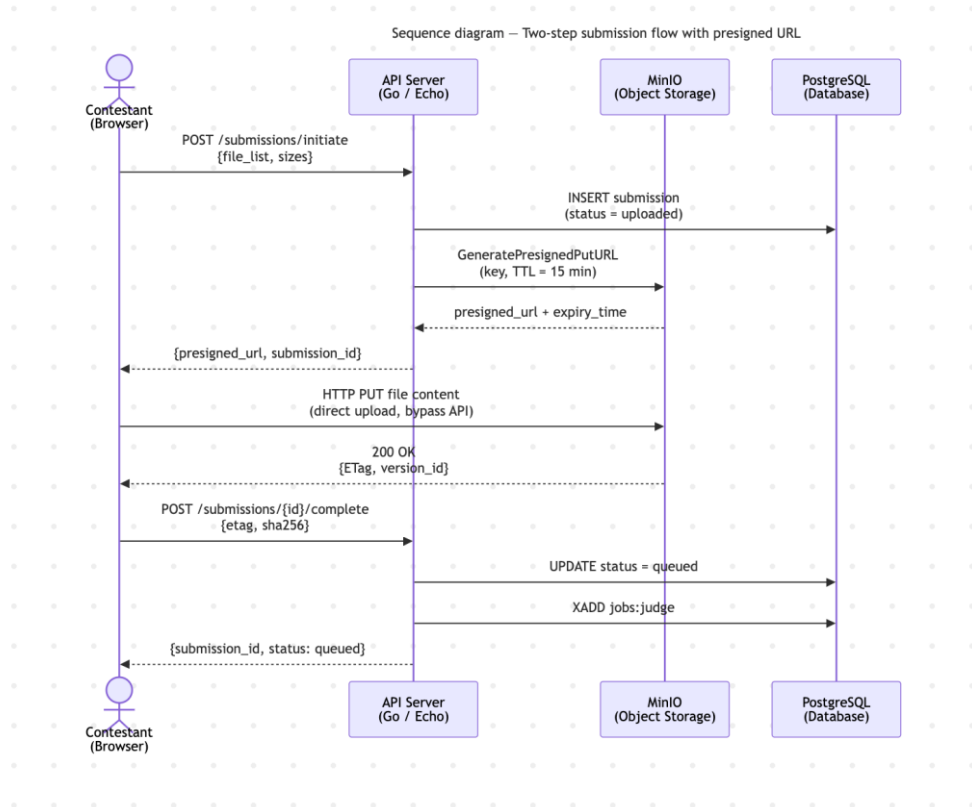
Bước đầu tiên được thực hiện qua endpoint POST /api/submissions/initiate. Handler InitiateUpload kiểm tra các điều kiện nghiệp vụ gồm: phase đang mở, người

dùng có contest entry hợp lệ, chưa vượt quá số lần nộp cho phép và kích thước tệp nằm trong giới hạn. Nếu hợp lệ, hệ thống tạo bản ghi submission và sinh presigned URL từ MinIO với thời hạn 15 phút.

Sau khi tải tệp trực tiếp lên MinIO, client gọi endpoint POST /api/submissions/{id}/complete. Handler CompleteUpload xác minh object key và kích thước thực tế của tệp trên MinIO. Nếu kiểm tra thành công, hệ thống ghi thông tin vào bảng submission_files, chuyển trạng thái submission sang queued và đưa job vào Redis Stream jobs:judge để chờ worker xử lý.

Nhờ cơ chế này, API Server chỉ xử lý metadata thay vì dữ liệu tệp thực tế, giúp giảm tải mạng và tăng khả năng phục vụ đồng thời khi nhiều thí sinh nộp bài.

Hình 4.1 trình bày trình tự xử lý của luồng nộp bài hai bước sử dụng presigned URL.



Hình 4.1: Sequence diagram luồng nộp bài sử dụng Presigned URL

4.2.2 Cơ chế phân phối job

Cơ chế phân phối job được hiện thực thông qua hai thành phần chính: hàm kiểm tra tính tương thích CanAcceptJob và thuật toán phân công ClaimNext.

Hàm `CanAcceptJob` nhận thông tin worker và job, sau đó kiểm tra các điều kiện cần thiết trước khi cho phép nhận việc. Các điều kiện bao gồm số slot còn trống theo loại job, khả năng thực thi phase final (Docker hoặc trusted native mode), yêu cầu GPU và giới hạn bộ nhớ. Hàm trả về kết quả dạng boolean và được sử dụng cho mọi quyết định phân công tác vụ.

Thuật toán `ClaimNext` được triển khai theo chiến lược Capability-aware FIFO. Khi worker gửi yêu cầu nhận việc, backend đọc một cửa sổ các job đang chờ trong Redis Stream, duyệt theo thứ tự FIFO và chọn job tương thích đầu tiên. Sau khi tìm được job phù hợp, hệ thống thực hiện thao tác claim nguyên tử trên Redis bằng Lua Script để đảm bảo một job chỉ được giao cho duy nhất một worker.

Khi claim thành công, backend tạo bản ghi lease trong bảng `volunteer_worker_claims`, cập nhật trạng thái submission sang running và trả về thông tin cần thiết để worker bắt đầu thực thi.

Trong quá trình xử lý, worker gửi heartbeat định kỳ để gia hạn lease. Nếu lease hết hạn do worker mất kết nối hoặc gặp sự cố, Timeout Watcher sẽ tự động thu hồi claim và đưa submission trở lại trạng thái queued để một worker khác có thể tiếp tục xử lý.

4.2.3 Cập nhật bảng xếp hạng

BKDNAIOJ hiện thực hai cơ chế cập nhật leaderboard: **full recompute** và **incremental update**.

Đường full recompute được cài đặt bằng một câu truy vấn SQL sử dụng nhiều CTE lồng nhau. Truy vấn này lựa chọn submission đại diện của mỗi entry theo cấu hình phase, tính điểm hiện thị, xác định thứ hạng bằng hàm `DENSE_RANK()` và ghi kết quả vào các bảng leaderboard thông qua cơ chế UPSERT. Việc sử dụng UPSERT giúp quá trình cập nhật diễn ra nguyên tử, tránh tình trạng người dùng nhìn thấy bảng xếp hạng trống hoặc không nhất quán trong khi đang tính toán lại.

Đường incremental update được sử dụng trong trường hợp submission mới chỉ làm thay đổi điểm của một entry mà không ảnh hưởng đến điểm cao nhất của phase. Khi đó hệ thống chỉ cập nhật bản ghi liên quan trong bảng leaderboard và Redis Sorted Set tương ứng, giúp giảm đáng kể chi phí xử lý so với việc tính toán lại toàn bộ phase.

Quá trình cập nhật được thực hiện bất đồng bộ bởi goroutine `LeaderboardBridge`. Thành phần này nhận kết quả chấm bài từ Redis Stream `jobs:results`, sau đó cập nhật tuần tự ba tầng leaderboard gồm task-phase, contest-phase và global ranking. Thiết kế

bất đồng bộ giúp quá trình chấm bài không bị ảnh hưởng bởi chi phí tính toán bằng xếp hạng, đồng thời đảm bảo dữ liệu hiển thị luôn được đồng bộ sau khi có kết quả mới.

4.2.4 Xử lý sự cố và khôi phục

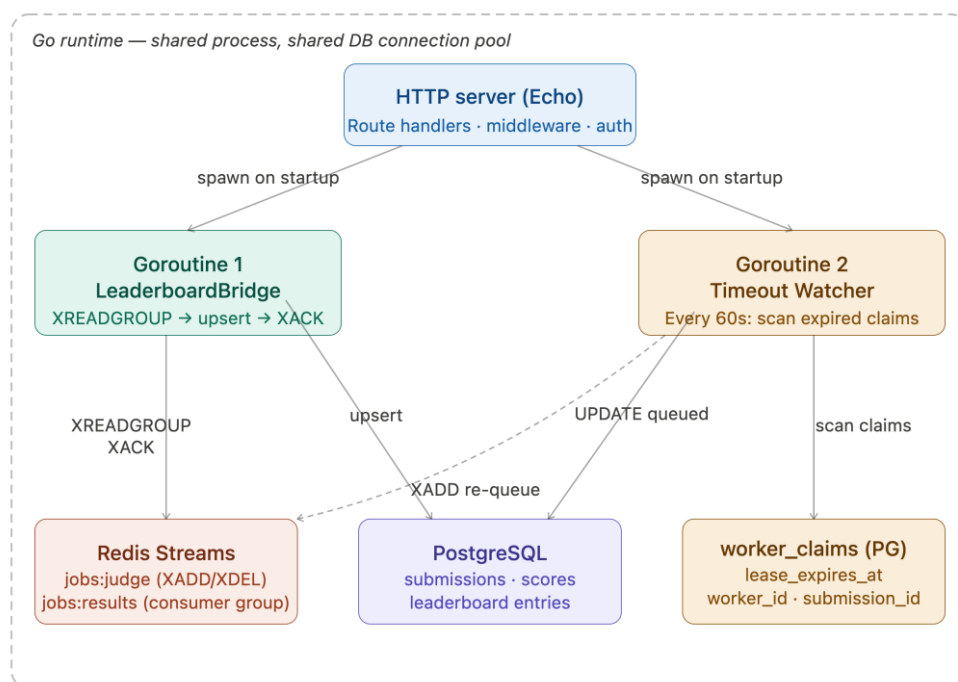
BKDNAIOJ triển khai cơ chế phục hồi tự động thông qua goroutine TimeoutWatcher, chạy định kỳ mỗi 60 giây để giám sát trạng thái các worker và submission đang thực thi.

Chức năng thứ nhất là **stale claim cleanup**. Hệ thống tìm các claim đã vượt quá thời gian lease do worker ngừng gửi heartbeat hoặc gặp sự cố trong quá trình xử lý. Với mỗi claim hết hạn, bản ghi claim được thu hồi, submission được đưa trở lại trạng thái queued và job được thêm lại vào Redis Stream để worker khác có thể tiếp nhận. Đồng thời, bộ đếm lỗi của worker được tăng lên nhằm phục vụ giám sát và đánh giá độ ổn định của worker.

Chức năng thứ hai là **orphan recovery**. Thành phần này phát hiện các submission đang ở trạng thái running nhưng không còn claim hợp lệ tương ứng trong cơ sở dữ liệu. Đây là tình huống có thể phát sinh do lỗi bất thường trong quá trình tạo hoặc cập nhật claim. Các submission như vậy sẽ được chuyển về trạng thái queued để tiếp tục được xử lý.

Nhờ hai cơ chế trên, hệ thống đảm bảo rằng mọi submission cuối cùng đều quay trở lại hàng đợi hoặc được xử lý thành công, tránh tình trạng bị kẹt vĩnh viễn ở trạng thái trung gian.

Hình 4.2 minh họa cách HTTP Server, LeaderboardBridge và TimeoutWatcher được triển khai dưới dạng các goroutine độc lập và hoạt động song song trong cùng tiến trình backend.



Hình 4.2: Hai goroutine nền chạy song song với HTTP server

4.3. Cài đặt Volunteer Agent

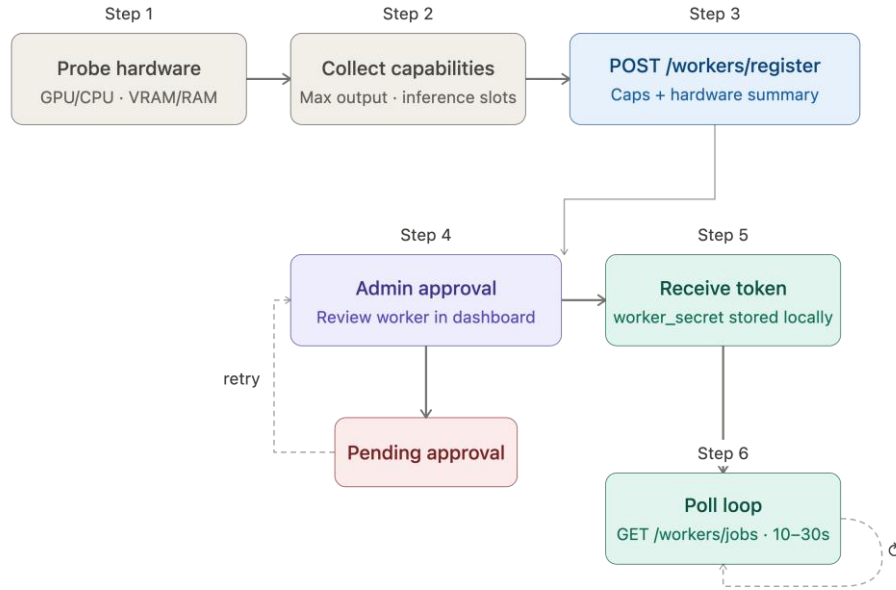
4.3.1 Thiết lập và đăng ký worker

Volunteer Agent được phân phối dưới dạng package Python `olpai-volunteer-agent` và hỗ trợ quy trình thiết lập thông qua một setup wizard chạy trên giao diện dòng lệnh. Mục tiêu của wizard là tự động thu thập thông tin phần cứng, đánh giá năng lực thực tế của máy và đăng ký worker với hệ thống mà không yêu cầu người dùng cấu hình thủ công.

Quy trình thiết lập gồm sáu bước chính: thu thập thông tin phần cứng, kiểm tra Docker và GPU, thực hiện benchmark, đề xuất cấu hình số slot xử lý đồng thời, xác thực với máy chủ và hoàn tất đăng ký worker. Sau khi đăng ký thành công, worker nhận một token định danh duy nhất để sử dụng trong các lần khởi động tiếp theo.

Trong bước benchmark, agent thực hiện một số phép đo đơn giản nhằm đánh giá năng lực thực tế của máy như hiệu năng CPU, tốc độ đọc ghi đĩa, thời gian khởi động Docker container và khả năng tính toán của GPU nếu CUDA khả dụng. Kết quả được tổng hợp thành một `WorkerProfile` và gửi về backend dưới dạng hồ sơ đăng ký.

Thông tin benchmark được lưu trong trường capabilities của worker và được sử dụng bởi scheduler trong quá trình phân phối job. Điều này giúp hệ thống đưa ra quyết định dựa trên năng lực thực tế của máy thay vì chỉ dựa trên thông tin cấu hình do người dùng khai báo.



Hình 4.3: Luồng sáu bước của setup wizard

4.3.2 Vòng lặp nhận và xử lý job

Khi agent được khởi động bằng lệnh `olpai start`, hệ thống tạo N luồng worker song song (N bằng tổng số slot được cấu hình) cùng một luồng heartbeat tổng thể của máy. Luồng heartbeat này định kỳ gửi trạng thái tài nguyên của worker về backend, bao gồm CPU, RAM và trạng thái hoạt động hiện tại.

Mỗi worker thread thực hiện vòng lặp poll liên tục: gửi yêu cầu nhận việc tới API, nhận job nếu có và bắt đầu quá trình thực thi. Trong thời gian xử lý, ba tác vụ được thực hiện song song. Thứ nhất, luồng heartbeat của job gửi tín hiệu gia hạn lease định kỳ mỗi 30 giây để tránh job bị thu hồi. Thứ hai, bộ giám sát tài nguyên lấy mẫu mức sử dụng RAM và GPU trong suốt quá trình chạy nhằm ghi nhận các chỉ số thực thi thực tế. Thứ ba, luồng xử lý chính thực hiện các bước tải artifact, chuẩn bị workspace, chạy inference hoặc judge script và thu thập kết quả.

Toàn bộ quá trình được bao bọc trong cơ chế cleanup bắt buộc nhằm đảm bảo heartbeat được dừng và workspace tạm thời được xóa ngay cả khi job gặp lỗi hoặc bị hủy giữa chừng. Thiết kế này giúp tránh rò rỉ tài nguyên và đảm bảo agent có thể hoạt động ổn định trong thời gian dài.

Sau khi hoàn thành, agent gửi kết quả về backend kèm theo các số liệu thực thi như thời gian chạy, RAM đỉnh điểm và VRAM đỉnh điểm. Các thông tin này được lưu trong bảng `job_execution_logs` phục vụ giám sát hệ thống và phân tích hiệu năng.

Để tăng khả năng chịu lỗi trong môi trường volunteer, việc gửi kết quả sử dụng cơ chế retry với exponential backoff trong tối đa 180 giây, cho phép xử lý các sự cố mạng tạm thời mà không làm mất kết quả chấm bài.

4.3.3 Thực thi phase final

Đối với các phase non-final (public_test, private_test), agent chỉ cần thực thi judge script trực tiếp trên file submission của thí sinh. Judge script đọc dữ liệu trong thư mục submission/, sử dụng tài nguyên trong assets/ và trả kết quả theo judging contract đã định nghĩa.

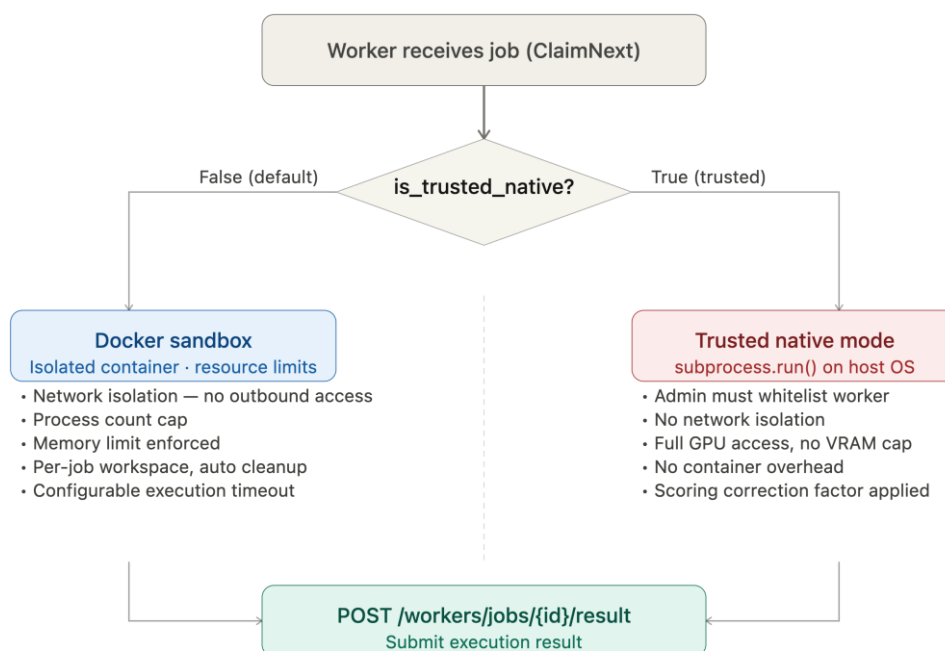
Đối với các phase final (final_public, final_private), quá trình thực thi gồm hai bước tuần tự: **inference** và **judging**. Trước khi chạy, agent tạo workspace theo cấu trúc chuẩn gồm bốn thư mục: submission/, assets/, generated/ và output/. Trong đó, thư mục submission/ luôn được tải mới cho từng job, còn các tài nguyên trong assets/ được quản lý thông qua cơ chế cache dựa trên mã băm SHA-256 nhằm giảm số lần tải lặp lại giữa các submissions cùng phase.

Quá trình inference được thực hiện theo một trong hai chế độ như minh họa ở Hình 4.4.

Trong chế độ **Docker Sandbox**, agent khởi tạo container từ image olpai-final-runtime và áp dụng các ràng buộc bảo mật gồm vô hiệu hóa mạng, giới hạn bộ nhớ và giới hạn số tiến trình. Workspace của job được mount vào container và script infer.py được thực thi cho đến khi hoàn thành hoặc vượt quá timeout_secs. Khi timeout xảy ra, container bị dừng cưỡng bức và job được đánh dấu thất bại.

Trong chế độ **Trusted Native Mode**, inference được thực thi trực tiếp trên môi trường Python của máy host. Chế độ này cho phép tận dụng GPU CUDA mà không phát sinh chi phí khởi tạo container, phù hợp với các worker GPU chuyên dụng đã được admin phê duyệt. Trước khi thực thi, agent kiểm tra inference_entrypoint để đảm bảo đường dẫn nằm bên trong thư mục submission và không truy cập ra ngoài workspace. Timeout vẫn được áp dụng thông qua cơ chế hủy tiến trình cưỡng bức khi vượt quá thời gian cho phép.

Sau khi inference hoàn tất, kết quả được ghi vào thư mục generated/. Agent tiếp tục thực thi judge script để tính điểm và sinh kết quả cuối cùng. Từ góc nhìn của judge script, quá trình chấm điểm final phase hoàn toàn giống output-only phase vì đầu vào đều là thư mục chứa kết quả dự đoán đã được chuẩn hóa.

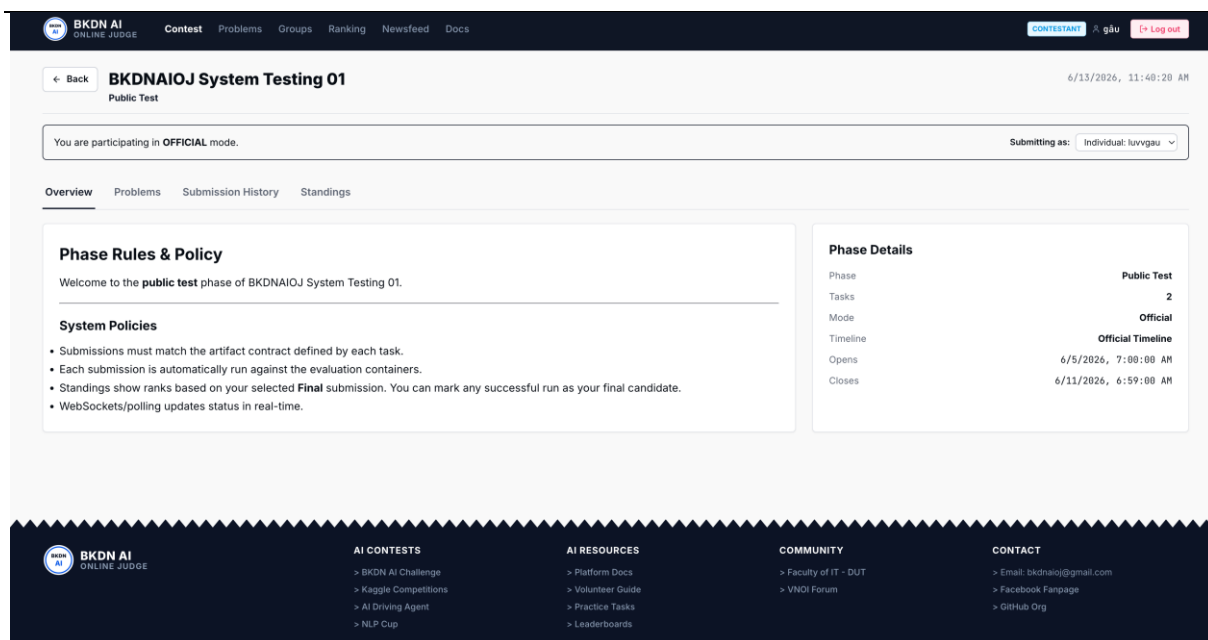


Hình 4.4: Hai chế độ thực thi của phase final: Docker Sandbox và Trusted Native.

4.4. Giao diện Người dùng

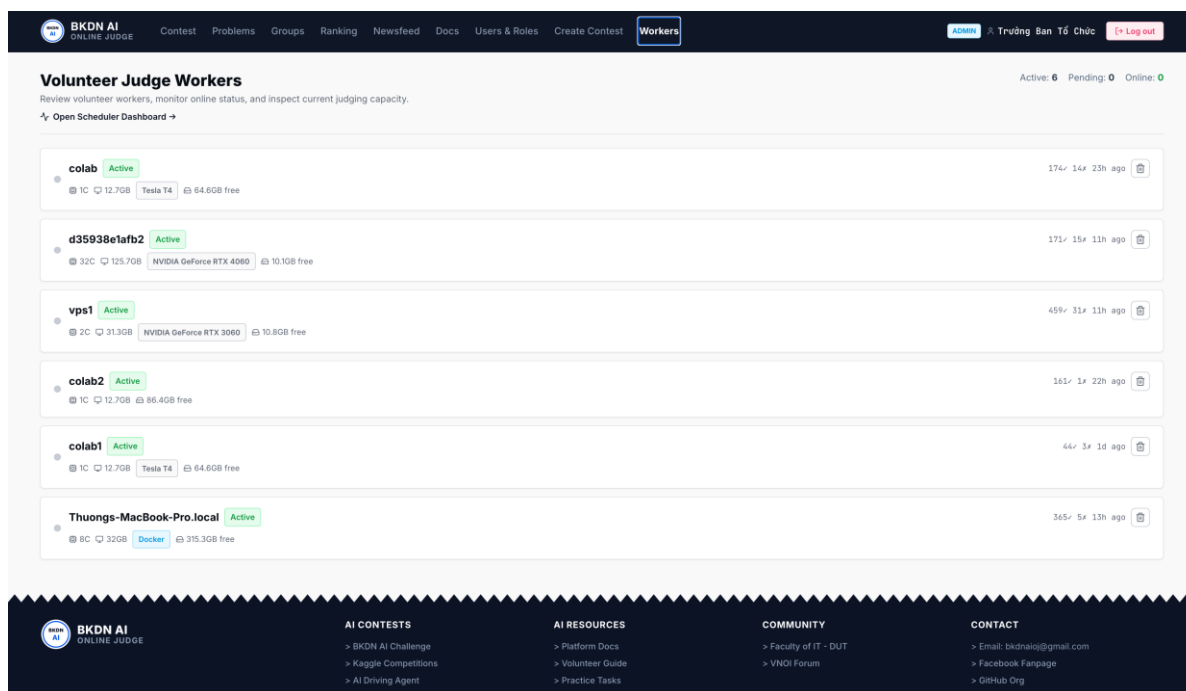
Giao diện BKDNAIOJ được xây dựng dưới dạng Single Page Application (SPA) với 18 trang chức năng, được chia thành hai nhóm chính: nhóm dành cho thí sinh và nhóm dành cho quản trị viên. Một số màn hình tiêu biểu của hệ thống được trình bày dưới đây.

Phase Hub (Hình 4.5) là màn hình trung tâm của thí sinh trong thời gian diễn ra cuộc thi. Màn hình hiển thị thông tin phase, thống kê nộp bài và lịch sử submission. Kết quả chấm bài và thứ hạng được cập nhật tự động thông qua cơ chế polling, giúp thí sinh theo dõi tiến trình xử lý mà không cần tải lại trang.



Hình 4.5: Phase Hub - tổng quan phase

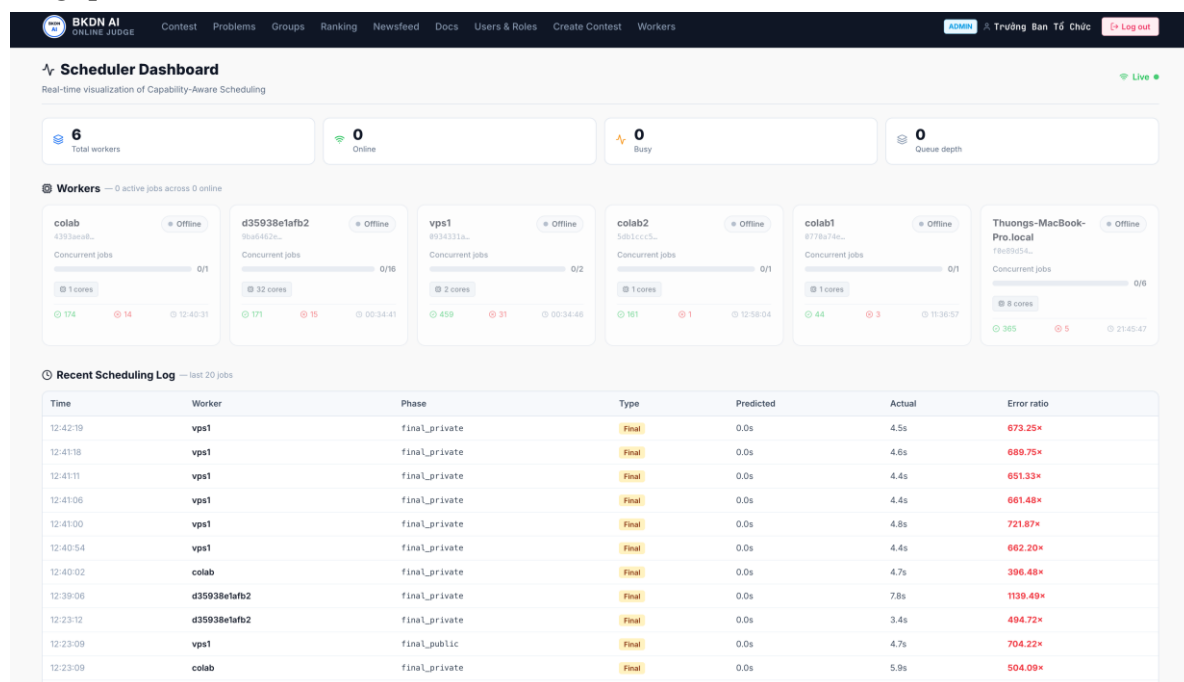
Admin Workers (Hình 4.6) hiển thị danh sách máy chấm tình nguyện đã đăng ký cùng trạng thái hoạt động hiện tại. Trang sử dụng cơ chế Server-Sent Events (SSE) để nhận cập nhật từ backend theo thời gian thực, cho phép quản trị viên nhanh chóng phát hiện worker mới kết nối, mất kết nối hoặc thay đổi trạng thái.



Hình 4.6: Admin Workers - danh sách worker

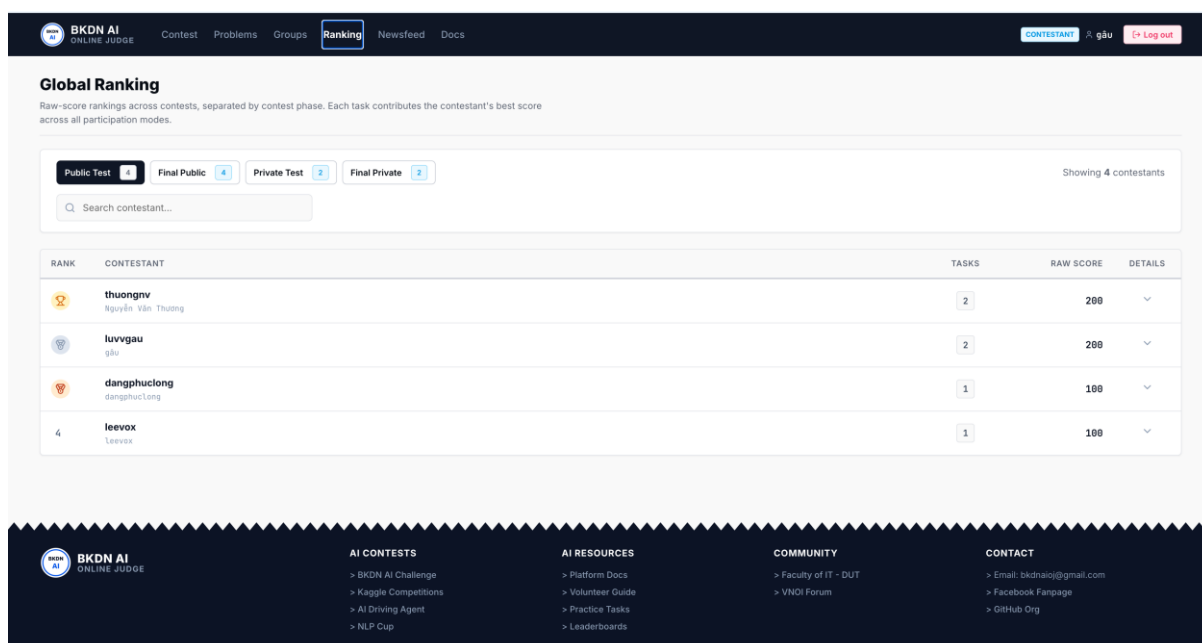
Scheduler Dashboard (Hình 4.7) cung cấp góc nhìn tổng quan về hoạt động của hệ thống chấm bài. Màn hình hiển thị số lượng submission theo từng trạng thái và tình

trạng hàng đợi hiện tại, hỗ trợ quản trị viên theo dõi và phát hiện các dấu hiệu tắc nghẽn trong quá trình vận hành.



Hình 4.7: Scheduler Dashboard - queue depth và phân bố trạng thái submission

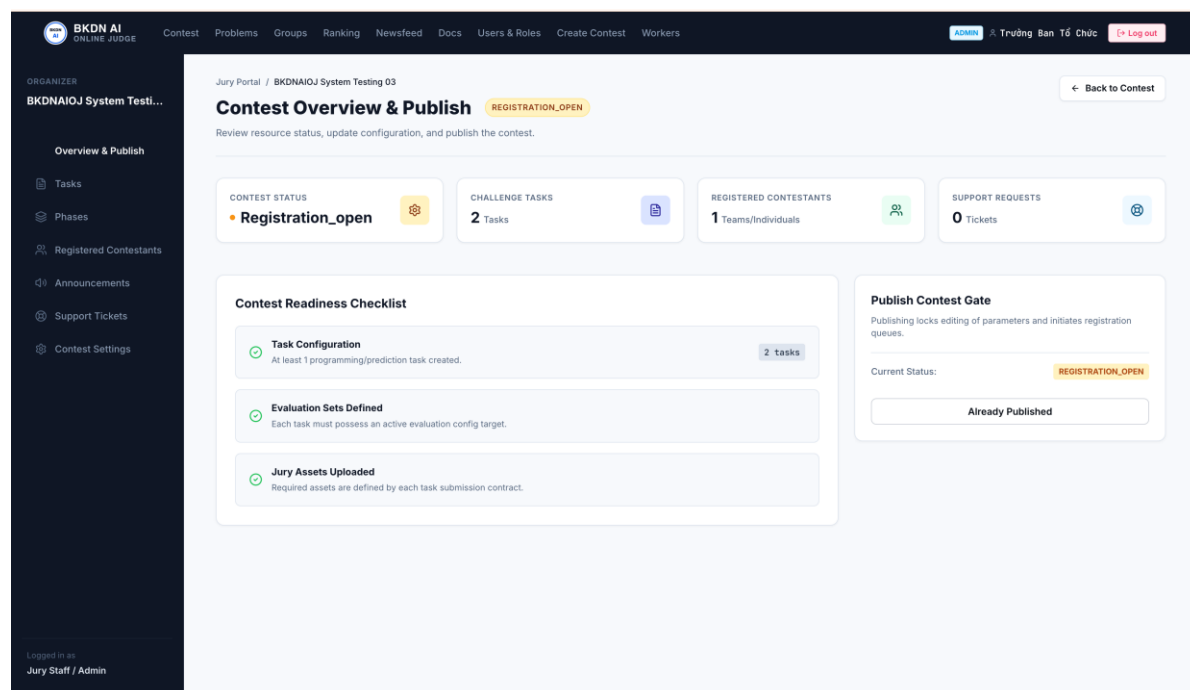
Global Rankings (Hình 4.8) là giao diện hiển thị bảng xếp hạng tổng hợp. Màn hình cho phép người dùng theo dõi điểm số và thứ hạng trên nhiều bài toán hoặc nhiều cuộc thi, đồng thời xem chi tiết điểm thành phần tương ứng.



Hình 4.8: Global Rankings - bảng xếp hạng tổng hợp với score breakdown

Admin Setup (Hình 4.9) là wizard hỗ trợ tạo cuộc thi theo nhiều bước. Giao diện hướng dẫn quản trị viên lần lượt cấu hình thông tin cuộc thi, bài toán, phase và tài nguyên

chấm bài. Dữ liệu đã nhập được lưu tạm trong quá trình thao tác nhằm giảm nguy cơ mất thông tin khi trang được tải lại.



Hình 4.9: Admin Setup - wizard tạo cuộc thi nhiều bước

4.5. Kiểm thử Hệ thống

4.5.1 Phương pháp kiểm thử

Hoạt động kiểm thử được thực hiện theo hai nhóm. Thứ nhất là kiểm thử chức năng nhằm xác nhận các chức năng nghiệp vụ hoạt động đúng theo đặc tả use case đã trình bày trong Chương 3. Thứ hai là kiểm thử tích hợp và hiệu năng (integration and performance testing) nhằm đánh giá khả năng phối hợp giữa các thành phần của hệ thống và khả năng xử lý khi số lượng submission tăng lên.

Các kiểm thử chức năng được thực hiện trên môi trường triển khai thực tế với backend, Redis và MinIO đang hoạt động, sử dụng dữ liệu thử nghiệm độc lập với các cuộc thi chính thức. Cách tiếp cận này giúp kết quả phản ánh sát hành vi thực tế của hệ thống thay vì chỉ kiểm thử trong môi trường giả lập.

4.5.2 Kiểm thử chức năng

Nhóm kiểm thử chức năng bao phủ toàn bộ các luồng nghiệp vụ chính của hệ thống, bao gồm quản lý tài khoản, đăng ký cuộc thi, nộp bài, chấm bài, cập nhật bảng xếp hạng, quản lý máy chấm tình nguyện và các cơ chế phục hồi lỗi. Các trường hợp kiểm thử được xây dựng dựa trên các use case đã đặc tả ở Chương 3.

Kết quả kiểm thử được trình bày trong Bảng 4.1.

Bảng 4.1: Kết quả kiểm thử chức năng

Mã TC	Chức năng kiểm thử	Kết quả mong đợi	Kết quả thực tế	Đánh giá
TC-01	Đăng ký tài khoản	Tạo tài khoản thành công	Đúng như mong đợi	Đạt
TC-02	Đăng nhập hợp lệ	Cấp JWT và truy cập hệ thống	Đúng như mong đợi	Đạt
TC-03	Đăng nhập sai thông tin	Từ chối xác thực, trả lỗi phù hợp	Đúng như mong đợi	Đạt
TC-04	Đăng ký tham gia cuộc thi	Tạo contest entry thành công	Đúng như mong đợi	Đạt
TC-05	Khởi tạo nộp bài	Sinh presigned URL và tạo submission	Đúng như mong đợi	Đạt
TC-06	Hoàn tất upload	Submission chuyển sang trạng thái queued	Đúng như mong đợi	Đạt
TC-07	Nộp bài khi phase đã đóng	Từ chối yêu cầu nộp bài	Đúng như mong đợi	Đạt
TC-08	Vượt giới hạn số lần nộp	Từ chối yêu cầu nộp bài	Đúng như mong đợi	Đạt
TC-09	Object key không hợp lệ	Từ chối xác nhận upload	Đúng như mong đợi	Đạt
TC-10	Xem bảng xếp hạng	Hiển thị đúng thứ hạng và điểm số	Đúng như mong đợi	Đạt
TC-11	Cài đặt volunteer agent	Agent cài đặt và khởi động thành công	Đúng như mong đợi	Đạt
TC-12	Setup wizard	Thu thập thông tin và đăng ký worker	Đúng như mong đợi	Đạt
TC-13	Đăng ký worker	Worker xuất hiện trong hệ thống	Đúng như mong đợi	Đạt

		với trạng thái chờ duyệt		
TC-14	Phê duyệt worker	Worker được phép nhận job	Đúng như mong đợi	Đạt
TC-15	Nhận job non-final	Worker claim và xử lý job thành công	Đúng như mong đợi	Đạt
TC-16	Worker không đủ năng lực	Job không được phân phối	Đúng như mong đợi	Đạt
TC-17	Chấm bài thành công	Điểm số được ghi nhận và leaderboard cập nhật	Đúng như mong đợi	Đạt
TC-18	Judge script lỗi	Submission chuyển sang trạng thái failed	Đúng như mong đợi	Đạt
TC-19	Worker mất kết nối	Submission được phục hồi và đưa lại hàng đợi	Đúng như mong đợi	Đạt
TC-20	Tạo cuộc thi	Cuộc thi được tạo thành công	Đúng như mong đợi	Đạt
TC-21	Artifact cache (lần đầu)	File được tải và lưu cache	Đúng như mong đợi	Đạt
TC-22	Artifact cache (lần sau)	Tái sử dụng file từ cache	Đúng như mong đợi	Đạt
TC-23	Kiểm tra path traversal	Từ chối thực thi ngoài workspace	Đúng như mong đợi	Đạt
TC-24	Cập nhật leaderboard thời gian thực	Giao diện tự động đồng bộ kết quả mới	Đúng như mong đợi	Đạt

Kết quả cho thấy toàn bộ 24 trường hợp kiểm thử đều đạt yêu cầu. Các chức năng cốt lõi như đăng ký tài khoản, tham gia cuộc thi, nộp bài, nhận kết quả chấm và cập nhật leaderboard hoạt động đúng theo thiết kế. Các trường hợp kiểm thử biên như phase đã

đóng, vượt giới hạn số lần nộp hoặc object key không hợp lệ đều được hệ thống phát hiện và từ chối đúng cách.

Bên cạnh đó, các cơ chế đảm bảo độ tin cậy của hệ thống cũng được xác nhận thông qua các tình huống lỗi có chủ đích. Khi worker ngừng hoạt động giữa quá trình chấm bài, Timeout Watcher có thể phát hiện claim hết hạn và đưa submission trở lại hàng đợi để tiếp tục xử lý. Các kiểm thử bảo mật cơ bản như ngăn chặn path traversal trong trusted native mode cũng cho kết quả đúng như mong đợi.

4.5.3 Kiểm thử tích hợp và hiệu năng

Bên cạnh kiểm thử chức năng, hệ thống còn được đánh giá ở mức tích hợp và hiệu năng nhằm đo lường khả năng xử lý submission trong điều kiện tải thực tế. Các thí nghiệm được thực hiện với số lượng submission tăng dần và nhiều worker hoạt động đồng thời, cho phép đánh giá toàn bộ pipeline từ lúc thí sinh nộp bài đến khi leaderboard được cập nhật.

Kết quả thực nghiệm, số liệu đo đạc và phân tích chi tiết được trình bày trong Chương 5.

4.6. Triển khai Production

4.6.1 Hạ tầng server và dịch vụ

BKDNAIOJ được triển khai trên hạ tầng kết hợp giữa dịch vụ tự quản lý và dịch vụ managed. PostgreSQL sử dụng Supabase, trong khi API server, Redis, MinIO và Caddy được triển khai trên một DigitalOcean Droplet thông qua Docker Compose.

Supabase cung cấp PostgreSQL managed cùng connection pooler tích hợp, giúp đơn giản hóa việc vận hành cơ sở dữ liệu và hỗ trợ kết nối TLS mặc định. Các migration được quản lý bằng goose và thực thi trực tiếp thông qua môi trường triển khai.

API server được đóng gói bằng Docker và triển khai cùng Redis và MinIO trên máy chủ Linux. Redis được sử dụng cho Redis Streams và các chức năng hỗ trợ khác của hệ thống, trong khi MinIO đảm nhiệm vai trò object storage cho submission, dataset và judge assets. Các thông tin nhạy cảm như JWT secret, SMTP credentials và thông tin truy cập MinIO được cấu hình thông qua biến môi trường trên máy chủ.

Bảng 4.2 trình bày các thành phần chính trong môi trường production.

Bảng 4.2: Cấu hình hạ tầng production

Thành phần	Công nghệ triển khai	Mô tả
------------	----------------------	-------

API Server	Go 1.23 + Echo	Chạy trong Docker container
Database	Supabase PostgreSQL	Dịch vụ PostgreSQL managed
Message Queue	Redis 7	Redis Streams cho phân phối job
Object Storage	MinIO	Lưu submission và artifact
Reverse Proxy	Caddy	Reverse proxy và TLS
Frontend	React 19 + Vite	Static files phục vụ bởi Caddy

Caddy được sử dụng làm reverse proxy và quản lý chứng chỉ TLS tự động thông qua Let's Encrypt. Frontend được build thành static files và được phục vụ trực tiếp bởi Caddy, trong khi các yêu cầu API và object storage được chuyển tiếp tới backend và MinIO tương ứng. API server và MinIO chỉ lắng nghe trên mạng nội bộ của máy chủ, toàn bộ truy cập từ Internet đều đi qua Caddy.

4.6.2 Triển khai Máy chấm tình nguyện

Máy chấm tình nguyện được phân phối dưới dạng package Python trên PyPI. Người dùng chỉ cần cài đặt package, chạy setup wizard để đăng ký worker và khởi động agent để tham gia hệ thống chấm bài.

```
pip install olpai-volunteer-agent
olpai setup
olpai start
```

Toàn bộ giao tiếp giữa worker và hệ thống được thực hiện thông qua các kết nối HTTPS từ worker tới API server. Thiết kế này không yêu cầu mở cổng mạng hoặc cấu hình NAT, giúp worker có thể hoạt động trên các mạng gia đình hoặc mạng trường học thông thường.

Agent hỗ trợ cài đặt dưới dạng dịch vụ hệ thống để tự động khởi động lại sau khi máy tính được bật lại.

```
olpai install-service
```

Trên Linux, agent được cài đặt dưới dạng systemd service; trên macOS, agent sử dụng LaunchAgent. Nhờ đó, worker có thể hoạt động liên tục mà không cần người dùng khởi động thủ công sau mỗi lần reboot.

Việc phân phối qua PyPI cũng giúp đơn giản hóa quá trình cập nhật. Khi có phiên bản mới, volunteer chỉ cần nâng cấp package và khởi động lại dịch vụ để sử dụng phiên bản mới nhất của age

CHƯƠNG 5: THỰC NGHIỆM VÀ ĐÁNH GIÁ

Các chương trước đã trình bày quá trình phân tích, thiết kế và triển khai hệ thống BKDNAIOJ, bao gồm kiến trúc backend, cơ chế quản lý submission theo phase, hệ thống phân phối công việc dựa trên máy chấm tình nguyện và mô hình cập nhật leaderboard. Tuy nhiên, hiệu quả của một hệ thống không chỉ được đánh giá thông qua thiết kế mà còn cần được kiểm chứng bằng các số liệu thực nghiệm trong điều kiện vận hành thực tế.

Mục tiêu của chương này là đánh giá khả năng hoạt động của BKDNAIOJ trên môi trường triển khai production thông qua các thí nghiệm có kiểm soát. Các đánh giá tập trung vào ba khía cạnh chính: tính đúng đắn của luồng xử lý đầu cuối, hiệu quả khai thác tài nguyên từ worker pool không đồng nhất và khả năng mở rộng khi tải hệ thống tăng lên.

Toàn bộ số liệu được thu thập trên hệ thống production đang vận hành. Để tránh ảnh hưởng đến dữ liệu của các cuộc thi thực tế, các thí nghiệm được thực hiện trên một bản sao (clone) của contest chính thức nhưng vẫn sử dụng chung hạ tầng backend, cơ sở dữ liệu PostgreSQL, Redis Streams, MinIO và tập máy chấm tình nguyện hiện có. Nhờ đó, các kết quả đo phản ánh sát hành vi thực tế của hệ thống thay vì các điều kiện lý tưởng trong môi trường mô phỏng.

5.1. Mục tiêu thực nghiệm

Các thực nghiệm trong chương này được xây dựng nhằm đánh giá hệ thống BKDNAIOJ trên ba khía cạnh chính: tính đúng đắn của quy trình xử lý submission, hiệu quả khai thác tài nguyên của worker pool và khả năng đáp ứng khi quy mô tải tăng lên.

Thứ nhất, đánh giá tính đúng đắn của quy trình xử lý submission. Nội dung đánh giá tập trung vào việc kiểm chứng toàn bộ vòng đời của một submission trong mô hình contest nhiều phase, từ thời điểm người dùng nộp bài, hệ thống tiếp nhận và đưa vào hàng đợi, worker nhận và thực thi job, cho đến khi kết quả được ghi nhận và bảng xếp hạng được cập nhật. Kết quả được đánh giá thông qua tỷ lệ submission hoàn thành thành công, tỷ lệ lỗi và khả năng xử lý các trường hợp bất thường như worker mất kết nối hoặc job bị thu hồi.

Thứ hai, đánh giá hiệu quả của cơ chế phân phối công việc trên worker pool không đồng nhất. Hệ thống sử dụng cơ chế lập lịch FIFO kết hợp kiểm tra khả năng tương thích và giới hạn slot của từng worker. Thực nghiệm được thiết kế để quan sát cách cơ chế này phân phối công việc khi pool bao gồm nhiều worker có cấu hình phần

cứng và năng lực thực thi khác nhau. Các chỉ số được phân tích bao gồm thời gian chờ trong hàng đợi, thời gian thực thi trên worker, thông lượng xử lý submission và mức độ phân bố tải giữa các worker.

Thứ ba, đánh giá khả năng mở rộng của hệ thống. Nội dung đánh giá tập trung vào việc xác định sức chứa thực tế của hệ thống với cấu hình worker pool hiện tại, đồng thời phân tích ảnh hưởng của việc gia tăng thời gian suy luận hoặc số lượng submission đồng thời. Việc đánh giá được thực hiện dựa trên số liệu đo trực tiếp từ các kịch bản thực nghiệm và phân tích ngoại suy từ dữ liệu thu được nhằm ước lượng khả năng phục vụ của hệ thống trong các cuộc thi có quy mô lớn hơn.

5.2. Thiết lập thực nghiệm

5.2.1 Contest thực nghiệm

Các thực nghiệm được thực hiện trên một contest riêng được tạo bằng cách sao chép (clone) từ contest đang vận hành trên hệ thống. Contest thực nghiệm giữ nguyên toàn bộ cấu hình của contest gốc, bao gồm hai bài toán sử dụng tập dữ liệu MNIST: bài toán phân loại chữ số viết tay và bài toán tấn công đối kháng đối với mô hình phân loại. Contest được tổ chức theo mô hình bốn phase cố định của BKDNAIOJ gồm Public Test, Private Test, Final Public và Final Private.

Hai phase Public Test và Private Test sử dụng hình thức nộp kết quả trực tiếp (*output-only*), trong đó người tham gia nộp file dự đoán đã được sinh sẵn bên ngoài hệ thống. Ngược lại, hai phase Final Public và Final Private sử dụng hình thức chấm bài dựa trên suy luận (*inference-based evaluation*). Trong các phase này, worker phải tải submission của thí sinh, thực thi chương trình suy luận để sinh kết quả dự đoán, sau đó chạy judge script để tính điểm.

Việc sử dụng contest clone thay vì thực hiện trực tiếp trên contest production giúp đảm bảo hai yêu cầu. Thứ nhất, toàn bộ submission vẫn đi qua đầy đủ pipeline xử lý của hệ thống, bao gồm API server, Redis Streams, máy chấm tình nguyện, cơ chế lease và LeaderboardBridge, do đó số liệu thu được phản ánh chính xác hành vi vận hành thực tế. Thứ hai, mọi dữ liệu thực nghiệm được tách biệt hoàn toàn khỏi dữ liệu của người dùng thật, tránh ảnh hưởng đến bảng xếp hạng, lịch sử submission và kết quả chính thức của hệ thống.

5.2.2 Cấu hình worker pool

Worker pool trong thực nghiệm gồm năm worker tình nguyện đang hoạt động tại thời điểm đo. Điểm đặc trưng của pool này là tính không đồng nhất rõ rệt về phần cứng và điều kiện vận hành: từ môi trường Google Colab với một nhân CPU ảo và GPU Tesla

T4 dùng chung tài nguyên với nhiều người dùng khác, đến máy chủ vật lý 32 nhân với RTX 4060, máy tính cá nhân macOS và VPS có GPU riêng. Sự đa dạng này không phải điều kiện lý tưởng để benchmark, nhưng lại phản ánh đúng thực tế vận hành của một hệ thống máy chấm tình nguyện: người quản trị không kiểm soát được phần cứng của từng tình nguyện viên. Bảng 5.1 trình bày cấu hình phần cứng chính của từng worker.

Bảng 5.1: Cấu hình phần cứng của worker pool

Worker	Hệ điều hành	CPU cores	Threads	RAM (GB)	Disk trống (GB)	GPU	VRAM (GB)
colab1	Linux	1	2	12.7	64.6	Tesla T4	15.0
colab2	Linux	1	2	12.7	86.4	-	-
vps1	Linux	32	64	125.7	10.1	RTX 4060	8.0
MacPro	macOS	8	16	32.0	315.3	-	-
vps2	Linux	2	4	31.3	10.8	RTX 3060	12.0

Ngoài cấu hình phần cứng, mỗi worker còn khai báo năng lực thực thi thông qua số lượng slot cho từng loại công việc. Các slot được tách riêng giữa job output-only và job final inference nhằm cho phép scheduler phân phối công việc phù hợp với khả năng của từng worker.

Bảng 5.2: Cấu hình năng lực thực thi của worker pool

Worker	Tổng số job tối đa	Slot nộp kết quả	Slot suy luận	Suy luận độc quyền	Chạy final trực tiếp
colab1	1	1	1	Có	Có
colab2	1	1	0	Có	Không
vps1	16	16	1	Có	Có
MacPro	8	8	0	Có	Không
vps2	2	2	2	Có	Có

Trong cấu hình thực nghiệm, các worker có GPU được vận hành ở chế độ **trusted native** để thực thi các phase final. Đây là lựa chọn phù hợp với các môi trường như Google Colab hoặc VPS GPU, nơi việc sử dụng Docker sandbox với GPU gặp nhiều hạn chế về quyền hệ thống. Ngược lại, worker trên macOS hỗ trợ Docker nhưng không có GPU nên chỉ tham gia xử lý các submission output-only.

Để cung cấp thêm thông tin về năng lực thực thi của từng worker, volunteer agent thực hiện một tập micro-benchmark trong quá trình đăng ký. Các phép đo bao gồm hiệu năng CPU, hiệu năng GPU (nếu có), băng thông bộ nhớ, tốc độ đọc/ghi đĩa và tốc độ giải nén dữ liệu. Kết quả benchmark được lưu trong hồ sơ worker và được backend sử dụng như một nguồn thông tin tham khảo trong quá trình kiểm tra tính tương thích giữa worker và job.

Bảng 5.3: Snapshot benchmark của worker pool

Worker	CPU (triệu phép/s)	Thời gian kiểm tra CPU (ms)	GPU đo được (TFLOP/s)	Giải nén (MB/s)	Đọc disk (MB/s)	Ghi disk (MB/s)	Bộ nhớ (MB/s)
colab1	12.8	39.1	4.2	489.3	1727.2	335.6	2694.2
colab2	16.9	29.6	-	556.9	2214.2	289.0	5531.9
vps1	5.7	88.5	8.6	393.6	754.1	21.4	1384.5
MacPr	9.3	53.8	-	468.0	919.4	1317.5	1877.3
vps2	6.6	75.9	7.5	443.4	1084.0	661.7	4079.3

Kết quả benchmark cho thấy worker pool có mức độ không đồng nhất đáng kể. Worker vps1 sở hữu GPU RTX 4060 với hiệu năng đo được cao nhất trong nhóm GPU (8.6 TFLOP/s), trong khi vps2 đạt 7.5 TFLOP/s với RTX 3060. Hai worker colab1 và colab2 có băng thông bộ nhớ và tốc độ đọc đĩa cao, trong khi MacBookPro có tốc độ ghi đĩa nổi bật nhờ SSD NVMe, thuận lợi cho các tác vụ tải và chuẩn bị artifact.

Các số liệu benchmark này được sử dụng nhằm mô tả mức độ không đồng nhất của worker pool và làm cơ sở phân tích kết quả thực nghiệm ở các phần sau. Trong phiên bản BKDNAIOJ được đánh giá trong đề án này, scheduler không sử dụng trực tiếp các chỉ số benchmark như TFLOPS, băng thông bộ nhớ hay tốc độ đĩa để lựa chọn worker. Việc phân phối job được thực hiện theo nguyên tắc FIFO kết hợp kiểm tra tính tương thích của worker đối với từng job, bao gồm loại phase, khả năng thực thi final phase, số slot còn trống và các ràng buộc tài nguyên cơ bản. Do đó, benchmark phản ánh đặc điểm của môi trường thực nghiệm hơn là một tham số tham gia trực tiếp vào thuật toán lập lịch.

5.2.3 **Thiết kế workload thực nghiệm**

Để đánh giá hệ thống dưới nhiều điều kiện vận hành khác nhau, thực nghiệm sử dụng ba nhóm workload bổ sung cho nhau, tương ứng với các mục tiêu đánh giá về tính đúng đắn chức năng, hiệu năng xử lý và khả năng phục vụ đồng thời.

Workload MNIST thực tế là workload chính của nghiên cứu. Nhóm workload này sử dụng các submission hợp lệ cho hai bài toán MNIST được cấu hình trong contest thực nghiệm và được chạy trên đầy đủ bốn phase của hệ thống. Mỗi submission trải qua toàn bộ vòng đời xử lý giống như trong một cuộc thi thực tế, từ upload file, tạo job, phân phối đến worker, thực thi chấm bài và cập nhật leaderboard. Workload này được sử dụng để đánh giá hiệu năng tổng thể của pipeline xử lý submission với các quy mô batch 50, 100, 300 và 500 submission.

Workload suy luận tổng hợp được xây dựng nhằm đánh giá riêng khả năng xử lý các phase final có yêu cầu thực thi chương trình suy luận. Thay vì sử dụng mô hình học máy phức tạp khó kiểm soát thời gian chạy, workload này thực hiện các phép nhân ma trận bằng PyTorch với kích thước và số vòng lặp được cấu hình trước để tạo ra các mức tải tính toán khác nhau trên CPU hoặc GPU. Cách tiếp cận này cho phép kiểm soát tương đối chính xác thời gian thực thi trong khi vẫn giữ nguyên pipeline inference → judging của hệ thống. Một số submission trong nhóm workload này được thiết kế cố ý thất bại nhằm kiểm tra cơ chế xử lý lỗi, thu hồi tài nguyên và khả năng đảm bảo các job lỗi không ảnh hưởng đến những job khác trong hàng đợi.

Workload đọc API được sử dụng để đánh giá khả năng phục vụ các yêu cầu đọc đồng thời từ phía người dùng trong thời gian diễn ra cuộc thi. Workload được tạo bằng công cụ K6 và gửi các request đã xác thực đến những endpoint được truy cập thường xuyên nhất, bao gồm danh sách cuộc thi, danh sách bài toán và lịch sử submission. Nhóm workload này được thực hiện song song với các batch chấm bài nhằm đánh giá mức độ ảnh hưởng của tải xử lý submission lên trải nghiệm sử dụng giao diện của contestant.

5.2.4 **Phương pháp thu thập và tính toán số liệu**

Để đo đạc chi tiết từng thành phần độ trễ trong vòng đời của submission, hệ thống được bổ sung cơ chế ghi nhận các mốc sự kiện quan trọng trong quá trình xử lý. Các mốc được theo dõi bao gồm thời điểm submission được đưa vào hàng đợi Redis Stream, thời điểm worker nhận job, thời điểm bắt đầu thực thi, thời điểm worker gửi kết quả về backend, thời điểm backend ghi nhận kết quả và thời điểm leaderboard được cập nhật. Đối với các submission gặp lỗi và phải đưa lại vào hàng đợi, các sự kiện tương ứng cũng được ghi nhận.

Các sự kiện được lưu kèm định danh submission, định danh worker (nếu có) và thời điểm xảy ra với độ phân giải millisecond. Bên cạnh đó, hệ thống cũng ghi nhận thông tin về quá trình phân phối công việc của scheduler, bao gồm số lượng job được xem xét, số lượng job tương thích và kết quả lựa chọn cuối cùng. Cách tiếp cận này cho phép tái dựng toàn bộ vòng đời của từng submission và thực hiện phân tích ở cả mức submission đơn lẻ lẫn mức batch thực nghiệm.

Từ các mốc thời gian thu thập được, bốn chỉ số được sử dụng trong các phần đánh giá tiếp theo:

- Thời gian chờ hàng đợi (Queue Wait Time): khoảng thời gian từ khi submission được đưa vào hàng đợi đến khi được một worker nhận xử lý.
- Thời gian thực thi (Execution Time): khoảng thời gian từ khi worker nhận job đến khi backend nhận được kết quả, bao gồm các bước tải artifact, thực thi inference (nếu có), chạy judge script và gửi kết quả.
- Độ trễ đầu cuối (End-to-End Latency): khoảng thời gian từ khi submission được đưa vào hàng đợi đến khi kết quả được ghi nhận thành công bởi hệ thống.
- Độ trễ cập nhật leaderboard (Leaderboard Update Latency): khoảng thời gian từ khi backend ghi nhận kết quả đến khi bảng xếp hạng tương ứng phản ánh thay đổi mới nhất.

Các chỉ số này được sử dụng làm cơ sở để đánh giá hiệu năng xử lý submission, hiệu quả khai thác worker pool và khả năng đáp ứng của hệ thống trong các phần thực nghiệm tiếp theo.

5.3. Kiểm thử chức năng đầu cuối

Trước khi tiến hành các batch thực nghiệm quy mô lớn, hệ thống được kiểm thử bằng một pilot workload gồm 32 submission. Workload này được xây dựng nhằm bao phủ toàn bộ các trường hợp chính của contest thực nghiệm, bao gồm cả hai bài toán, cả bốn phase và cả hai hình thức đánh giá là output-only và inference-based. Mục tiêu của giai đoạn pilot là xác nhận tính đúng đắn của toàn bộ pipeline xử lý submission trong môi trường production trước khi thực hiện các phép đo hiệu năng.

Kết quả cho thấy toàn bộ 32 submission đều được xử lý thành công. Các submission đi qua đầy đủ vòng đời của hệ thống, từ tiếp nhận yêu cầu nộp bài, đưa vào hàng đợi, phân phối tới worker, thực thi chấm bài, ghi nhận kết quả cho đến cập nhật bảng xếp hạng. Không ghi nhận trường hợp submission bị kẹt ở trạng thái queued hoặc running sau khi batch kết thúc. Kết quả tổng hợp được trình bày trong Bảng 5.4.

Bảng 5.4: Kết quả pilot workload

Chỉ số	Giá trị
Số submission	32
Số submission hoàn thành thành công	32
Tỷ lệ thành công	100%
Số submission kẹt sau batch	0

Kết quả pilot xác nhận hệ thống vận hành đúng đắn đối với cả hai hình thức đánh giá và toàn bộ bốn phase của mô hình contest. Do không ghi nhận submission thất lạc, submission bị kẹt hoặc lỗi trong quá trình cập nhật kết quả, hệ thống được xem là đáp ứng yêu cầu về tính đúng đắn chức năng và sẵn sàng cho các thực nghiệm hiệu năng ở các phân tiếp theo.

5.4. Hiệu năng xử lý submission

5.4.1 Kết quả batch tăng tải

Sau khi hoàn thành giai đoạn pilot, hệ thống được đánh giá dưới bốn mức tải tăng dần gồm 50, 100, 300 và 500 submission. Các submission được phân bố trên cả bốn phase của contest thực nghiệm, bao gồm cả các phase output-only và các phase yêu cầu suy luận. Mục tiêu của thực nghiệm là đánh giá khả năng duy trì tính ổn định của pipeline xử lý submission khi khối lượng công việc tăng lên.

Bảng 5.5: So sánh các batch workload MNIST

Batch	Tỷ lệ thành công	Median queue wait (s)	P95 queue wait (s)	Peak concurrent jobs	Average concurrent jobs	Throughput thành công (submission/phút)
50	1.0	4.5	29.9	11	4.3	51.7
100	1.0	10.0	64.4	7	3.7	43.7
300	1.0	44.5	121.3	7	3.5	35.1
500	1.0	52.4	105.2	7	3.9	47.8

Kết quả cho thấy hệ thống duy trì tỷ lệ hoàn thành 100% ở tất cả các mức tải. Không ghi nhận submission bị mất, bị kẹt trong hàng đợi hoặc duy trì trạng thái running sau khi batch kết thúc. Điều này xác nhận cơ chế phân phối công việc, thực thi trên worker và phục hồi lỗi hoạt động ổn định ngay cả khi số lượng submission tăng lên gấp mười lần so với workload pilot.

Đối với độ trễ hàng đợi, giá trị median tăng từ 4.5 giây ở batch 50 lên 52.4 giây ở batch 500. Xu hướng này phù hợp với đặc điểm của hệ thống hàng đợi: khi tốc độ tạo submission vượt quá tốc độ xử lý tức thời của worker pool, các submission mới phải chờ lâu hơn trước khi được nhận xử lý. Giá trị P95 cũng tăng từ 29.9 giây lên trên 100 giây ở các batch lớn, cho thấy một bộ phận submission phải chờ đáng kể trong các giai đoạn tải cao.

Thông lượng xử lý dao động trong khoảng 35-52 submission mỗi phút. Mặc dù batch 500 có số lượng submission lớn nhất, thông lượng đạt được vẫn ở mức 47.8 submission/phút, cao hơn batch 300. Điều này cho thấy hiệu năng thực tế của hệ thống không chỉ phụ thuộc vào kích thước batch mà còn chịu ảnh hưởng bởi trạng thái của worker pool tại thời điểm thực nghiệm, bao gồm tốc độ poll, tải nền của từng máy và điều kiện mạng của các máy chấm tình nguyện.

Số lượng job chạy đồng thời đạt cực đại 11 ở batch 50 và dao động quanh mức 7 ở các batch còn lại. Trong khi đó, mức độ đồng thời trung bình tương đối ổn định, nằm trong khoảng 3.5-4.3 job. Kết quả này cho thấy worker pool được khai thác liên tục trong suốt quá trình thực nghiệm, đồng thời phản ánh giới hạn thực tế của tập worker hiện có.

5.4.2 Phân tích độ trễ vòng đời submission

Để phân tích chi tiết các thành phần độ trễ trong vòng đời submission, batch 500 được lựa chọn làm đối tượng nghiên cứu do có quy mô lớn nhất và bao gồm số lượng cân bằng giữa submission final và non-final. Kết quả được trình bày trong Bảng 5.6.

Bảng 5.6: Độ trễ vòng đời submission trong batch 500

Nhóm	Chỉ số	Count	Median (s)	P95 (s)	Max (s)
Tất cả	Queue wait	500	52.4	105.2	110.2
Tất cả	Thời gian thực thi worker	500	5.1	9.6	35.2
Tất cả	Leaderboard freshness	500	1.4	2.8	4.4
Tất cả	End-to-end	500	57.0	115.1	124.6
Non-final	Queue wait	250	44.1	53.3	58.3
Non-final	Thời gian thực thi worker	250	2.2	3.7	5.8
Non-final	End-to-end	250	47.2	58.0	62.7
Final	Queue wait	250	97.7	106.7	110.2

Final	Thời gian thực thi worker	250	7.9	10.6	35.2
Final	End-to-end	250	106.8	117.1	124.6

Kết quả cho thấy thời gian chờ hàng đợi là thành phần chiếm tỷ trọng lớn nhất trong độ trễ đầu cuối của hệ thống. Với toàn bộ 500 submission, median queue wait đạt 52.4 giây, trong khi median thời gian thực thi trên worker chỉ đạt 5.1 giây. Điều này cho thấy giới hạn hiện tại của hệ thống không nằm ở tốc độ chấm bài của từng worker, mà nằm ở số lượng slot xử lý đồng thời có sẵn trong worker pool.

Sự khác biệt giữa submission final và non-final thể hiện rõ ở cả thời gian chờ hàng đợi lẫn thời gian thực thi. Đối với non-final submission, median queue wait là 44.1 giây và median thời gian thực thi là 2.2 giây. Trong khi đó, các submission final có median queue wait lên tới 97.7 giây và median thời gian thực thi 7.9 giây. Như vậy, submission final không chỉ cần nhiều thời gian xử lý hơn mà còn phải cạnh tranh trên một tập tài nguyên nhỏ hơn.

Nguyên nhân chủ yếu xuất phát từ cấu hình worker pool trong thực nghiệm. Trong năm worker tham gia, chỉ có ba worker (colab1, vps1 và vps2) được phép thực thi các phase final, trong khi toàn bộ năm worker đều có thể xử lý phần lớn các submission non-final. Do đó, khi số lượng submission final tăng lên, hàng đợi của nhóm worker hỗ trợ inference trở thành điểm nghẽn của toàn hệ thống. Kết quả này phản ánh đúng đặc điểm của cơ chế phân phối Capability-aware FIFO: submission chỉ được giao cho những worker đáp ứng yêu cầu tương thích của job.

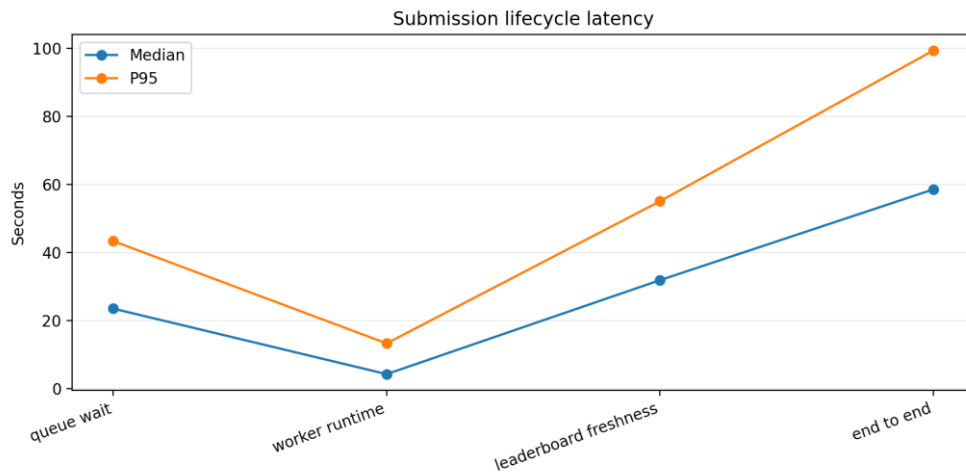
Đối với thời gian thực thi trên worker, submission final có median cao hơn khoảng 3.6 lần so với non-final (7.9 giây so với 2.2 giây). Chênh lệch này đến từ các bước bổ sung trong pipeline final, bao gồm tải mô hình, khởi tạo môi trường suy luận, thực thi inference và sinh file kết quả trước khi judge script được gọi. Ngược lại, non-final submission chỉ cần thực hiện bước đánh giá trực tiếp trên file kết quả đã được thí sinh chuẩn bị sẵn.

Một kết quả khác là độ trễ cập nhật bảng xếp hạng duy trì ở mức thấp trong toàn bộ batch. Giá trị median leaderboard freshness đạt 1.4 giây, P95 đạt 2.8 giây và giá trị lớn nhất chỉ 4.4 giây. So với độ trễ đầu cuối của submission, phần chi phí dành cho cập nhật leaderboard là rất nhỏ. Điều này cho thấy cơ chế xử lý bất đồng bộ thông qua Redis

Stream và LeaderboardBridge không tạo ra điểm nghẽn đáng kể trong pipeline xử lý kết quả.

Nhìn chung, kết quả của batch 500 cho thấy thành phần ảnh hưởng lớn nhất đến trải nghiệm của người dùng là thời gian chờ hàng đợi, đặc biệt đối với các phase final yêu cầu suy luận. Trong khi đó, chi phí cập nhật leaderboard gần như không đáng kể và thời gian thực thi của từng submission vẫn duy trì ở mức thấp. Điều này cho thấy việc mở rộng năng lực xử lý của hệ thống trong tương lai nên tập trung vào việc tăng số lượng worker hỗ trợ inference hoặc tăng số slot final khả dụng.

Hình 5.1 trực quan hóa sự phân bố của các thành phần độ trễ, cho phép so sánh trực tiếp giữa final và non-final submission.



Hình 5.1: Độ trễ vòng đời submission trong batch 500

5.5. Đánh giá workload tính toán tổng hợp

Nhóm workload thứ hai được xây dựng nhằm bổ sung cho workload MNIST bằng các submission có tải tính toán được kiểm soát. Thay vì sử dụng các cơ chế giả lập thời gian thực thi, mỗi submission thực hiện phép nhân ma trận bằng PyTorch với số vòng lặp và kích thước ma trận được cấu hình trước. Sau khi hoàn thành tính toán, submission vẫn sinh ra output hợp lệ theo định dạng của bài toán MNIST và được chấm điểm theo quy trình thông thường. Cách tiếp cận này đảm bảo thời gian đo bao gồm toàn bộ chi phí thực tế của pipeline, từ tải artifact, khởi tạo môi trường thực thi, thực hiện suy luận, ghi kết quả cho đến gửi phản hồi về backend.

Bên cạnh các submission hợp lệ, một phần workload được thiết kế để kích hoạt các nhánh xử lý lỗi bằng cách sinh output không đúng định dạng hoặc phát sinh ngoại lệ trong quá trình suy luận. Mục tiêu là đánh giá khả năng xử lý lỗi của hệ thống và xác

nhận rằng các submission thất bại vẫn kết thúc đúng trạng thái thay vì bị kẹt trong pipeline xử lý.

Bảng 5.7: Kết quả workload tính toán tổng hợp

Workload	Số job	Thành công	Thất bại có chủ đích	Thời gian batch (s)	Throughput terminal (job/phút)	P95 queue wait (s)
Compute pilot	20	16	4	32.9	36.5	15.2
Compute mixed	100	76	24	155.3	38.6	34.8
Compute final-heavy	100	76	24	160.3	37.4	37.4

Kết quả cho thấy toàn bộ các job thất bại đều thuộc nhóm được thiết kế có chủ đích để kích hoạt nhánh lỗi. Trong cả hai batch 100 job, 24 submission lỗi đều được worker phát hiện, ghi nhận trạng thái thất bại và gửi kết quả về backend thành công. Không ghi nhận trường hợp submission bị duy trì ở trạng thái running sau khi batch kết thúc.

Thông lượng xử lý của workload tính toán tổng hợp dao động trong khoảng 36-39 job/phút. So với workload MNIST ở Mục 5.4, giá trị này thấp hơn do mỗi submission phải thực hiện thêm khối lượng tính toán đáng kể trước khi sinh kết quả. Hai batch 100 job cho kết quả tương đối ổn định, với throughput lần lượt đạt 38.6 và 37.4 job/phút, trong khi P95 queue wait nằm trong khoảng 34.8-37.4 giây.

Kết quả thực nghiệm cho thấy pipeline xử lý inference và cơ chế xử lý lỗi của hệ thống hoạt động ổn định dưới tải tính toán cao hơn workload MNIST thông thường. Đồng thời, các submission thất bại không ảnh hưởng đến quá trình xử lý của các submission còn lại trong cùng batch và không tạo ra tình trạng tắc nghẽn trong hàng đợi.

5.6. Phân phối job và mức sử dụng worker

Một mục tiêu quan trọng của thực nghiệm là đánh giá khả năng phân phối công việc của scheduler trên một worker pool không đồng nhất. Bảng 5.8 trình bày số lượng job được xử lý bởi từng worker trong batch 500.

Bảng 5.8: Phân phối job theo worker trong batch 500

Worker	Tổng job	Thành công	Job nộp kết quả	Job final	Median thời gian thực thi (s)	P95 thời gian thực thi (s)
vps2	129	129	0	129	8.0	8.9
colab1	74	74	0	74	6.3	8.2
colab2	75	75	75	0	3.2	3.6
vps1	47	47	0	47	9.7	12.2
MacPro	175	175	175	0	1.5	3.7

Kết quả cho thấy sự phân tách rõ ràng giữa hai nhóm worker. Ba worker có khả năng thực thi phase final - vps2, colab1 và vps1 - xử lý toàn bộ các job final và không nhận bất kỳ job nộp kết quả trực tiếp nào. Ngược lại, hai worker CPU - colab2 và MacBookPro - chỉ xử lý các job nộp kết quả trực tiếp. Đây là bằng chứng định lượng cho thấy cơ chế kiểm tra năng lực trong thuật toán ClaimNext hoạt động đúng như thiết kế: worker không công bố slot suy luận sẽ không nhận job final, ngay cả khi vẫn còn slot output khả dụng.

Đáng chú ý, MacBookPro xử lý tổng cộng 175 job, cao nhất trong toàn bộ worker pool, nhờ sở hữu 8 output slot và thời gian thực thi ngắn với median chỉ 1.5 giây. Trong nhóm worker hỗ trợ suy luận, vps2 xử lý nhiều job final nhất với 129 job, mặc dù chỉ có 2 inference slot. Kết quả này cho thấy worker liên tục được cấp phát công việc mới sau khi hoàn thành các job trước đó.

Tổng số job được ghi nhận trong Bảng 5.8 là 500, đúng bằng kích thước của batch thực nghiệm. Điều này xác nhận rằng toàn bộ submission trong batch đều được phân phối và xử lý bởi các worker đã đăng ký, không ghi nhận trường hợp submission bị thất lạc hoặc được xử lý ngoài worker pool.

Số lượng job đồng thời lớn nhất quan sát được trong batch 500 là 7, thấp hơn tổng số slot lý thuyết của toàn bộ worker pool. Khoảng cách này xuất phát từ hai nguyên nhân chính. Thứ nhất, các slot được phân loại theo loại công việc nên không phải mọi slot đều có thể xử lý mọi submission. Ví dụ, 8 output slot của MacBookPro không thể thực thi các job final, trong khi các inference slot trên worker GPU chỉ dành cho các submission yêu cầu suy luận. Thứ hai, workload MNIST có thời gian thực thi tương đối ngắn (median 5.1 giây), khiến các worker nhanh chóng hoàn thành công việc và quay lại vòng lặp poll, thay vì duy trì số lượng lớn job chạy đồng thời trong thời gian dài.

Thuật toán lập lịch được sử dụng trong thực nghiệm là FIFO có kiểm tra năng lực. Khi một worker còn slot trống, backend duyệt hàng đợi theo thứ tự đến và giao job tương thích đầu tiên, thay vì trì hoãn để chờ một worker được đánh giá là tối ưu hơn. Với FIFO, nguyên tắc “worker còn trống và tương thích thì nhận việc ngay” giúp hạn chế thời gian nhàn rỗi của worker và duy trì thông lượng xử lý ổn định.

5.7. Đánh giá tải đọc API

Bên cạnh luồng xử lý submission, hệ thống còn phải phục vụ các yêu cầu đọc dữ liệu từ giao diện người dùng trong suốt thời gian diễn ra cuộc thi. Các thao tác phổ biến bao gồm xem danh sách cuộc thi, danh sách bài toán, lịch sử submission và theo dõi tiến trình thi đấu. Để đánh giá khả năng phục vụ đồng thời của API server, thực nghiệm sử dụng công cụ K6 với bảy mức tải người dùng ảo: 50, 100, 200, 300, 500, 800 và 1000.

Trong K6, một người dùng ảo (virtual user) đại diện cho một phiên người dùng hoạt động liên tục trong suốt thời gian kiểm thử, không phải một request đơn lẻ. Ba mức tải đầu tiên (50-200 người dùng ảo) được sử dụng như các kịch bản load test thông thường, trong khi các mức từ 300 người dùng ảo trở lên được sử dụng như các kịch bản stress test nhằm đánh giá hành vi của hệ thống khi tải tiếp tục gia tăng. Các người dùng ảo được phân bố vòng tròn trên tám tài khoản thực nghiệm đã đăng ký contest clone. Mỗi người dùng thực hiện tuần tự ba request đã xác thực: truy vấn danh sách contest, danh sách bài toán và lịch sử submission, sau đó nghỉ một giây trước khi bắt đầu vòng lặp tiếp theo. Kết quả được trình bày trong Bảng 5.9.

Bảng 5.9: Kết quả kiểm thử tải đọc API

Người dùng ảo	Request rate (req/s)	Error rate	Overall P95 (ms)	Contest P95 (ms)	Tasks P95 (ms)	Submissions P95 (ms)
50	83.4	0%	463.7	284.4	530.7	407.1
100	133.1	0%	1091.1	938.9	1120.7	1162.2
200	140.2	0%	2277.6	1292.5	2493.9	1622.8
300	145.2	0%	3795.7	1394.4	5065.4	2918.6
500	144.5	0%	5978.3	2946.3	6885.4	5079.9
800	160.4	0%	8789.4	3350.8	9056.9	3059.3
1000	149.9	0.03%	12014.3	6282.4	13624.6	4641.9

Kết quả cho thấy hệ thống duy trì tỷ lệ lỗi bằng 0 trong toàn bộ các mức tải từ 50 đến 800 người dùng ảo. Toàn bộ request ở ba endpoint khảo sát đều nhận được phản hồi

hợp lệ từ server. Chỉ tại mức tải 1000 người dùng ảo mới xuất hiện lỗi timeout với tỷ lệ khoảng 0.03%, cho thấy API server vẫn duy trì khả năng phục vụ ổn định ngay cả khi số lượng phiên đồng thời tăng lên rất cao.

Về độ trễ, P95 tăng dần theo mức tải. Ở mức 50 người dùng ảo, P95 tổng thể đạt 463.7 ms. Khi tăng lên 100 và 200 người dùng ảo, P95 lần lượt đạt 1091.1 ms và 2277.6 ms. Từ mức 300 người dùng ảo trở lên, độ trễ tiếp tục gia tăng đáng kể, với P95 tổng thể đạt 3795.7 ms ở mức 300 người dùng, 5978.3 ms ở mức 500 người dùng, 8789.4 ms ở mức 800 người dùng và 12014.3 ms ở mức 1000 người dùng.

Sự khác biệt giữa các endpoint phản ánh mức độ phức tạp của truy vấn. Endpoint danh sách contest duy trì độ trễ thấp nhất ở hầu hết các mức tải do chủ yếu truy xuất dữ liệu cấu hình tương đối ổn định. Trong khi đó, endpoint danh sách bài toán có độ trễ cao nhất ở các mức stress test, với P95 đạt 9056.9 ms tại mức 800 người dùng ảo và 13624.6 ms tại mức 1000 người dùng ảo. Endpoint lịch sử submission cũng ghi nhận xu hướng tăng độ trễ theo tải nhưng ở mức thấp hơn so với endpoint danh sách bài toán trong phần lớn các trường hợp.

Các kết quả trên cho thấy khi tải tăng, hệ thống chủ yếu suy giảm về độ trễ thay vì phát sinh lỗi chức năng. Nói cách khác, API server vẫn duy trì khả năng xử lý request nhưng thời gian phản hồi tăng dần do chi phí truy vấn và tổng hợp dữ liệu lớn hơn dưới áp lực đồng thời cao.

Nhìn chung, hệ thống duy trì tỷ lệ lỗi bằng 0 đến mức tải 800 người dùng ảo và chỉ xuất hiện tỷ lệ timeout rất nhỏ ở mức 1000 người dùng ảo. Nếu lấy ngưỡng P95 khoảng 3 giây làm tiêu chí tham khảo cho các thao tác đọc dữ liệu, kết quả cho thấy cấu hình hiện tại đáp ứng tốt các mức tải đến khoảng 200 người dùng ảo đồng thời. Các hướng tối ưu trong tương lai bao gồm tăng cường cache cho dữ liệu ít thay đổi, giảm số lượng truy vấn phụ thuộc trong mỗi request và tối ưu các endpoint tổng hợp dữ liệu động như danh sách bài toán và lịch sử submission..

5.8. Ước lượng sức chứa bằng phân tích độ nhảy

Workload MNIST được sử dụng trong thực nghiệm có thời gian thực thi của các submission final tương đối ngắn, với median khoảng 8 giây như đã trình bày ở Mục 5.4. Tuy nhiên, đặc điểm này không đại diện cho mọi bài toán AI trong thực tế. Nhiều bài toán có thể yêu cầu thời gian suy luận dài hơn đáng kể, chẳng hạn các mô hình ngôn ngữ lớn, mô hình xử lý ảnh độ phân giải cao hoặc các mô hình học sâu nhiều tầng. Trong

các trường hợp này, thời gian chiếm giữ inference slot có thể kéo dài hàng chục giây hoặc vài phút.

Do không có điều kiện xây dựng nhiều workload thực nghiệm với các mức tải suy luận khác nhau, nghiên cứu sử dụng phương pháp phân tích độ nhạy dựa trên số liệu đo được từ batch 500. Cụ thể, toàn bộ cấu hình worker pool, tỷ lệ submission final và cơ chế lập lịch được giữ nguyên; chỉ thời gian thực thi của các final submission được nhân với các hệ số $1\times$, $5\times$ và $10\times$. Kết quả thu được không đại diện cho phép đo thực nghiệm trực tiếp mà là một kịch bản giả định nhằm đánh giá ảnh hưởng của thời gian inference đến sức chứa của hệ thống.

Bảng 5.10 trình bày kết quả ước lượng trong trường hợp batch gồm 500 submission, trong đó 50% là final submission.

Bảng 5.10: Ước lượng sức chứa với 5 worker, 500 submission, 50% final

Hệ số thời gian final	Throughput ước lượng (submission/phút)	P95 queue wait (s)	P95 queue wait final (s)	Số đội nếu 2 submission/10 phút
$\times 1$	73.6	341.1	361.5	367
$\times 5$	14.7	1705.5	1807.6	73
$\times 10$	7.4	3411.0	3615.3	36

Kết quả cho thấy thời gian suy luận có ảnh hưởng trực tiếp đến thông lượng xử lý của hệ thống. Khi thời gian thực thi final tăng gấp 5 lần, throughput ước lượng giảm từ 73.6 xuống 14.7 submission/phút. Khi tăng lên 10 lần, throughput tiếp tục giảm xuống còn 7.4 submission/phút. Đồng thời, P95 queue wait tăng tương ứng từ vài trăm giây lên hàng nghìn giây.

Xu hướng này phản ánh đặc điểm của kiến trúc hiện tại: số lượng inference slot trong worker pool là hữu hạn, trong khi các final submission phải chiếm giữ inference slot trong toàn bộ thời gian thực thi. Khi thời gian thực thi tăng lên, khả năng giải phóng slot giảm xuống tương ứng, làm giảm tốc độ xử lý chung của hệ thống.

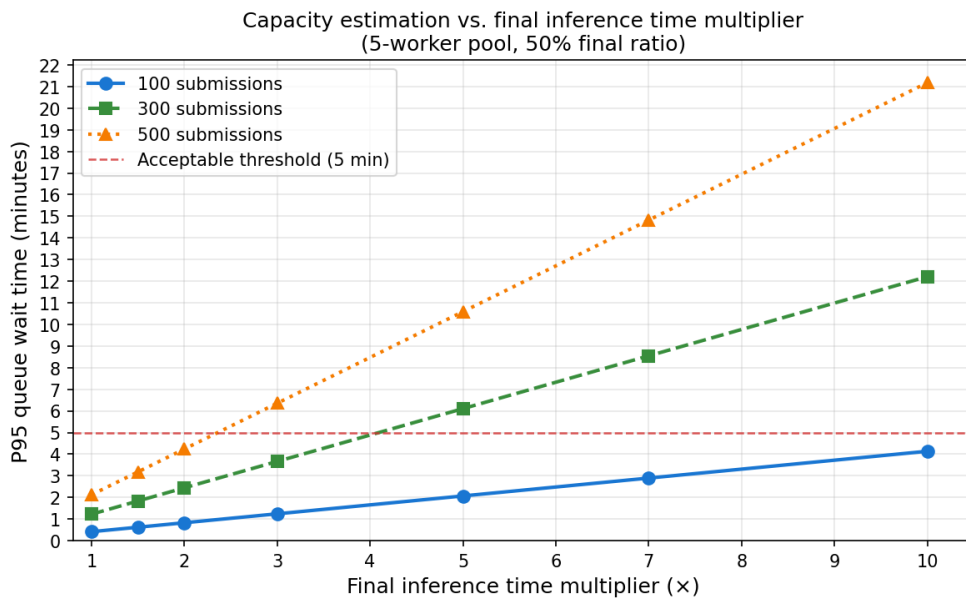
Để đánh giá tác động của quy mô batch, cùng phương pháp trên được áp dụng cho kịch bản 1000 submission với tỷ lệ final giữ nguyên ở mức 50%.

Bảng 5.11: Ước lượng sức chứa với 5 worker, 1000 submission, 50% final

Hệ số thời gian final	Throughput ước lượng (submission/phút)	P95 queue wait (s)	P95 queue wait final (s)	Số đội nếu 2 submission/10 phút
×1	75.2	692.9	731.5	375
×5	15.0	3464.3	3657.4	75
×10	7.5	6928.5	7314.7	37

So sánh hai bảng cho thấy throughput dài hạn gần như không thay đổi khi số lượng submission tăng từ 500 lên 1000 trong cùng một cấu hình worker pool. Ngược lại, thời gian chờ hàng đợi tăng đáng kể khi kích thước batch tăng. Điều này phù hợp với bản chất của hệ thống hàng đợi: năng lực xử lý được quyết định chủ yếu bởi số lượng worker và số inference slot khả dụng, trong khi độ dài hàng đợi phụ thuộc vào lượng công việc đang chờ xử lý.

Hình 5.2 minh họa xu hướng gia tăng của P95 queue wait khi thời gian thực thi final tăng dần.



Hình 5.2: Ước lượng sức chứa theo hệ số thời gian final inference

Từ các kết quả trên có thể thấy rằng, đối với các workload có thời gian suy luận ngắn tương đương MNIST, worker pool hiện tại đáp ứng tốt nhu cầu xử lý submission. Tuy nhiên, khi thời gian inference tăng lên nhiều lần, số lượng inference slot trở thành nút thắt chính của hệ thống. Trong bối cảnh đó, việc bổ sung thêm worker GPU hoặc

tăng số inference slot khả dụng sẽ mang lại hiệu quả cải thiện thông lượng rõ rệt hơn so với việc chỉ bổ sung các worker CPU.

5.9. Thảo luận

5.9.1 Trả lời RQ1 - Tính đúng đắn đầu cuối

Kết quả thực nghiệm cho thấy RQ1 được xác nhận trong phạm vi workload khảo sát. Cả pilot workload (32 submission) và các batch MNIST chính đều đạt tỷ lệ hoàn thành 100%, không ghi nhận submission bị mất hoặc bị kẹt ở trạng thái queued hay running sau khi batch kết thúc.

Đặc biệt, hệ thống xử lý thành công cả hai hình thức nộp bài được hỗ trợ bởi BKDNAIOJ: nộp kết quả trực tiếp (non-final phase) và nộp chương trình suy luận (final phase). Toàn bộ vòng đời submission - từ upload, đưa vào hàng đợi, phân phối đến worker, thực thi, ghi nhận kết quả và cập nhật leaderboard - đều vận hành đúng trên cả bốn phase của contest.

Bên cạnh đó, workload tính toán tổng hợp cho thấy các submission thất bại được xử lý đúng theo thiết kế. Khi inference hoặc judge script phát sinh lỗi, worker ghi nhận trạng thái thất bại và gửi kết quả về backend thay vì để submission duy trì ở trạng thái running. Điều này cho thấy cơ chế xử lý lỗi và phục hồi trạng thái hoạt động đúng trong các tình huống được khảo sát.

5.9.2 Trả lời RQ2 - Hiệu quả khai thác worker pool không đồng nhất

Kết quả thực nghiệm xác nhận rằng cơ chế lập lịch FIFO kết hợp kiểm tra năng lực worker phân phối công việc phù hợp với khả năng thực thi của từng nhóm worker. Trong batch 500, toàn bộ 250 final submission được xử lý bởi các worker hỗ trợ suy luận, trong khi 250 non-final submission được xử lý bởi các worker chỉ hỗ trợ nộp kết quả trực tiếp. Không ghi nhận trường hợp worker không đủ năng lực nhận nhầm loại job không tương thích.

Thông lượng đạt 47.8 submission/phút với P95 queue wait là 105.2 giây ở batch lớn nhất. Chênh lệch độ trễ giữa final submission và non-final submission chủ yếu xuất phát từ chi phí suy luận và số lượng inference slot hạn chế trong worker pool, thay vì từ cơ chế lập lịch.

Một quan sát đáng chú ý là số job đồng thời cao nhất chỉ đạt 7, thấp hơn tổng số slot lý thuyết của toàn bộ worker pool. Điều này phản ánh đặc điểm của hệ thống hiện tại, trong đó slot được phân loại theo loại công việc và không phải mọi slot đều có thể xử lý mọi submission. Đồng thời, workload MNIST có thời gian thực thi tương đối ngắn

nên worker thường giải phóng slot nhanh trước khi hàng đợi tích lũy đủ lớn để tận dụng tối đa toàn bộ năng lực song song của hệ thống.

Kết quả này cho thấy việc đánh giá năng lực worker pool chỉ dựa trên tổng số slot là chưa đủ. Khả năng xử lý thực tế phụ thuộc vào cơ cấu slot giữa các loại công việc và tỷ lệ phân bổ submission trong workload.

5.9.3 Trả lời RQ3 - Sức chứa và khả năng mở rộng

RQ3 được đánh giá theo hai góc độ bổ sung cho nhau: số liệu đo trực tiếp từ workload thực nghiệm và phân tích độ nhạy dựa trên thời gian inference.

Ở góc độ thực nghiệm, hệ thống xử lý thành công batch 500 submission với tỷ lệ hoàn thành 100%, median queue wait 52.4 giây và P95 queue wait 105.2 giây. Kết quả này cho thấy worker pool hiện tại có thể vận hành ổn định với workload tương đương MNIST trong điều kiện khảo sát.

Ở góc độ phân tích độ nhạy, khi thời gian thực thi của final submission tăng lên 5-10 lần, throughput giảm gần tương ứng với hệ số tăng thời gian inference, trong khi queue wait tăng đáng kể. Điều này cho thấy số lượng inference slot là yếu tố quyết định sức chứa của hệ thống đối với các contest AI yêu cầu suy luận nặng.

Kết quả cũng cho thấy sức chứa không phải là một giá trị cố định mà phụ thuộc đồng thời vào cấu hình worker pool, tỷ lệ final submission trong contest và thời gian thực thi trung bình của mỗi job. Do đó, việc mở rộng hệ thống cần được xem xét trong bối cảnh workload cụ thể thay vì chỉ dựa trên số lượng worker.

Kết quả kiểm thử tải đọc API bổ sung thêm một khía cạnh đánh giá về khả năng phục vụ người dùng. Hệ thống duy trì tỷ lệ lỗi 0 tới 800 người dùng đọc đồng thời và chỉ xuất hiện khoảng 0.03% timeout ở mức 1000 người dùng ảo. Tuy nhiên, endpoint danh sách bài toán và lịch sử submission là hai điểm có độ trễ cao nhất ở tải lớn, đặc biệt sau mốc 300-500 người dùng ảo. Do đó, khi mở rộng quy mô contest, cần tối ưu song song cả hai tầng: tầng xử lý submission (bổ sung GPU worker) và tầng phục vụ request đọc (tối ưu truy vấn và cache).

5.10. Hạn chế của thực nghiệm

Mặc dù các kết quả thực nghiệm cung cấp cơ sở đánh giá thực tế cho hệ thống BKDNAIOJ, vẫn tồn tại một số giới hạn cần được xem xét khi diễn giải kết quả.

Thứ nhất, workload MNIST chưa đại diện cho toàn bộ các bài toán AI. Các bài toán được sử dụng trong thực nghiệm có dữ liệu tương đối nhỏ và thời gian suy luận

ngắn, với median inference khoảng 8 giây. Nhiều bài toán thực tế có thể yêu cầu thời gian thực thi dài hơn đáng kể, dẫn đến đặc điểm tải và mức sử dụng tài nguyên khác biệt.

Thứ hai, worker pool mang tính không đồng nhất và biến động theo thời gian. Một phần tài nguyên trong thực nghiệm đến từ Google Colab và các VPS thuê ngoài, vốn chịu ảnh hưởng bởi tải nền, chính sách phân bổ tài nguyên và điều kiện mạng tại thời điểm chạy. Do đó, các kết quả đo được phản ánh hành vi thực tế của hệ thống nhưng có thể không tái lập hoàn toàn trong các lần chạy khác.

Thứ ba, workload tính toán tổng hợp không thay thế hoàn toàn workload AI thực tế. Mặc dù sử dụng các phép tính ma trận thực thay vì cơ chế giả lập thời gian chờ, workload này vẫn chưa mô phỏng đầy đủ các chi phí thường gặp trong các mô hình học sâu quy mô lớn như tải checkpoint lớn, tiền xử lý dữ liệu phức tạp hoặc khởi tạo nhiều thành phần phụ thuộc. Vì vậy, workload tổng hợp phù hợp để đánh giá pipeline thực thi và xử lý lỗi, nhưng không nên được xem là benchmark hiệu năng GPU tổng quát.

Thứ tư, kết quả phân tích sức chứa ở Mục 5.8 là kết quả ngoại suy từ số liệu đo được chứ không phải phép đo trực tiếp. Các giá trị throughput và queue wait trong các kịch bản thời gian inference tăng $5\times$ và $10\times$ được xây dựng dựa trên giả định giữ nguyên worker pool, tỷ lệ submission và đặc điểm workload. Do đó, các kết quả này nên được hiểu như một công cụ hỗ trợ phân tích xu hướng hơn là một dự báo chính xác cho mọi kịch bản triển khai.

5.11. Tổng kết

Chương này đã trình bày kết quả thực nghiệm và đánh giá hệ thống BKDNAIOJ trên môi trường production thông qua một contest thực nghiệm được triển khai trên cùng hạ tầng với hệ thống vận hành thực tế.

Về tính đúng đắn chức năng, kết quả pilot và các batch thực nghiệm cho thấy hệ thống xử lý đầy đủ vòng đời của submission, từ khâu nộp bài, đưa vào hàng đợi, phân phối cho worker, thực thi chấm bài đến cập nhật bảng xếp hạng. Trong batch lớn nhất gồm 500 submission, hệ thống ghi nhận tỷ lệ hoàn thành 100% và không xuất hiện submission bị kẹt sau khi batch kết thúc.

Về hiệu năng xử lý, các kết quả đo cho thấy thời gian chờ hàng đợi tăng theo quy mô batch, trong khi thời gian thực thi trên worker phụ thuộc chủ yếu vào loại phase. Các

submission thuộc nhóm final có thời gian chờ và thời gian thực thi cao hơn đáng kể so với nhóm non-final do phải thực hiện thêm bước suy luận trước khi chấm điểm. Trong batch 500, hệ thống đạt thông lượng 47.8 submission/phút, median queue wait 52.4 giây và P95 queue wait 105.2 giây. Đồng thời, độ trễ cập nhật leaderboard duy trì ở mức thấp với P95 dưới 3 giây, cho thấy tăng cập nhật bảng xếp hạng không phải là nút thắt chính của hệ thống.

Kết quả phân phối job xác nhận cơ chế FIFO kết hợp kiểm tra khả năng tương thích và số slot công bố hoạt động đúng như thiết kế. Các job final được phân phối cho nhóm worker có khả năng thực thi suy luận, trong khi các job non-final được xử lý bởi nhóm worker phù hợp với loại công việc tương ứng. Điều này cho thấy hệ thống có khả năng khai thác hiệu quả một worker pool không đồng nhất về phần cứng và năng lực thực thi.

Đối với tải đọc API, hệ thống duy trì tỷ lệ lỗi bằng 0 trong các mức tải từ 50 đến 800 người dùng ảo và chỉ xuất hiện tỷ lệ lỗi rất thấp ở mức 1000 người dùng ảo. Kết quả này cho thấy API server vẫn duy trì khả năng phục vụ ổn định khi số lượng request đồng thời tăng cao, mặc dù độ trễ phản hồi tăng dần theo tải.

Cuối cùng, kết quả phân tích độ nhạy cho thấy sức chứa của hệ thống phụ thuộc mạnh vào thời gian thực thi của các job final và số lượng slot suy luận khả dụng trong worker pool. Khi thời gian inference tăng lên, thông lượng giảm và thời gian chờ hàng đợi tăng đáng kể. Do đó, đối với các bài toán AI có chi phí suy luận lớn, việc bổ sung worker có GPU hoặc tăng số slot suy luận là hướng mở rộng hiệu quả hơn so với chỉ tăng số lượng worker CPU.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đồ án đã trình bày quá trình nghiên cứu, thiết kế, cài đặt và đánh giá hệ thống **BKDNAIOJ** - một nền tảng web hỗ trợ tổ chức, chấm thi và ôn luyện cho các cuộc thi Trí tuệ nhân tạo trong môi trường học thuật. Xuất phát từ nhu cầu thực tế của Olympic Trí tuệ Nhân tạo Sinh viên Việt Nam, hệ thống được xây dựng nhằm hỗ trợ các cuộc thi AI có nhiều giai đoạn đánh giá, nhiều dạng bài nộp và yêu cầu khai thác tài nguyên tính toán phân tán.

1. Kết quả đạt được

Trước hết, đồ án đã xây dựng được một nền tảng hoàn chỉnh phục vụ tổ chức cuộc thi AI, bao gồm các chức năng chính như quản lý cuộc thi, bài toán, phase, người tham gia, nộp bài, chấm điểm tự động, cập nhật bảng xếp hạng và giám sát worker. Hệ thống hỗ trợ cả hai hình thức chấm bài phổ biến trong các cuộc thi AI: chấm trực tiếp trên file kết quả và chấm thông qua thực thi chương trình suy luận của thí sinh.

Về mô hình nghiệp vụ, đồ án đã hiện thực hóa mô hình contest bốn phase gồm **Public Test**, **Final Public**, **Private Test** và **Final Private**. Mô hình này giúp kết hợp giữa việc phản hồi nhanh cho thí sinh trong quá trình phát triển lời giải và khả năng xác minh tính tái lập của mô hình ở các giai đoạn đánh giá quan trọng.

Về kiến trúc hệ thống, BKDNAIOJ được thiết kế theo hướng phân tán, tách biệt rõ giữa các thành phần quản lý nghiệp vụ, lưu trữ dữ liệu, hàng đợi chấm bài và hệ thống worker. Cách thiết kế này giúp hệ thống xử lý tốt các file bài nộp có kích thước lớn, phân phối job đến nhiều worker khác nhau và duy trì bảng xếp hạng với độ trễ thấp.

Một kết quả quan trọng của đồ án là cơ chế phân phối job trên worker pool không đồng nhất. Hệ thống sử dụng chiến lược **Capability-aware FIFO**, trong đó job được cấp phát theo thứ tự hàng đợi nhưng vẫn kiểm tra khả năng tương thích của worker trước khi giao việc. Cơ chế này giúp tránh trường hợp final job được giao cho worker không có khả năng thực thi suy luận, đồng thời tận dụng được các worker đang rảnh trong hệ thống.

Về triển khai, hệ thống đã được hiện thực hóa thành một sản phẩm có thể vận hành thực tế. Volunteer agent được đóng gói độc lập, hỗ trợ thiết lập worker, ghi nhận thông tin phản cứng và tham gia xử lý job sau khi được quản trị viên phê duyệt. Giao diện hệ thống hỗ trợ cả thí sinh và quản trị viên, đáp ứng các nghiệp vụ chính trong quá trình tổ chức cuộc thi.

Về thực nghiệm, hệ thống được đánh giá trên môi trường production thông qua một contest clone tách biệt với dữ liệu người dùng thật. Batch lớn nhất gồm 500 submission đạt tỷ lệ hoàn thành 100%, không ghi nhận submission bị kẹt và không có lỗi phân phối sai loại worker. Trong batch này, hệ thống đạt thông lượng 47,8 submission/phút, median queue wait 52,4 giây và P95 queue wait 105,2 giây. Pipeline cập nhật leaderboard duy trì median freshness 1,4 giây và P95 freshness 2,8 giây.

Ngoài ra, kiểm thử tải đọc API cho thấy hệ thống duy trì tỷ lệ lỗi HTTP bằng 0 tới mức 800 người dùng ảo đồng thời. Ở mức 1000 người dùng ảo, hệ thống chỉ xuất hiện timeout với tỷ lệ rất thấp, khoảng 0,03%. Kết quả này cho thấy hệ thống có khả năng phục vụ đồng thời ở mức tải cao, mặc dù vẫn cần tối ưu thêm các endpoint đọc dữ liệu động khi quy mô tăng.

Tổng hợp lại, đề án đã đạt được mục tiêu xây dựng một nền tảng self-hosted hỗ trợ tổ chức và đánh giá các cuộc thi AI quy mô học thuật, có khả năng chấm bài phân tán, cập nhật bảng xếp hạng tự động và vận hành được trên hạ tầng production với số liệu thực nghiệm cụ thể.

2. Hạn chế

Mặc dù hệ thống đã hoàn thành các chức năng cốt lõi, đề án vẫn còn một số hạn chế.

Thứ nhất, quy mô thực nghiệm còn giới hạn. Worker pool trong quá trình đánh giá gồm năm worker với phần cứng không đồng nhất. Cấu hình này phù hợp để kiểm chứng hành vi của hệ thống trong môi trường thực tế, nhưng chưa đủ để kết luận cho các cuộc thi quy mô lớn hơn hoặc các workload inference kéo dài hàng chục phút.

Thứ hai, workload thực nghiệm chính dựa trên dữ liệu MNIST, có kích thước nhỏ và thời gian suy luận tương đối ngắn. Do đó, kết quả batch 500 phản ánh tốt năng lực của hệ thống với workload tương đương, nhưng chưa đại diện đầy đủ cho các bài toán AI có mô hình lớn, dữ liệu đầu vào lớn hoặc bước tiền xử lý phức tạp.

Thứ ba, trusted native mode được sử dụng cho một số worker GPU nhằm tận dụng tài nguyên trong các môi trường khó triển khai Docker GPU. Cách tiếp cận này có hiệu quả trong thực nghiệm, nhưng mức độ cô lập thấp hơn Docker sandbox và chỉ phù hợp với các worker được quản trị viên tin tưởng.

Thứ tư, tầng đọc API bắt đầu bộc lộ độ trễ cao ở một số endpoint tổng hợp dữ liệu khi tải tăng mạnh. Kết quả kiểm thử bằng K6 cho thấy các endpoint như danh sách

bài toán và lịch sử submission vẫn còn dư địa tối ưu, đặc biệt đối với dữ liệu ít thay đổi nhưng được truy cập lặp lại nhiều lần.

3. Hướng phát triển

Từ các kết quả và hạn chế trên, hệ thống có thể tiếp tục được phát triển theo một số hướng sau.

Thứ nhất, tối ưu tăng đọc API bằng cách bổ sung cache cho các dữ liệu cấu hình ít thay đổi, giảm số truy vấn phụ và tối ưu các endpoint tổng hợp dữ liệu động như danh sách bài toán, lịch sử submission và bảng xếp hạng.

Thứ hai, mở rộng cơ chế hàng đợi và lập lịch. Chiến lược Capability-aware FIFO hoạt động tốt trong workload thực nghiệm, nhưng vẫn có thể gặp head-of-line blocking khi nhiều job không tương thích nằm ở đầu hàng đợi. Các phiên bản sau có thể nghiên cứu tách queue theo loại job hoặc bổ sung index phụ để worker tìm job phù hợp nhanh hơn.

Thứ ba, tăng cường profiling cho final inference. Hệ thống có thể bổ sung cơ chế chạy thử trên tập mẫu nhỏ để ước lượng thời gian chạy, RAM, VRAM và execution path trước khi chấm chính thức. Dữ liệu này có thể phục vụ cảnh báo sớm, timeout động và lập lịch chính xác hơn.

Thứ tư, hoàn thiện sandbox GPU. Trong tương lai, hệ thống cần nghiên cứu các giải pháp cô lập GPU tốt hơn để giảm phụ thuộc vào trusted native mode, đặc biệt khi mở rộng cho các worker không hoàn toàn tin cậy.

Thứ năm, cải thiện công cụ vận hành cho ban tổ chức. Khi số lượng contest, task và worker tăng, hệ thống cần dashboard đầy đủ hơn để theo dõi queue depth, slot utilization, worker reliability, error reason và xu hướng latency theo thời gian.

Với các hướng phát triển trên, BKDNAIOJ có thể tiếp tục được hoàn thiện từ một nền tảng phục vụ cuộc thi AI quy mô học thuật thành một hệ thống ổn định hơn, có khả năng đáp ứng các cuộc thi lớn hơn, workload đa dạng hơn và yêu cầu vận hành nghiêm ngặt hơn.

TÀI LIỆU THAM KHẢO

- [1] NeurIPS Foundation, “NeurIPS Competition Track,” 2024. [Online]. Available: <https://neurips.cc/Conferences/2024/CompetitionTrack>.
- [2] A. Goldbloom, “Kaggle: The Home of Data Science,” *Kaggle Blog*, 2010. [Online]. Available: <https://www.kaggle.com/about>.
- [3] I. Guyon et al., “Design of the 2015 ChaLearn AutoML Challenge,” in *Proc. International Joint Conference on Neural Networks (IJCNN)*, Killarney, Ireland, 2015, pp. 1–8.
- [4] T. Degeling, J. Kuipers, W. Peeters, and G. Reckers, “DOMjudge: An Open Source Programming Contest Judge System,” *Olympiads in Informatics*, vol. 6, pp. 52–58, 2012.
- [5] M. Mirzayanov, “Codeforces: New Generation of Online Judges,” in *Proc. 5th International Conference on Olympiad Informatics*, 2011.
- [6] A. Pavão et al., “CodaLab Competitions: An Open Source Platform to Organize Scientific Challenges,” *Journal of Machine Learning Research*, vol. 24, no. 198, pp. 1–6, 2023.
- [7] AICrowd, “AICrowd: Collaborative AI Challenges,” 2023. [Online]. Available: <https://www.aicrowd.com/>.
- [8] D. P. Anderson, “BOINC: A System for Public-Resource Computing and Storage,” in *Proc. 5th IEEE/ACM International Workshop on Grid Computing*, Pittsburgh, PA, USA, 2004, pp. 4–10.
- [9] D. H. J. Epema, M. Livny, R. van Dantzig, X. Evers, and J. Pruyne, “A Worldwide Flock of Condors: Load Sharing Among Workstation Clusters,” *Future Generation Computer Systems*, vol. 12, no. 1, pp. 53–65, 1996.
- [10] Redis Ltd., “Introduction to Redis Streams,” *Redis Documentation*, 2023. [Online]. Available: <https://redis.io/docs/data-types/streams/>.
- [11] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman, 1979.
- [12] M. Fowler and G. Hohpe, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA, USA: Addison-Wesley, 2004.
- [13] Amazon Web Services, “Amazon S3 User Guide,” *AWS Documentation*, 2024. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/>.

-
- [14] M. Burrows, “The Chubby Lock Service for Loosely-Coupled Distributed Systems,” in *Proc. 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Seattle, WA, USA, 2006, pp. 335–350.
- [15] D. Merkel, “Docker: Lightweight Linux Containers for Consistent Development and Deployment,” *Linux Journal*, no. 239, Mar. 2014.
- [16] I. Goldberg, D. Wagner, R. Thomas, and E. A. Brewer, “A Secure Environment for Untrusted Helper Applications: Confining the Wily Hacker,” in *Proc. 6th USENIX Security Symposium*, San Jose, CA, USA, 1996.
- [17] A. Gupta and I. S. Mumick, Eds., *Materialized Views: Techniques, Implementations, and Applications*. Cambridge, MA, USA: MIT Press, 1999.
- [18] A. A. Donovan and B. W. Kernighan, *The Go Programming Language*. Boston, MA, USA: Addison-Wesley, 2016.
- [19] PostgreSQL Global Development Group, “PostgreSQL 15 Documentation,” 2023. [Online]. Available: <https://www.postgresql.org/docs/15/>.
- [20] Meta Platforms Inc., “React — A JavaScript Library for Building User Interfaces,” 2023. [Online]. Available: <https://react.dev/>.
- [21] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.